МИНОБРНАУКИ РОССИИ
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«МИРЭА – Российский технологический университет»
РТУ МИРЭА

Практическое занятие № 13-14
по дисциплине «Программирование корпоративных систем»

Тема работы: КОМПЛЕКСНАЯ УМНАЯ АВТОМАТИЗАЦИЯ МАШИННОЕ ЗРЕНИЕ И ГЕНЕРАТИВНЫЙ ИИ В RPA

Вариант: 11

Выполнил:	ЭФБО-09-23		Калинин Н.В.
	(группа)	*(подпись)*	*(Фамилия И.О.)*
Принял:	доц.		Громов Е.В.
	(должность)	*(подпись)*	*(Фамилия И.О.)*

Москва

ТЕОРЕТИЧЕСКАЯ ЧАСТЬ

Введение в предметную область

В предыдущих практических работах мы преимущественно взаимодействовали с пользовательским интерфейсом – веб-страницами, десктоп-приложениями, эмулируя действия человека. Это универсальный, но не всегда оптимальный подход. Многие бизнес-процессы требуют массовой обработки файлов, извлечения данных из документов, управления папками – и здесь классическая UI-автоматизация становится избыточной, медленной и нестабильной.

В реальных RPA-проектах робот всё чаще работает не с экраном, а напрямую с операционной системой и документами. Это открывает новые возможности, но и порождает специфические вызовы:

- Прямой доступ к файловой системе – создание, копирование, перемещение, удаление тысяч файлов без открытия проводника.
- Чтение документов различных форматов – PDF, текстовые файлы, CSV, где данные могут быть спрятаны в ячейках, таблицах или даже отсканированных страницах.
- Неструктурированность информации – документы приходят в разном виде: одни содержат чёткие таблицы, другие – хаотичный текст, из которого нужно вычленить ключевые поля.
- Ограничения и барьеры – заблокированные файлы (открыты другим процессом), защищённые паролем документы, огромные размеры, неподдерживаемые кодировки.
- Устойчивость к сбоям – отсутствие ожидаемого файла, ошибка при открытии, повреждённый PDF – робот должен не падать, а логировать проблему и продолжать работу с остальными документами.

Эти вызовы и составляют содержание нашей тринадцатой и четырнадцатой практических работ. Мы переходим от «имитации действий человека» к «системной автоматизации на уровне ОС и данных». Вы научитесь проектировать роботов, которые работают быстро, надёжно и без лишних движений мышью.

Мы изучим архитектуру взаимодействия RPA-робота с файловой системой через вызовы API, а не через графический интерфейс. Освоим базовые блоки Puzzle RPA для работы с каталогами и файлами: получение списка файлов по

маске, копирование, перемещение, удаление, создание папок. Научимся обрабатывать ошибки: проверять существование файла, перехватывать исключения, записывать проблемные файлы в отдельный лог, использовать повторные попытки. И, наконец, соберём комплексные сценарии – пакетную обработку входящих документов, формирование сводных отчётов, автоматическое архивирование.

В результате вы будете осознанно проектировать RPA-решения, способные:

- Обрабатывать тысячи документов за минуты, не открывая ни одного окна.
- Извлекать структурированные данные из хаотичных входящих файлов и приводить их к единому формату.
- Устойчиво работать при отсутствии файлов, неверных путях, повреждённых документах – с полным логированием и восстановлением.
- Автоматизировать цепочки «пришёл документ → извлекли данные → записали в базу → переместили в архив» без участия человека.

Вы перейдёте от уровня «робота, который кликает по кнопкам» к уровню «разработчика, создающего высокопроизводительные, отказоустойчивые и масштабируемые файловые RPA-решения».

Зачем роботу работа с ОС и документами

На первых этапах обучения RPA мы привыкли, что робот «смотрит» на экран, как человек: находит кнопки, поля ввода, выпадающие списки, кликает, печатает, скроллит. Такой подход называется UI-автоматизацией (User Interface automation). Он универсален: робот может работать практически с любым прило-жением, которое видит пользователь.

Однако у UI-автоматизации есть серьёзные ограничения:

- Низкая скорость – каждое действие (наведение, клик, ожидание появления элемента) занимает секунды.
- Нестабильность – малейшее изменение интерфейса (сместилась кнопка, изменился ID, появился баннер) ломает сценарий.
- Зависимость от окружения – робот требует активного рабочего стола, запрещённой блокировки экрана, корректного разрешения монитора.
- Избыточность – если нужно обработать 1000 Excel-файлов, открывать каждый через графический интерфейс Excel – катастрофически медленно.

Именно поэтому в промышленной RPA значительная часть задач решает-ся без эмуляции действий человека – через прямое обращение к операционной системе и файлам.

Работа с операционной системой – это выполнение файловых операций напрямую через вызовы API Windows/Linux:

- создание, удаление, перемещение, копирование папок и файлов;
- проверка существования пути, получение списка файлов по маске (например, все *.txt в папке);
- работа с атрибутами (только чтение, скрытый, системный);
- архивация (ZIP, RAR);
- запуск программ, управление процессами.

Работа с документами – это извлечение данных из файлов, созданных офисными приложениями, без открытия их графического интерфейса:

- парсинг PDF (текстовый слой или OCR для сканов);
- обработка Word, текстовых файлов (TXT, CSV), XML, JSON;
- запись структурированных данных в Excel, CSV, базы данных (через ODBC).

Рассмотрим реальные бизнес-процессы, которые невозможно эффективно реализовать через UI-автоматизацию.

Пример 1. Обработка входящих счетов.

Каждый день на почту приходят 200 PDF-счетов от поставщиков. Робот должен скачать их, извлечь номер счёта, дату, сумму, поставщика, записать в ERP-систему.

- Через UI: открыть каждый PDF в Adobe Reader (5 секунд), выделить текст (2 секунды), скопировать (1 секунда), переключиться в ERP, вставить, сохра-нить… 200×15 секунд = 50 минут только на открытие. Плюс постоянные сбои из-за всплывающих окон Adobe.
- Через прямое чтение PDF: робот за 2 минуты пройдёт по всем файлам, извлечёт текст через специальную библиотеку и запишет в ERP через API. Ни одного открытого окна.

Пример 2. Формирование сводных отчётов.

Менеджеры скидывают в общую папку 50 Excel-файлов с продажами по регионам. Надо собрать все данные в единую таблицу.

- Через UI: открыть Excel, скопировать диапазон, переключиться в сводный файл, вставить… риск ошибиться диапазоном, потерять фокус.
- Через прямое чтение: робот одной командой прочитает все файлы, сложит данные в массив и запишет итог. Скорость – сотни файлов в минуту.

После изучения этой практической работы и выполнения заданий вы сможете:

- Создавать роботов, которые полностью обрабатывают входящие документы без участия человека – читают, извлекают данные, классифицируют, перемещают, архивируют.
- Проектировать отказоустойчивые файловые сценарии – робот не упадёт, если файла нет или он повреждён, а запишет ошибку в лог и продолжит работу.
- Извлекать структурированные данные из PDF, текстовых файлов напрямую, без запуска офисных приложений.
- Строить пакетные обработчики для тысяч документов с формированием сводных отчётов.
- Различать, когда эффективнее использовать UI-автоматизацию, а когда – прямые вызовы ОС и чтение документов.

Работа с ОС и документами не отменяет классическую UI-автоматизацию. Во многих сценариях они комбинируются:

- Робот через UI заходит в CRM, нажимает «Выгрузить отчёт» → сохраняет фаил на диск.

- Затем через прямое чтение обрабатывает этот фаил, извлекает данные.
- Результат через UI вводит обратно в CRM или отправляет письмо через SMTP (без открытия Outlook).

Грамотный RPA-разработчик владеет обоими подходами и выбирает оптимальный под каждую задачу.

Отличие от классического UI-взаимодействия

При классическом UI-подходе, чтобы скопировать файл, робот вынужден открыть проводник, найти нужный файл, нажать комбинацию клавиш «Копировать», перейти в другую папку и нажать «Вставить». Всё это требует наличия видимого рабочего стола, мыши и клавиатуры. При работе напрямую с операционной системой робот вызывает одну встроенную команду копирования, которая исполняется на уровне ядра системы за доли секунды, и при этом никакие окна не открываются.

Аналогично обстоит дело с проверкой существования файла. UI-робот должен попытаться найти иконку файла на рабочем столе или в окне проводника, что ненадёжно и медленно. Робот, использующий системные вызовы, просто запрашивает у файловой системы информацию о существовании файла по указанному пути – это мгновенная операция, не зависящая от графического интерфейса.

UI-робот предъявляет жёсткие требования к окружению. Ему обязательно нужен активный рабочий стол. Сессия пользователя не должна быть заблокирована или свёрнута, иначе робот не сможет «увидеть» окна. Разрешение экрана, масштаб интерфейса, тема оформления – всё это должно быть строго определено, потому что координаты кликов и селекторы элементов привязаны к пикселям и свойствам отображения. Любое всплывающее окно, уведомление, баннер или системный диалог могут перекрыть нужную кнопку и нарушить выполнение сценария. Кроме того, UI-робот требует, чтобы на машине были установлены и лицензированы все приложения, с которыми он работает (текстовые редакторы, почтовые клиенты, браузеры).

Робот, работающий на уровне операционной системы и документов, не нуждается в активном рабочем столе. Ему достаточно прав на чтение и запись в нужных каталогах, а также доступа к сетевым папкам, если файлы лежат на сервере. Он может выполняться на заблокированной виртуальной машине, в контейнере или даже на сервере без графической подсистемы. Ему не нужны установленные офисные приложения – все необходимые библиотеки для чтения документов уже встроены в Puzzle RPA. Это делает такого робота гораздо более гибким и пригодным для промышленного использования.

Разница в скорости между UI-подходом и прямым системным взаимодействием огромна. Когда робот открывает документ через графический интерфейс приложения, происходит запуск процесса этого приложения, загрузка файла в оперативную память, отрисовка окна, инициализация всех элементов интерфейса – это занимает несколько секунд даже на быстром компьютере. Затем робот тра-тит время на поиск нужного элемента, клик, чтение данных. При прямом чтении

документа через специализированную библиотеку файл открывается в памяти без отображения на экране, и робот получает доступ к данным практически мгновенно – за сотые доли секунды. Разница может составлять от десяти до пятидесяти раз. При обработке сотен или тысяч документов эта разница превращается в часы против минут.

UI-робот очень чувствителен к любым изменениям. Обновилась программа – кнопка переместилась на два пикселя или изменила свой идентификатор – и робот перестаёт её находить. Пользователь изменил разрешение экрана на мониторе – координаты кликов становятся неверными. Появилось неожиданное всплываю-щее окно с предложением обновить программу или принять cookies – и робот за-висает, ожидая привычную кнопку, которая теперь скрыта. Немного увеличилась задержка в работе интернета или приложения – элемент появился на полсекунды позже, и робот уже прошёл мимо. Всё это делает UI-автоматизацию хрупкой и требующей постоянного сопровождения.

Робот, работающий напрямую с операционной системой и документами, ломается только в строго определённых случаях. Это может быть изменение пути к файлам или папкам, изменение формата документов (например, если структура текстового файла стала другой), пропадание прав доступа или недоступность сетевого диска. Пока эти условия стабильны, робот будет работать без сбоев года-ми, потому что он не зависит от пикселей, цветов, анимаций и всплывающих окон. Системный подход значительно устойчивее к хаотичным изменениям ин-терфейса, но требует, чтобы структура данных и расположение файлов были за-ранее известны и стабильны.

UI-робот практически не масштабируется. Он может выполняться только на одной виртуальной машине в каждый момент времени (или на нескольких, но с отдельными лицензиями на каждую). При увеличении объёма данных время работы растёт линейно, и ускорить процесс невозможно – робот всё равно будет открывать каждый документ через графический интерфейс. Более того, на одной машине нельзя запустить несколько копий UI-робота одновременно, потому что они будут конфликтовать за мышь, клавиатуру и окна.

Робот, работающий на уровне операционной системы, открывает совершенно иные возможности. Он может обрабатывать файлы в многопоточном режиме – одновременно обрабатывать несколько документов на разных ядрах процессора. Его можно легко развернуть в контейнере Docker без графической среды, что позволяет быстро создавать десятки экземпляров для параллельной обработки. Если нужно обработать сто тысяч файлов, можно запустить десять копий робота на десяти серверах, каждая из которых работает со своей частью общей папки. Такой подход масштабируется горизонтально и позволяет обрабатывать данные практически любого объёма.

Классическое UI-взаимодействие незаменимо в ситуациях, когда у приложения нет программного интерфейса (API) и оно не предоставляет доступа к данным иначе как через экран. Например, авторизация в веб-приложении, клик по кнопке

в старой учётной системе, заполнение формы в программе, которая не умеет читать файлы напрямую – здесь без UI не обойтись.

Прямая работа с операционной системой и документами предпочтительна во всех случаях, где данные уже лежат в файлах или их можно выгрузить в файлы. Создание, переименование, перемещение, удаление тысяч файлов и папок, проверка существования путей, получение списка файлов по маске, чтение текстовых документов, работа с вложениями и архивами – всё это делается в разы быстрее и надёжнее без участия графического интерфейса. Более того, многие задачи (например, массовое переименование ста тысяч файлов по сложному правилу) через UI просто нереализуемы за разумное время.

UI-взаимодействие и системная работа с операционной системой и документами – это два инструмента в одном арсенале профессионала. Прямая работа с файловой системой и документами быстрее, стабильнее, масштабируемее и не требует активного рабочего стола. Грамотный разработчик всегда оценивает возможность не-UI решения до того, как писать сложные селекторы и ожидания. В следующих разделах мы разберём, как именно робот взаимодействует с файловой системой «под капотом» – архитектуру этого взаимодействия, и затем освоим конкретные блоки Puzzle RPA для работы с документами.

Архитектура: как робот взаимодействует с файловой системой

Чтобы понять, как робот работает с файловой системой, нужно представить себе несколько уровней, которые отделяют понятный разработчику блок в Puzzle RPA от реальных операций на диске. На верхнем уровне находится студия разработки, где разработчик перетаскивает визуальные блоки, такие как «Скопировать файл», «Получить список файлов» или «Создать папку». Эти блоки являются удобной обёрткой, скрывающей всю сложность нижележащих уровней. Когда разработчик собирает схему робота, студия преобразует его визуальные инструкции в программный код. В случае Puzzle RPA этим кодом является Python – мощный язык, который имеет богатые встроенные библиотеки для взаимодействия с операционной системой.

Архитектура Puzzle RPA включает несколько ключевых компонентов, каждый из которых отвечает за свою часть работы. Студия разработки представляет собой визуальный конструктор, где разработчик создаёт сценарии роботов. Оркестратор обеспечивает централизованное управление, запуск и мониторинг роботов, позволяя развёртывать их на множестве машин. Агент – это легковесный компонент, который устанавливается на каждую виртуальную или физическую машину, где должны выполняться роботы. Именно агент непосредственно запускает робота и исполняет его инструкции. Когда роботу требуется выполнить операцию с файловой системой, агент обращается к операционной системе через её программные интерфейсы. В операционной системе эти вызовы преобразуются в команды для драйверов файловой системы, которые, в свою очередь, взаимодействуют с физическим или виртуальным диском. Все эти уровни скрыты от разработчика, но понимание их существования помогает осознать, почему одни опера-

ции выполняются быстро, а другие могут зависеть от множества внешних факторов.

Под капотом Puzzle RPA все блоки работы с файловой системой реализованы с использованием стандартных библиотек Python. Python предоставляет несколько мощных модулей для взаимодействия с операционной системой.

Когда разработчик в Puzzle RPA настраивает блок «Переместить файл», указывая исходный и целевой пути, студия генерирует код на Python, который внут-ри себя использует, например, функцию shutil.move(). Этот код затем исполняется агентом. Таким образом, робот не открывает проводник Windows и не эмулирует перетаскивание файла мышью – он просто вызывает встроенную функцию, которая мгновенно выполняет операцию на уровне файловой системы. Именно поэтому такие операции происходят в десятки раз быстрее, чем если бы их выполнял человек или UI-робот.

Важнейший аспект архитектуры взаимодействия робота с файловой системой – это контекст, в котором работает робот, и права доступа, которыми он обладает. Агент Puzzle RPA запускается от имени определённой учётной записи пользователя на целевой машине. Чаще всего это выделенная служебная учётная запись, которая имеет доступ только к тем папкам и файлам, которые необходимы для выполнения сценариев. Именно от прав этой учётной записи зависит, сможет ли робот прочитать файл, создать папку или удалить документ.

Например, если робот пытается записать файл в папку C:\Program Files, но учётная запись, от которой он запущен, не имеет на это прав, операционная система вернёт ошибку «Отказано в доступе». Puzzle RPA транслирует эту ошибку в понятное сообщение, которое разработчик может увидеть в логах и обработать с помощью блоков обработки исключений. Поэтому при проектировании роботов, работающих с файловой системой, всегда нужно учитывать, от какой учётной записи будет запускаться робот, и заранее предоставлять этой записи необходимые разрешения на целевые каталоги.

Большинство операций с файловой системой являются синхронными. Это означает, что когда робот вызывает команду скопировать файл, его выполнение останавливается в ожидании завершения этой операции. Операционная система сообщает агенту, что копирование закончено, и только после этого робот переходит к следующему блоку. Для небольших файлов это происходит практически мгновенно, и разработчик может не задумываться о синхронности. Однако при работе с очень большими файлами (например, копирование архива размером несколько гигабайт) операция может занять заметное время. В таких случаях робот будет «висеть» на этом блоке до полного завершения копирования, и это нормальное поведение.

Понимание синхронности важно для правильного проектирования сценариев. Если после копирования файла робот сразу же пытается его открыть для чтения, копирование к этому моменту уже гарантированно завершено, и файл доступен. Разработчику не нужно вручную добавлять задержки или проверки готовности файла – система уже обеспечила последовательность операций.

Когда робот работает с файлами на сетевом диске или в облачном хранилище (например, ownCloud, OneDrive, корпоративная файловая шара), архитектура взаимодействия усложняется. В этом случае робот обращается к сетевому ресурсу через протоколы, поддерживаемые операционной системой (например, SMB для Windows-шары или WebDAV для ownCloud). Операционная система выступает посредником: она принимает запрос от робота, устанавливает соединение с удалённым сервером, выполняет операцию и возвращает результат. Если сетевое соединение нестабильно или сервер недоступен, робот может получить ошибку таймаута или отказа в доступе.

При работе в виртуальных средах, таких как VDI или удалённые рабочие столы, важно помнить, что файловая система, с которой работает робот, – это файловая система самой виртуальной машины, а не локального компьютера пользователя. Если роботу нужно получить доступ к файлам на компьютере пользователя, эти файлы должны быть предварительно скопированы на виртуальную машину или предоставлены через сетевую папку.

Архитектура взаимодействия робота Puzzle RPA с файловой системой представляет собой многоуровневую систему, где визуальные блоки разработчика транслируются в код на Python, который через системные вызовы обращается к операционной системе. Ключевыми компонентами являются студия разработки, оркестратор и агент, который непосредственно исполняет код на целевой машине. Права доступа к файлам и папкам определяются учётной записью, от которой запущен агент. Понимание этой архитектуры помогает разработчику правильно проектировать сценарии, предвидеть возможные проблемы с доступом и эффективно использовать возможности файловой системы, не прибегая к медленной и нестабильной UI-автоматизации.

Документ как источник данных

Для человека документ – это привычный носитель информации. Мы открываем Excel, видим аккуратные строки и столбцы, понимаем, где шапка таблицы, где итоги, где примечания. Открываем PDF-счёт – видим дату, номер, сумму, поставщика. Человеческий мозг мгновенно выделяет структуру из визуального представления. Для робота же документ – это всего лишь последовательность байтов на диске. Чтобы извлечь из него осмысленные данные, роботу нужны чёткие правила: где именно лежит номер счёта, в какой ячейке сумма, как отделить заголовок от деталей.

Проблема усугубляется тем, что в реальном бизнесе документы редко бывают идеально однотипными. Менеджеры добавляют лишние строки, меняют формат даты, вставляют комментарии, объединяют ячейки, вкладывают отсканированные изображения вместо текста. Один и тот же тип документа от разных контрагентов может выглядеть совершенно по-разному. Это превращает задачу автоматического извлечения данных из документов в настоящее искусство балансирования между жёсткостью правил (чтобы не ошибиться) и гибкостью (чтобы не пропустить полезные данные).

В контексте RPA документ рассматривается не как файл для открытия и просмотра, а как контейнер, из которого нужно извлечь конкретные поля и значения. Робот не «читает» документ как человек – он парсит его структуру. Для каждого формата документа существуют свои методы извлечения данных.

Текстовые файлы (TXT, CSV, лог-файлы) – это самый простой случай. Данные в них уже представлены в виде последовательности символов. Роботу нужно лишь правильно определить кодировку (UTF-8, Windows-1251, KOI8-R), чтобы не получить «кракозябры», а затем с помощью поиска подстрок или регулярных выражений найти нужные фрагменты. CSV-файлы дополнительно имеют разделители (чаще всего запятая или точка с запятой) – их можно разобрать на строки и столбцы.

PDF-файлы – самый сложный случай. PDF бывает двух типов. Первый тип – текстовый PDF, где текст сохраняется как символы. Из такого файла робот может извлечь текст построчно, хотя порядок строк может не соответствовать визуальному восприятию. Второй тип – отсканированный PDF, где страница является просто картинкой. Здесь уже требуются технологии оптического распознавания символов (OCR), которые превращают изображение в текст. Это медленнее, ме-нее точно и требует дополнительных вычислительных ресурсов.

Word-файлы (DOCX) сочетают свойства текстовых и форматированных документов. Из них можно извлекать текст, игнорируя форматирование, или пытаться сохранить структуру абзацев и таблиц.

Независимо от формата документа, процесс превращения его в структурированные данные, понятные роботу, проходит через несколько обязательных этапов.

Первый этап – обнаружение документа. Робот должен знать, где искать файлы. Это может быть конкретная папка на локальном диске, сетевая директория, входящие вложения в почтовом ящике или результат выгрузки из какой-либо системы. Обычно этот этап реализуется через блок «Получить список файлов» с фильтром по маске (например, «все файлы *.pdf в папке C:\Invoices»).

Второй этап – открытие и чтение документа. Робот с помощью соответствующего блока («Прочитать из PDF», «Прочитать текстовый файл») загружает содержимое документа в память. Важно понимать, что робот не открывает приложение – он использует встроенную библиотеку, которая напрямую разбирает внутреннюю структуру файла. Это быстро и не требует установленного офисного пакета.

Третий этап – извлечение нужных полей. Здесь разработчик должен чётко указать, где именно в документе находятся требуемые данные. Поиск по заголовку («найти столбец с заголовком "Сумма" и взять значение из первой строки»). Для PDF это чаще всего поиск по ключевым словам («найти строку, содержащую "Итого:", и взять число после неё») или регулярные выражения. Для текстовых файлов – позиция в строке, разделители или опять же регулярные выражения.

Четвёртый этап – приведение к единому формату. Извлечённые данные часто приходят в «сыром» виде. Сумма может содержать пробелы, лишние символы валюты или неверную десятичную точку. Дата может быть в формате «10.12.2024»,

«2024-12-10» или «Dec 10, 2024». Робот должен нормализовать эти значения: удалить лишние символы, преобразовать дату в стандартный формат, привести числа к типу float или integer. Без этого этапа дальнейшая обработка (например, запись в базу данных) может завершиться ошибкой.

Пятый этап – логирование и обработка ошибок. Не все документы можно успешно обработать. Файл может быть повреждён, защищён паролем, содержать неожиданную структуру или вовсе оказаться пустым. Робот должен распознать такие ситуации, записать в лог имя проблемного файла и причину ошибки, после чего продолжить обработку остальных документов. Остановка всего процесса из-за одного «плохого» файла – грубая ошибка проектирования.

Самая большая головная боль при работе с документами – это их изменчивость. В идеальном мире все счета-фактуры от одного поставщика имели бы одинаковое расположение полей. В реальности бухгалтер мог добавить лишнюю строку, и тогда сумма съедет на ячейку ниже. Менеджер мог вставить примечание перед таблицей, и тогда нужные данные окажутся не на третьей строке, а на пя-той. Контрагент прислал счёт в новой версии бланка – и привычные ключевые слова изменились.

Существует два подхода к борьбе с этой проблемой. Первый – жёсткая привязка к координатам или ключевым словам, которая работает только при неизменном шаблоне. Этот подход прост в реализации, но хрупок. Второй – использование правил, основанных на поиске и относительном позиционировании. Например: «найти строку, содержащую слово "Итого", затем взять число из следующей строки» или «найти столбец с заголовком "Сумма НДС" и взять значение из той же строки, где в столбце "№ п/п" стоит последнее значение». Такие правила сложнее писать, но они значительно устойчивее к небольшим изменениям структуры.

В промышленной RPA часто применяют комбинированный подход: сначала пытаются извлечь данные по гибким правилам, а в случае неудачи – по жёсткому шаблону, а если и это не помогает – отправляют документ на ручную обработку с уведомлением оператора.

Документ в RPA – это не просто файл, а источник данных, который требует программного извлечения, очистки и структурирования. Процесс превращения хаотичного набора байтов в упорядоченные поля проходит этапы обнаружения, чтения, извлечения, нормализации и обработки ошибок. Главная сложность – нестабильность структуры документов в реальном бизнесе, что вынуждает разработчика проектировать гибкие, устойчивые правила извлечения данных, а также предусматривать механизмы ручного вмешательства в сложных случаях. Puzzle RPA предоставляет необходимые блоки для работы с PDF и текстовыми файлами, но успех решения зависит от грамотного применения этих инструментов и учёта особенностей каждого формата. В следующих разделах мы рассмотрим конкретные проблемы и ограничения при работе с документами, а затем перейдём к практическим заданиям.

Проблемы и ограничения при работе с документами

На первый взгляд может показаться, что работа с документами – это просто: взял файл, прочитал данные, записал результат. Однако в реальных бизнес-процессах разработчик RPA сталкивается с множеством проблем, которые могут превратить простую задачу в долгий и болезненный процесс отладки. Документы, которые создают люди, редко соответствуют строгим ожиданиям робота. Люди добавляют лишние строки, оставляют пустые ячейки, используют разные форматы дат, вставляют комментарии куда попало, сохраняют файлы в устаревших версиях форматов. Робот же, в отличие от человека, не умеет «догадываться» – ему нужны чёткие, однозначные правила. Именно на стыке человеческой небрежно-сти и роботизированной педантичности возникают основные проблемы при автоматической обработке документов.

Прежде чем робот вообще сможет начать работать с документом, он должен получить к нему доступ. И здесь возникает целый класс проблем. Файл может быть открыт другим пользователем или процессом. Например, менеджер открыл Excel-отчёт и ушёл на обед, забыв его закрыть. Робот пытается прочитать этот файл, но операционная система блокирует доступ, потому что файл уже исполь-зуется другим приложением. В результате робот получает ошибку «Отказано в доступе» или «Файл занят другим процессом».

Другая проблема – права доступа. Робот обычно запускается от имени служебной учётной записи, которая может не иметь прав на чтение определённых папок или файлов. Особенно часто это встречается в корпоративных сетях, где папки с финансовыми документами защищены строгими политиками безопасности. Разработчик может ошибочно указать путь к сетевой папке, не проверив, имеет ли учётная запись робота доступ к этому ресурсу. В результате робот либо не найдёт файл, либо не сможет его открыть, либо не сможет сохранить обработанный результат.

Третья проблема – некорректные пути. Путь к файлу может быть слишком длинным (в Windows ограничение 260 символов, хотя современные версии могут работать с более длинными путями при определённых настройках). Путь может содержать недопустимые символы, пробелы без кавычек, кириллицу, которая неверно интерпретируется в разных кодировках. Сетевой путь может стать временно недоступным из-за проблем с сетевым соединением или перезагрузки файлового сервера.

Когда файл успешно открыт, начинаются проблемы, связанные с его внутренним устройством. Для текстовых файлов главная проблема – кодировка. Текстовый файл – это просто последовательность байтов. Чтобы превратить байты в символы, нужна таблица соответствия – кодировка. Самые распространённые кодировки в русскоязычной среде – UTF-8, Windows-1251, KOI8-R. Если робот откроет файл в кодировке Windows-1251, ожидая UTF-8, он увидит бессмысленный набор символов, который в народе называют «кракозябрами». Разработчик дол-жен либо заранее знать, в какой кодировке приходят файлы, либо научить робота

определять кодировку автоматически (что не всегда возможно на сто процентов надёжно).

PDF-файлы – чемпионы по количеству проблем. Текстовый PDF не гарантирует, что текст будет извлечён в том порядке, в котором его видит человек. Библиотеки чтения PDF могут возвращать текст в порядке, определяемом внутренней структурой документа, который может не совпадать с визуальным порядком чтения. Например, текст в колонках может быть извлечён как первая колонка целиком, потом вторая – хотя человек читает построчно слева направо. Это делает практически невозможным надёжное извлечение табличных данных из многостраничных PDF без специальных алгоритмов.

Отсканированные PDF добавляют ещё один уровень сложности – распознавание текста. OCR (оптическое распознавание символов) никогда не бывает идеальным. Качество распознавания зависит от чёткости сканирования, шрифта, языка документа, наличия водяных знаков, печатей и других артефактов. Цифра «0» может быть распознана как буква «О», единица как палочка, пробелы могут исчезать или появляться там, где их нет. Даже современные OCR-движки с поддержкой нейросетей дают ошибку в несколько процентов, что при обработке тысяч документов означает десятки файлов, которые потребуют ручной проверки.

Даже если файл открылся и текст извлёкся корректно, остаётся главная проблема: где именно в этом тексте лежат нужные данные. Человек видит документ и интуитивно понимает, где номер счёта, где дата, где сумма. Роботу же нужно чёткое правило. Но реальные документы нестабильны. В одном счёте номер может быть в верхнем левом углу, в другом – в правом. В одном документе дата написана «10.12.2024», в другом – «December 10, 2024», в третьем – «2024-12-10». Сум-ма может быть с пробелом между тысячами, с символом валюты перед или после числа, с десятичной запятой вместо точки.

Разработчик может попытаться использовать поиск по ключевым словам: найти слово «Итого», а затем взять число из следующей строки. Но что, если в документе есть несколько слов «Итого» (промежуточный итог, общий итог, итог по НДС)? Что, если между словом «Итого» и числом есть лишний перевод стро-ки? Что, если слово написано с опечаткой – «Итог»? Человек поймёт, робот – нет.

Ещё одна проблема – пустые или повреждённые файлы. Пользователь мог создать документ, ничего в него не записать и отправить. Или файл мог повредиться при передаче по почте. Или документ защищён паролем, и робот не может его открыть. В таких случаях попытка чтения приведёт к ошибке, и робот должен быть готов к этому.

Работа с документами – не бесплатная операция. Робот загружает файл в оперативную память, чтобы с ним работать. Если документ очень большой (например, PDF с тысячей страниц), его загрузка может занять много времени и потребовать много памяти. В виртуальной среде с ограниченными ресурсами это может привести к замедлению работы всей системы или даже к падению робота с ошибкой нехватки памяти.

Особенно требовательны к ресурсам операции OCR для отсканированных PDF. Каждая страница должна быть обработана, превращена в изображение, проанализирована нейросетевой моделью. Это занимает секунды на страницу, а то и десятки секунд для сложных страниц. Если робот должен обработать 500-страничный отсканированный договор, время выполнения может измеряться часами.

Пакетная обработка тысяч файлов тоже требует внимания к производительности. Робот, который открывает каждый файл, читает его, закрывает и переходит к следующему, работает последовательно. Если файлов много, общее время мо-жет стать неприемлемым. Здесь возникает потребность в многопоточности – одновременной обработке нескольких файлов на разных ядрах процессора. Однако многопоточность усложняет код и требует осторожного обращения с общими ресурсами (например, с одним и тем же выходным Excel-файлом, куда записываются результаты).

Перечисленные проблемы не означают, что автоматизация работы с документами невозможна. Они означают, что разработчик должен проектировать робота с учётом этих проблем. Вот несколько ключевых стратегий.

Первая стратегия – валидация входных данных. Перед тем как пытаться читать документ, робот должен проверить, существует ли файл, не пуст ли он, не заблокирован ли другим процессом. Если файл защищён паролем, возможно, стоит пропустить его и записать в лог, а не пытаться взломать.

Вторая стратегия – гибкие правила извлечения данных. Вместо жёсткой привязки к координатам стоит использовать поиск по ключевым словам, регулярные выражения, относительное позиционирование. Например: «найти строку, содержащую слово "Сумма", затем взять первое число в этой строке». Такие правила сложнее писать, но они выдерживают небольшие изменения структуры документа.

Третья стратегия – нормализация данных. После извлечения сырых значений их нужно очистить: удалить лишние пробелы, заменить запятую на точку, преобразовать дату в единый формат. Это резко снижает вероятность ошибок при дальнейшей обработке.

Четвёртая стратегия – обработка исключений и fallback-механизмы. Если робот не может извлечь данные из документа (например, структура оказалась неожиданной), он не должен падать. Он должен записать ошибку в лог, возможно, отправить проблемный файл в отдельную папку для ручной обработки и продолжить работу со следующим документом.

Пятая стратегия – логирование и мониторинг. Робот должен записывать, какие файлы были обработаны успешно, какие с ошибками, и по какой причине произошла ошибка. Это позволяет оператору быстро находить проблемные документы и исправлять их или настраивать правила извлечения.

Работа с документами в RPA сопряжена с множеством проблем: от банального отсутствия прав доступа до сложностей с кодировками, форматами и нестабильной структурой документов. Ограничения производительности и возможно-

сти платформы также накладывают свои рамки. Однако все эти проблемы решаемы при правильном проектировании. Разработчик должен предусматривать валидацию файлов, использовать гибкие правила извлечения данных, нормализовать результаты, обрабатывать исключения и вести подробное логирование. В следующем разделе мы рассмотрим, как сделать робота устойчивым к этим проблемам – тему обработки ошибок при работе с файловой системой.

Устойчивость робота: обработка ошибок при работе с файловой системой

Один из важнейших принципов устойчивого робота – это валидация входных условий перед выполнением критической операции. Вместо того чтобы просто попытаться прочитать файл и надеяться на успех, робот сначала проверяет, существует ли файл по указанному пути. Если файла нет – нет смысла пытаться его читать, сразу можно перейти к обработке ошибки.

Также перед чтением файла полезно проверить, не пуст ли он. Для этого можно использовать блок «Получить размер файла» и сравнить с нулём. Пустой файл не содержит полезных данных, и попытка его обработки заведомо бессмысленна.

Если робот работает с сетевыми папками, стоит добавить проверку доступности сетевого ресурса. Например, можно попытаться получить список файлов в корне папки; если операция завершилась ошибкой, вероятно, сетевой диск отключён или у робота нет прав.

Особенно важна предварительная проверка при операциях удаления и перезаписи. Прежде чем удалить файл, убедитесь, что он существует и не является тем самым единственным экземпляром важных данных. Прежде чем перезаписать существующий файл, подумайте, не нужно ли сначала создать резервную копию.

Некоторые ошибки носят временный характер. Например, файл может быть временно заблокирован антивирусом, который его сканирует. Сетевой диск может быть недоступен секунду из-за перегрузки сети. В таких случаях имеет смысл не сразу считать операцию проваленной, а повторить её через небольшую паузу несколько раз.

Особенно полезны повторные попытки при работе с сетевыми ресурсами и при обработке вложений из почты, где письмо может быть ещё не до конца за-гружено на сервер. Важно только не переусердствовать с количеством попыток и длительностью пауз, чтобы робот не зависал на десятки минут из-за одного проблемного файла.

Устойчивый робот не только не падает, но и оставляет после себя понятный след о том, что пошло не так. Логирование ошибок – это не роскошь, а необходимость. Когда через неделю после запуска робота обнаружится, что часть документов не обработалась, без логов вы никогда не поймёте, какие именно файлы вызвали проблему и по какой причине.

Хорошей практикой является создание отдельного файла-лога ошибок в формате CSV, куда робот дописывает строки с информацией о проблемных документах. Такой файл потом можно открыть в Excel и быстро проанализировать, ка-

кие файлы не обработались и почему. Например, можно вести колонки: «Имя файла», «Дата и время ошибки», «Код ошибки», «Текст ошибки», «Действие робота». Это значительно упрощает сопровождение робота.

Очень полезная архитектурная практика – организовать файловое хранилище так, чтобы робот мог изолировать проблемные документы, не смешивая их с уже обработанными и с теми, что ещё ждут обработки.

Стандартная схема включает три папки. Папка «Входящие» (или «Incoming») – сюда складируются документы, ожидающие обработки. После того как робот успешно обработал файл, он перемещает его в папку «Успех» (или «Processed»). Если при обработке файла произошла ошибка, и робот не смог её преодолеть да-же после повторных попыток, файл перемещается в папку «Ошибки» (или «Errors»).

Такая схема даёт несколько преимуществ. Во-первых, папка «Входящие» никогда не засоряется уже обработанными файлами. Во-вторых, оператор в любой момент может зайти в папку «Ошибки» и посмотреть, какие документы робот не смог обработать, разобраться в причинах и, возможно, обработать их вручную или скорректировать настройки робота. В-третьих, если робот запускается по расписанию, он каждый раз видит только новые, ещё не обработанные файлы.

В папку «Ошибки» вместе с файлом можно также сохранять текстовый файл с пояснением причины ошибки. Например, если робот не смог прочитать PDF из-за пароля, он может создать рядом с файлом файл с расширением .error, в котором напишет: «Документ защищён паролем. Автоматическая обработка невозможна». Это очень помогает при ручном разборе.

Устойчивость робота при работе с файловой системой достигается сочетанием нескольких техник: предварительной проверки условий (существует ли файл, не пуст ли он), использования блока «Обработка ошибок» для перехвата исклю-чений, механизма повторных попыток для временных сбоев, подробного логиро-вания всех ошибок с контекстом, изоляции проблемных файлов в отдельные пап-ки. Проектирование устойчивого робота требует от разработчика предусмотрительности и понимания, что ошибки неизбежны, но они не должны приводить к полной остановке процесса. В следующем разделе мы перейдём к практическим блокам Puzzle RPA и выполним два практических задания, где применим все рассмотренные принципы.

Основные блоки Puzzle RPA для работы с ОС и документами

В предыдущих разделах мы подробно разобрали теоретические основы работы робота с операционной системой и документами: зачем это нужно, чем отличается от классической UI-автоматизации, как устроена архитектура взаимодействия с файловой системой, с какими проблемами можно столкнуться и как сделать робота устойчивым к ошибкам. Это необходимый фундамент, который позволяет понять логику и ограничения такого подхода.

Однако реальные бизнес-задачи редко решаются одной лишь теорией. На практике роботу приходится ежедневно обрабатывать сотни входящих докумен-

тов, перебирать файлы в папках, извлекать данные. Для автоматизации этих действий требуются конкретные визуальные инструменты, которые предоставляет платформа Puzzle RPA.

Сейчас мы переходим к изучению основных блоков Puzzle RPA для работы с операционной системой и документами. Цель этой практической работы – осво-ить на практике блоки, которые позволяют роботу напрямую взаимодействовать с файловой системой, читать и записывать документы различных форматов, а также обрабатывать возникающие ошибки без остановки процесса.

В результате освоения этих блоков вы сможете создавать не просто роботов, которые эмулируют действия человека перед экраном, а промышленные RPA-решения, способные в фоновом режиме за минуты обрабатывать тысячи документов, корректно реагировать на сбои и работать без участия оператора.

Приступаем к изучению основных блоков Puzzle RPA для работы с операционной системой и документами.

Блок «Дописать в файл»



Блок предназначен для добавления текстовой информации в конец существующего файла. В отличие от блока «Записать в файл», который перезаписывает содержимое файла с нуля, блок «Дописать в файл» сохраняет всё, что уже было в файле, и добавляет новые данные после последнего символа. Это особенно полезно при формировании логов, накоплении результатов обработки в одном фай-ле или дозаписи данных в текстовые и JSON-файлы без потери предыдущей информации.

Блок поддерживает два формата файлов: обычные текстовые файлы (с расширениями .txt, .log, .csv и другими) и JSON-файлы (.json). При работе с JSON-файлами блок корректно обрабатывает структуру данных, дописывая новые элементы в массив или добавляя новые ключи в объект, в зависимости от текущего содержимого файла и передаваемых данных.

Параметры блока:

- Путь к файлу

  Тип данных: str (строка)

  Обязательный параметр. Указывает полный путь к файлу, в который необходимо дописать данные. Если файл по указанному пути не существует, блок вызовет ошибку, поэтому перед использованием блока «Дописать в файл» рекомендуется убедиться в существовании файла с помощью блока «Существует ли файл/папка» или создать файл заранее блоком «Записать в файл».

- Записать текст

Тип данных: str (строка) или dict (словарь)

Содержит данные, которые будут добавлены в конец файла. Если файл текстовый, параметр должен иметь строковый тип. Если файл JSON, параметр может быть как строкой (валидной JSON-строкой), так и словарём – в последнем случае блок автоматически преобразует словарь в JSON-формат перед записью.

- Кодировка

Тип данных: str (строка)

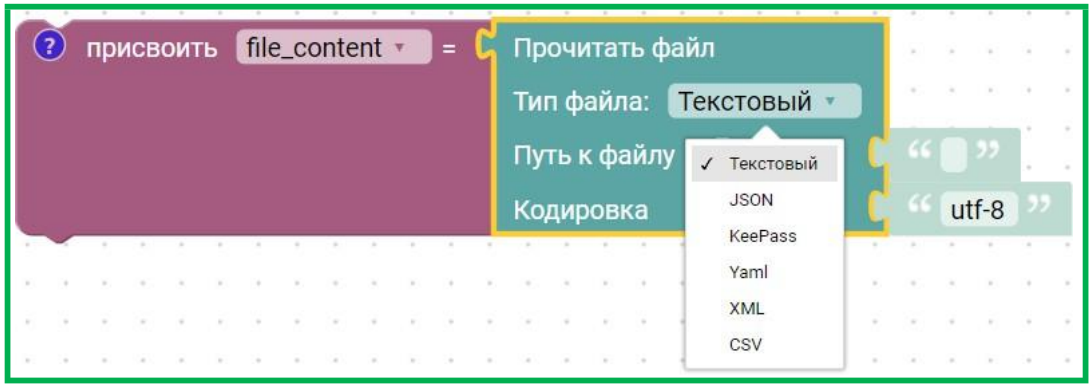
Определяет кодировку символов, в которой будет выполняться запись. Наиболее распространённые значения: utf-8 (рекомендуется для многоязычных текстов), windows-1251 (для совместимости с русскоязычными Windows-приложениями), koi8-r. Выбор кодировки должен соответствовать той, в которой файл был создан изначально, иначе возможно появление некорректно отображаемых символов («кракозябров»).

Особенности работы:

При дозаписи в текстовый файл блок не добавляет автоматически символ перевода строки перед новыми данными. Если требуется, чтобы каждое новое сообщение начиналось с новой строки, разработчик должен явно включить символы \n в начало или конец передаваемой строки.

При работе с JSON-файлами блок пытается интерпретировать существующее содержимое файла как валидную JSON-структуру. Если файл содержит массив, новые данные добавляются как следующий элемент массива. Если файл содержит объект, блок пытается добавить новые ключи. В случае, когда файл пуст или содержит невалидный JSON, операция дописывания может завершиться ошибкой. Рекомендуется для JSON-файлов использовать согласованную структуру с самого начала.

Блок «Прочитать файл»



Блок предназначен для считывания содержимого файлов различных форматов и преобразования их в структурированные данные, доступные для дальней-шей обработки роботом. В отличие от специализированных блоков для работы с офисными форматами (Excel, Word, PDF), данный блок ориентирован на текстовые и обменные форматы данных, такие как обычные текстовые файлы, JSON, CSV, YAML, XML, а также файлы баз паролей KeePass. Это универсальный инструмент для загрузки конфигураций, чтения экспортированных отчётов, импорта

данных из обменных файлов и работы с любыми текстовыми источниками информации.

Блок автоматически работает в соответствии с явно указанным форматом. В зависимости от выбранного типа, возвращаемое значение может быть простой строкой (для текстовых файлов), словарём или списком словарей (для JSON, YAML, XML), табличной структурой (для CSV) или специализированным объектом (для KeePass). Это позволяет гибко встраивать блок в различные сценарии – от простого чтения логов до сложной обработки структурированных данных.

Параметры блока:

- Путь к файлу

Обязательный параметр. Указывает полный путь к файлу, который необходимо прочитать. Блок проверяет существование файла перед чтением; если файл не найден, генерируется ошибка. Рекомендуется предварительно проверять нали-чие файла с помощью блока «Существует ли файл/папка» или обрабатывать исключение через блок «Обработка ошибок».

- Тип файла

Определяет формат, в котором следует интерпретировать содержимое файла. Доступные значения: «Текстовый», «JSON», «KeePass», «Yaml», «XML», «CSV». От выбора типа зависит, как блок будет разбирать содержимое и в каком виде вернёт результат. При выборе «Текстовый» файл читается как есть, без какой-либо структуризации.

- Кодировка (доступна для типов «Текстовый» и «CSV»)

Задаёт кодировку символов, в которой был сохранён файл. Наиболее распространённые значения: utf-8 (стандарт для современных систем), windows-1251 (для файлов, созданных в русскоязычных версиях Windows), koi8-r. Неправиль-ный выбор кодировки приводит к появлению некорректно отображаемых симво-лов («кракозябров»). Для JSON, YAML, XML и KeePass блок обычно автоматиче-ски определяет кодировку, но в некоторых случаях может потребоваться явное указание.

- Разделитель CSV (доступен при типе «CSV»)

Определяет символ, разделяющий столбцы в CSV-файле. Чаще всего используются запятая (,), точка с запятой (;) или табуляция (\t). Правильный выбор разделителя критически важен для корректного разбора табличных данных.

- Первая строка – заголовок (доступен при типе «CSV»)

Если флаг установлен, блок интерпретирует первую строку CSV-файла как имена столбцов. В этом случае результат возвращается в виде списка словарей, где ключи – значения из первой строки, а значения – данные соответствующих столбцов. Если флаг снят, данные возвращаются в виде списка списков (двумерного массива), и доступ к столбцам осуществляется по индексу.

- Возвращаемое значение

Тип возвращаемого значения зависит от выбранного типа файла:

Тип файла	Возвращаемое значение
Текстовый	Строка (str), содержащая всё содержимое файла

Тип файла	Возвращаемое значение
JSON	Словарь (dict) или список словарей (list) в зависимости от структуры JSON
YAML	Словарь (dict) или список
XML	Специализированная структура (обычно словарь с тегами и атрибутами)
CSV	Список словарей (list[dict]) если есть заголовок, иначе список списков (list[list])
KeePass	Объект, содержащий записи из базы паролей

Полученные данные могут быть сохранены в переменную и использованы в последующих блоках – например, в циклах для перебора записей, в условных операторах для проверки значений или в блоках записи для сохранения обработанной информации.

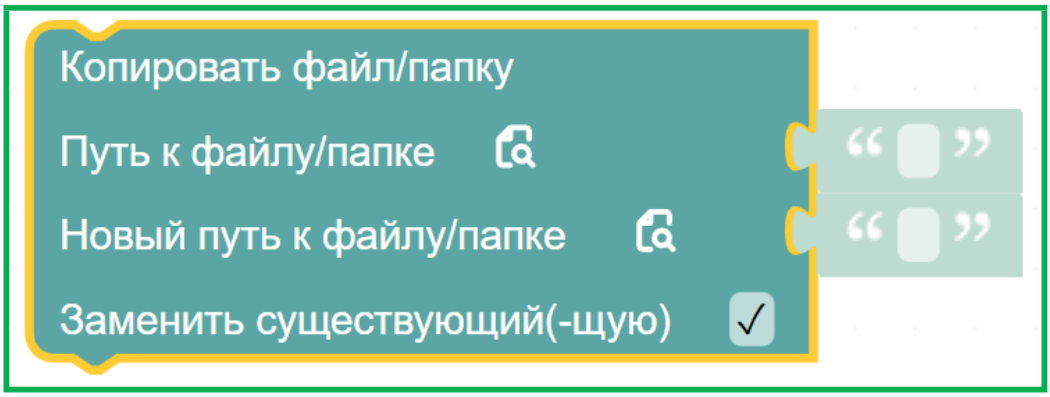
Особенности работы:

При чтении больших текстовых файлов (от десятков мегабайт) блок загружает всё содержимое в оперативную память. Для работы с очень большими файлами следует рассмотреть возможность построчного чтения с помощью специализированных блоков или встроенных скриптов, чтобы избежать перегрузки памяти.

При чтении JSON-файла блок ожидает строго валидный JSON. Наличие лишних запятых, комментариев (не предусмотренных стандартом) или неэкранированных символов вызовет ошибку. Рекомендуется перед использованием проверять JSON-файлы внешними валидаторами.

При чтении CSV важно правильно указать разделитель и кодировку. Если данные содержат символы перевода строки внутри ячеек, это может нарушить разбор; в таких случаях следует использовать более надёжные форматы, напри-мер, Excel.

Блок «Копировать файл/папку»



Блок предназначен для копирования файла или целой папки из одного места в другое. Это одна из базовых операций файловой системы, которая позволяет роботу дублировать данные, создавать резервные копии, перемещать документы между каталогами (с сохранением оригинала) или подготавливать файлы для дальнейшей обработки. В отличие от блока «Переместить файл/папку», который удаляет исходный объект, данный блок оставляет оригинал нетронутым и создаёт его точную копию в указанном целевом каталоге.

Блок поддерживает копирование как отдельных файлов, так и целых папок со всем их содержимым (включая вложенные подпапки и файлы). При копировании папки блок рекурсивно обходит все внутренние элементы и воссоздаёт структуру каталогов в месте назначения. Разработчик может управлять поведением блока в случае, если в целевой папке уже существует объект с таким же именем: либо заменить его, либо вызвать ошибку.

Параметры блока:

- Исходный путь

Обязательный параметр. Указывает полный путь к копируемому файлу или папке. Блок проверяет существование исходного объекта; если он не найден, генерируется ошибка. Для файла путь должен включать имя файла и расширение (например, C:\Reports\итог.xlsx). Для папки путь указывает на саму папку, содержимое которой будет скопировано (например, C:\Incoming).

- Путь назначения

Обязательный параметр. Задаёт путь к папке, в которую следует скопировать файл или папку. Если указанная папка не существует, блок в зависимости от настроек может её создать или вызвать ошибку. Важно понимать, что при копировании файла в Путь назначения следует указывать именно целевую папку, а не полный путь с новым именем файла (хотя некоторые реализации позволяют и переименовывать). При копировании папки в папку назначения внутри неё создаётся подпапка с именем исходной папки, а затем копируется её содержимое.

- Заменить существующий

Определяет поведение блока, если в папке назначения уже есть объект с тем же именем. Если флаг установлен, существующий файл или папка будут перезаписаны новой копией. Если флаг снят и конфликт имён обнаружен, блок генерирует ошибку, и выполнение робота останавливается (если ошибка не обработана специальными блоками). Рекомендуется для ответственных сценариев сначала проверять наличие целевого объекта с помощью блока «Существует ли файл/папка» и принимать решение о перезаписи осознанно.

- Возвращаемое значение

Блок не возвращает значимого значения. Успешное выполнение означает, что копирование завершено, и робот может переходить к следующему действию. В случае ошибки (например, исходный объект не найден, нет прав на запись в целевую папку, конфликт имён без флага замены) блок генерирует исключение, которое может быть перехвачено блоком «Обработка ошибок».

Особенности работы:

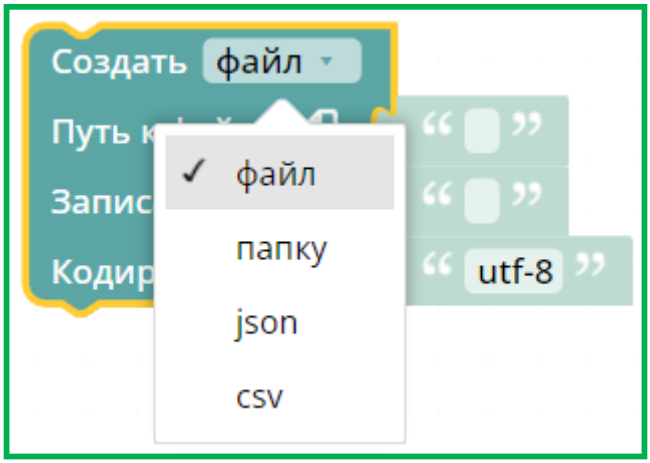
При копировании папки блок копирует всё её содержимое рекурсивно, включая все вложенные подпапки и файлы, независимо от глубины вложенности. При этом права доступа к файлам могут как сохраняться, так и сбрасываться в зависимости от операционной системы и настроек учётной записи, от которой запущен робот.

Если целевая папка не существует, блок не создаёт её автоматически (в отличие от некоторых других блоков). Разработчик должен предварительно создать папку с помощью блока «Создать папку» или убедиться в её существовании.

Копирование больших файлов (гигабайты) выполняется синхронно, то есть робот ждёт завершения операции. На время копирования процесс робота может казаться «зависшим» – это нормально. Для копирования огромных объёмов данных рекомендуется выполнять операцию в отдельном потоке или использовать асинхронные сценарии.

При копировании файла можно также изменить его имя, указав в пути назначения не только папку, но и новое имя файла. Например: исходный путь C:\temp\отчёт.xlsx, путь назначения C:\archive\отчёт_2026.xlsx. В этом случае файл будет скопирован и переименован. Однако если путь назначения оканчива-ется на слеш (указывает на папку), то имя файла сохраняется исходным.

Блок «Создать файл»



Блок предназначен для создания новых объектов в файловой системе: текстовых файлов, папок, JSON-файлов и CSV-файлов. В зависимости от выбранного режима, блок может создать пустой текстовый файл, сразу записать в него дан-ные, создать папку или сгенерировать структурированный JSON/CSV-файл. Это универсальный инструмент для начальной подготовки файлового окружения робота: создания папок для архивации, инициализации лог-файлов, формирования экспортных данных в требуемом формате.

Блок имеет выпадающее меню, где разработчик выбирает тип создаваемого объекта: «файл», «папку», «json» или «csv». В зависимости от выбора, набор доступных параметров меняется. Важная особенность: если файл или папка по указанному пути уже существуют, блок перезаписывает их (для файлов) или использует существующую папку (для папки – без удаления содержимого). Поведение с перезаписью не управляется отдельным чекбоксом – оно жёстко задано.

Режимы работы блока

1. Создать файл (текстовый)

Параметры:

- Путь к файлу (str) – полный путь к создаваемому файлу, включая имя и расширение (например, C:\Logs\robot.log). Если родительские папки не существуют, блок вызовет ошибку.

- Записать текст (bool, чекбокс) – если флаг установлен, после создания файла в него будет записан указанный текст. Если флаг снят, создаётся пустой файл.
- Текст для записи (str) – данные, которые будут записаны в файл (доступен только при активном чекбоксе «Записать текст»).
- Кодировка (str) – кодировка текстового файла (например, utf-8, windows-1251).

Особенности:

Если файл уже существует, он будет полностью перезаписан (старое содержимое удаляется). Блок не выдаёт предупреждения. Рекомендуется перед использованием проверять существование файла, если важны предыдущие данные.

2. Создать папку

Параметры:

- Путь до директории (str) – путь к родительской папке, внутри которой будет создана новая папка. Например, C:\Reports.
- Имя папки (str) – имя создаваемой папки (например, 2025_04_21).

Особенности:

Если папка с таким именем уже существует, блок не вызывает ошибку и не перезаписывает её содержимое – он просто продолжает работу. Если родительский путь не существует, блок вызовет ошибку. Этот режим удобен для гарантирован-ного создания целевой директории перед началом обработки файлов.

3. Создать JSON

Параметры:

- Путь к файлу (str) – полный путь к создаваемому JSON-файлу (обычно расширение .json).
- Данные в формате словаря (dict) – структурированные данные, которые будут преобразованы в JSON и записаны в файл. Может быть передан как перемен-ная-словарь или встроенный словарь.
- Кодировка (str) – кодировка файла (рекомендуется utf-8).

Особенности:

Если файл уже существует, он будет перезаписан. Блок автоматически преобразует словарь в валидную JSON-строку с корректными отступами (форматирование может зависеть от версии Puzzle RPA). При передаче не словаря (например, строки) блок может вызвать ошибку.

4. Создать CSV

Параметры:

- Путь к файлу (str) – полный путь к создаваемому CSV-файлу (расширение .csv).
- Данные в формате CSV (str или список списков) – данные для записи. Формат зависит от реализации: обычно это либо готовая CSV-строка, либо двумерный массив (список строк, каждая строка – список значений).
- Кодировка (str) – кодировка файла (например, utf-8, cp1251).

Особенности:

Блок не добавляет заголовки столбцов автоматически – разработчик должен включить их в передаваемые данные. Если файл существует, он перезаписывает-

ся. Разделитель полей не настраивается явно; предполагается, что данные уже содержат корректные разделители (обычно запятая или точка с запятой).

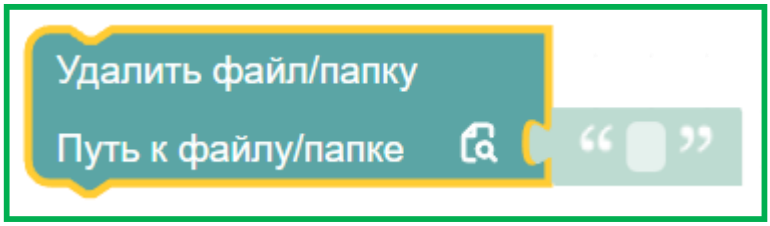
Возвращаемое значение

Блок не возвращает значимого значения. Успешное выполнение означает, что объект создан (или уже существовал в случае с папкой). В случае ошибки (недопустимый путь, недостаточно прав, неверный формат данных для JSON/CSV) блок генерирует исключение.

Особенности и ограничения

- Перезапись без предупреждения – для файлов (текстовых, JSON, CSV) блок всегда перезаписывает существующий файл. Для папки – не перезаписывает, но и не сообщает о существовании. Разработчик должен учитывать это при проектировании сценариев.
- Родительские папки не создаются – если в пути к файлу указаны несуществующие каталоги, блок вызовет ошибку. Предварительно необходимо создать их с помощью блока «Создать папку».
- Кодировка – для текстовых, JSON и CSV-файлов обязательно указывайте корректную кодировку. Для русскоязычных данных чаще всего используется utf-8 или windows-1251.
- JSON-данные – блок ожидает именно словарь. Если нужно сохранить список словарей (массив), это также возможно, но проверьте в документации вашей версии Puzzle RPA.

Блок «Удалить файл/папку»



Блок предназначен для удаления файла или целой папки с диска по указанному пути. Это необратимая операция, которая полностью стирает объект из файловой системы. При удалении папки блок рекурсивно удаляет всё её содержимое – все вложенные папки, файлы и подпапки любой глубины. Блок не перемещает удаляемые объекты в «Корзину»; они исчезают сразу и могут быть восстановлены только специальными средствами (если это вообще возможно). Поэтому использование данного блока требует особой осторожности и, как правило, предварительной проверки условий.

Параметры блока

- Путь (для режима «файл»)

Полный путь к удаляемому файлу/папке, включая имя файла и расширение (например, C:\Temp\old_report.xlsx). Блок проверяет существование файла; если файл не найден, генерируется ошибка.

Возвращаемое значение

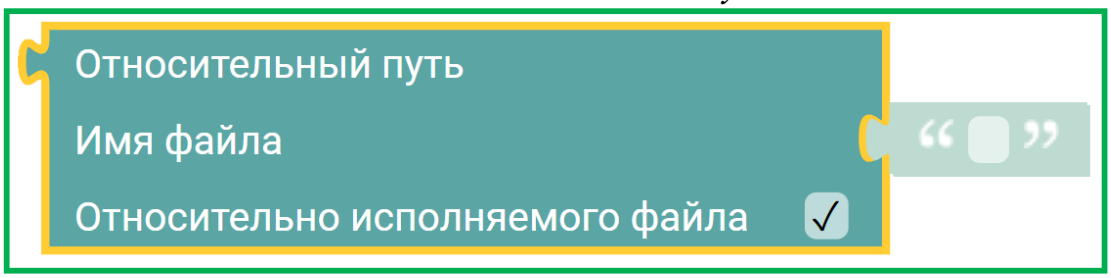
Блок не возвращает значимого значения. Успешное выполнение означает, что файл или папка удалены. В случае ошибки (объект не найден, нет прав на

удаление, файл занят другим процессом, папка не пуста, но выбран режим удаления файла и т.п.) блок генерирует исключение, которое может быть перехвачено блоком «Обработка ошибок».

Рекомендации по безопасному использованию:

- Всегда предваряйте удаление проверкой существования объекта.
- Для особо важных данных сначала создавайте резервную копию (блок «Копировать файл/папку»).
- В сценариях массового удаления используйте блок «Обработка ошибок», чтобы робот не останавливался из-за одного проблемного файла.
- Логируйте факт удаления (запись в лог-файл с указанием пути и времени) – это поможет при расследовании инцидентов.

Блок «Относительный путь»



Блок предназначен для формирования пути к файлу или папке относительно определённой базовой директории, а не от корня диска. Это позволяет создавать переносимых роботов, которые не зависят от жёстко прописанных абсолютных путей (например, C:\Users\Имя\Documents\file.txt). Вместо этого робот определяет местоположение файла относительно своего собственного расположения или относительно папки ресурсов проекта. Такой подход критически важен при развёртывании роботов на разных машинах, в виртуальных средах или при передаче исполняемых файлов другим пользователям, у которых структура каталогов может отличаться.

Параметры блока:

- Имя файла (или путь относительно базовой директории)

Указывает имя файла или относительный путь к файлу/папке, начиная от базовой директории. Можно указывать вложенные папки, например data\config.json или templates\letter.docx. Не следует начинать путь с обратного слеша или точки.

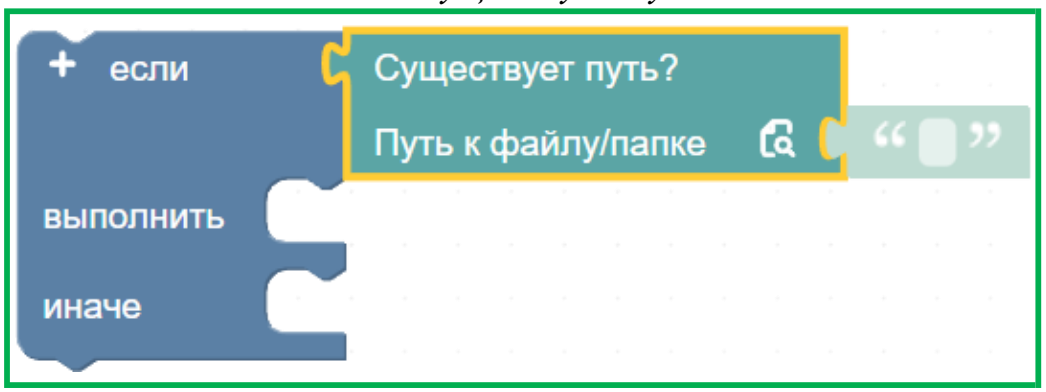
- Относительно исполняемого файла (чекбокс)

Определяет, от какой директории отсчитывается относительный путь. Если флаг установлен, базовой директорией считается папка, в которой находится исполняемый файл робота (при компиляции в .exe). Если флаг снят, базовая директория определяется по правилам платформы (см. раздел «Особенности работы»).

- Возвращаемое значение

Блок возвращает строку (str) – полный абсолютный путь к указанному файлу или папке, построенный на основе текущего контекста выполнения. Этот путь можно сохранить в переменную и использовать в других блоках, например, в «Прочитать файл», «Копировать файл/папку» или «Отправить письмо» (для вложений).

Блок «Существует путь»



Блок предназначен для проверки существования указанного файла или каталога в файловой системе. Это один из ключевых блоков для построения устойчивых роботов, так как позволяет заранее убедиться, что файл или папка доступны, прежде чем выполнять с ними какие-либо операции (чтение, копирование, удаление). Блок не изменяет файловую систему, а только возвращает логический результат проверки.

Параметры блока:

- Путь

Полный путь к файлу или папке, существование которого необходимо проверить. Путь может быть абсолютным (например, C:\Temp\report.xlsx) или относительным (например, .\data\config.json). Рекомендуется использовать блок «Относительный путь» для формирования переносимых путей.

- Возвращаемое значение

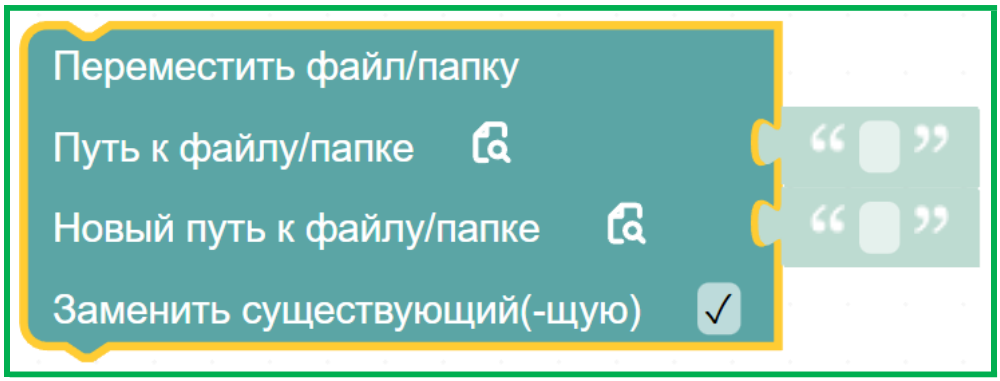
Блок возвращает логическое значение (bool):

Истина (True) – объект (файл или папка) существует по указанному пути и его тип соответствует выбранному режиму.

Ложь (False) – объект не существует, либо его тип не соответствует режиму (например, ожидали файл, а там папка, или наоборот).

Это значение может быть использовано в условном блоке «Если» для принятия решений.

Блок «Переместить файл/папку»



Блок предназначен для перемещения файла или целой папки из одного места в другое. В отличие от копирования, при перемещении исходный объект удаляется из старой папки и появляется в новой. Это эквивалентно операции «Вырезать –

Вставить» в проводнике Windows. Перемещение выполняется быстро, особенно если исходная и целевая папки находятся на одном диске (в этом случае операционная система просто изменяет ссылку на файл, не перемещая данные физиче-ски). Блок поддерживает перемещение как отдельных файлов, так и папок со всем их содержимым.

Параметры блока:

- Исходный путь

Полный путь к перемещаемому файлу или папке. Для файла путь должен включать имя и расширение (например, C:\Temp\report.docx). Для папки – путь к самому каталогу (например, C:\Incoming\Documents). Блок проверяет существование исходного объекта; если он не найден, генерируется ошибка.

- Путь назначения

Целевая папка, в которую будет перемещён объект. Важно: указывается именно папка, а не полный путь с новым именем (хотя некоторые реализации позволяют и переименование). Например, если нужно переместить файл от-чёт.xlsx в папку C:\Archive, в поле «Путь назначения» следует указать C:\Archive. Файл окажется по пути C:\Archive\отчёт.xlsx. Если целевая папка не существует, блок вызовет ошибку.

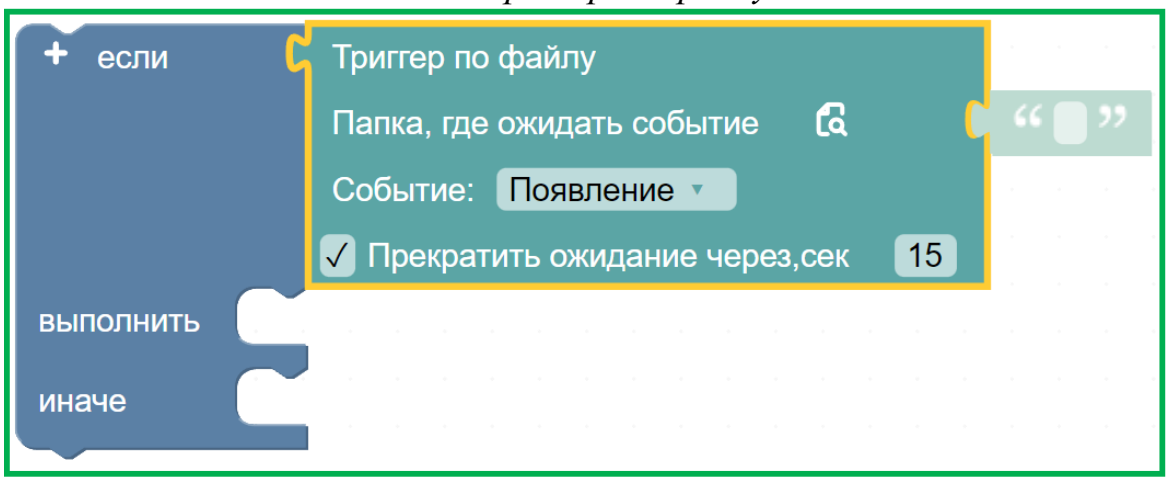
- Заменить существующий

Определяет поведение при конфликте имён. Если флаг установлен и в папке назначения уже есть файл (или папка) с тем же именем, существующий объект будет удалён, а на его место переместится новый. Если флаг снят и конфликт обнаружен, блок генерирует ошибку. Для папок при включённой замене поведение может различаться: в одних реализациях удаляется вся целевая папка вместе с содержимым, в других – происходит слияние. Рекомендуется перед перемещением проверять наличие целевого объекта с помощью блока «Существует путь» и, при необходимости, обрабатывать конфликт вручную.

- Возвращаемое значение

Блок не возвращает значимого значения. Успешное выполнение означает, что объект перемещён. В случае ошибки (исходный объект не найден, целевая папка не существует, нет прав, конфликт имён без замены) блок генерирует ис-ключение.

Блок «Триггер по файлу»



Блок предназначен для ожидания наступления определённого события с файлами в указанной папке. Он позволяет роботу приостановить выполнение до тех пор, пока в заданном каталоге не появится новый файл или пока не исчезнет существующий. Это мощный инструмент для построения событийно-ориентированных сценариев, когда робот должен реагировать на изменения в файловой системе в реальном времени, а не просто опрашивать папку по расписанию. Блок особенно полезен для автоматизации процессов, где документы поступают нерегулярно или в неизвестное время, и запускать робота каждую минуту неэффективно.

Параметры блока:

- Путь к папке

Указывает полный путь к каталогу, за которым блок будет следить. Папка должна существовать на момент вызова блока; если её нет, блок может вызвать ошибку или вернуть «ложь» в зависимости от реализации. Рекомендуется предварительно проверять существование папки с помощью блока «Существует путь».

- Событие

Определяет, какое изменение в папке должно произойти, чтобы блок завершил ожидание с результатом «истина». Доступные значения:

«Появление» – блок ожидает, что в папке появится новый файл (или папка). При этом отслеживается именно создание нового объекта, а не изменение существующего.

«Исчезновение» – блок ожидает, что из папки будет удалён какой-либо файл или папка. Полезно для сценариев, где робот ждёт, пока другой процесс или пользователь заберёт файл.

- Прекратить ожидание через, сек

Тип данных: bool (логический, чекбокс) и int (целое число, поле для ввода)

Если флаг установлен, блок ожидает событие не более указанного количества секунд. По истечении таймаута блок завершается и возвращает «ложь». Если флаг снят, блок будет ждать бесконечно долго (пока событие не наступит). Для промышленных роботов рекомендуется всегда устанавливать разумный таймаут (например, 300–3600 секунд в зависимости от бизнес-процесса), чтобы избежать «зависания» робота.

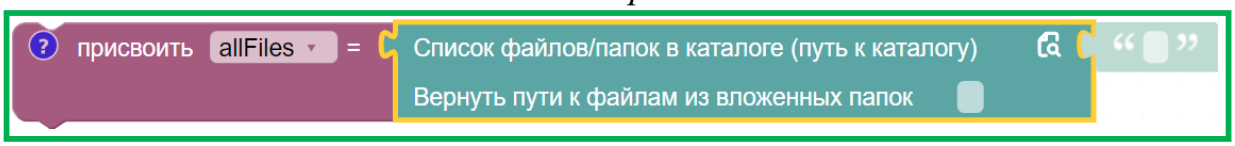
- Возвращаемое значение

Блок возвращает логическое значение (bool):

- Истина (True) – ожидаемое событие (появление или исчезновение файла) произошло в течение заданного времени.
- Ложь (False) – событие не наступило до истечения таймаута (если он был установлен) либо произошла ошибка (например, указанная папка не существует).

Результат может быть использован в условном блоке «Если» для принятия решения: продолжить обработку появившегося файла, повторить ожидание, отправить уведомление о таймауте и т.п.

Блок «Список файлов/папок»



Блок предназначен для получения списка всех файлов (или папок) в указанной директории. Он сканирует содержимое каталога и возвращает массив полных путей к каждому найденному объекту. Это один из ключевых блоков для пакет-ной обработки документов, когда робот должен пройтись по всем файлам в папке и выполнить с каждым из них однотипные действия (прочитать, скопировать, переместить, удалить, извлечь данные). Блок может работать как с файлами только верхнего уровня, так и рекурсивно обходить все вложенные папки, включая подпапки любой глубины.

Параметры блока:

- Путь к папке

Полный путь к каталогу, содержимое которого нужно получить. Папка должна существовать; если её нет, блок вызовет ошибку. Рекомендуется предва-рительно проверять существование папки с помощью блока «Существует путь».

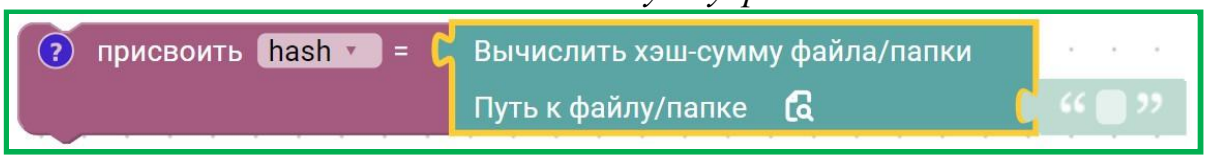
- Вернуть пути к файлам из вложенных папок (чекбокс)

Определяет глубину сканирования. Если флаг установлен, блок рекурсивно обходит все вложенные папки внутри указанной директории и включает в результат файлы (или папки) из всех уровней вложенности. Если флаг снят, блок ограничивается только объектами, находящимися непосредственно в указанной папке (без захода в подпапки).

- Возвращаемое значение

Блок возвращает массив (list) строк, где каждый элемент – полный абсолютный путь к файлу или папке, найденному в соответствии с настройками. Если в указанной папке нет объектов подходящего типа, возвращается пустой массив. Полученный массив удобно использовать в цикле «Для каждого» для последовательной обработки каждого элемента.

Блок «Вычислить хэш-сумму файла/папки»



Блок предназначен для вычисления хэш-суммы (контрольной суммы) файла или целой папки на основе их содержимого. Хэш-сумма – это уникальная строка фиксированной длины, которая получается в результате применения криптографической хэш-функции (например, SHA-1) к данным. Даже незначительное изменение содержимого файла (один символ, один бит) приводит к кардинальному изменению хэш-суммы. Это свойство делает блок незаменимым для проверки целостности данных, обнаружения изменений в файлах, сравнения документов (идентичны ли два файла), а также для контроля, что файл не был повреждён при передаче или хранении.

Важная особенность блока: при вычислении хэш-суммы папки учитывается содержимое всех входящих в неё файлов (включая вложенные подпапки), но не учитываются имена файлов, даты создания, атрибуты и другие метаданные. Пустые файлы также не влияют на результат. Таким образом, две папки с одинаковым набором файлов, имеющих одинаковое содержимое, но разными именами или структурой вложенности, дадут одинаковую хэш-сумму. Если же содержимое хотя бы одного файла отличается, хэш-сумма папки будет разной.

Параметры блока:

- Путь к файлу/папке

Обязательный параметр. Указывает полный путь к файлу или папке, для которых необходимо вычислить хэш-сумму. Блок проверяет существование объекта по указанному пути; если объект не найден, генерируется ошибка. Примеры: C:\Documents\contract.pdf или D:\Invoices\2024.

- Возвращаемое значение

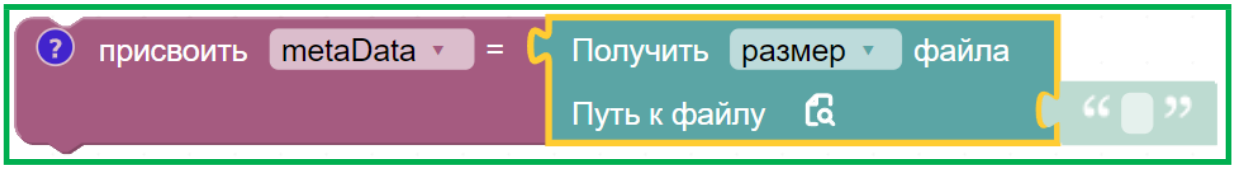
Блок возвращает строку (str), содержащую вычисленное значение хэш-суммы в шестнадцатеричном формате. Длина строки зависит от используемого алгоритма хэширования (в Puzzle RPA обычно используется SHA-1, дающий 40 симво-лов, или другой алгоритм).

Пример возвращаемого значения:

8339c3a8bef6d51ddfe301f9ddf267577899c2ca.

Это значение можно сохранить в переменную, сравнить с эталонной хэш-суммой или записать в лог для аудита.

Блок «Получить метаданные файла»



Блок предназначен для получения значения выбранного метапараметра файла по указанному пути. С помощью этого блока можно узнать размер файла в байтах, дату его последнего изменения, дату создания, полное имя с расширением или только расширение. Это бывает нужно для проверки свойств файлов перед обработкой, логирования, условных переходов (например, если файл слишком большой или слишком старый) или для динамического формирования имён.

Если файл по указанному пути не существует, блок завершается ошибкой с сообщением «По указанному пути файл не найден». Поэтому перед использованием блока рекомендуется убедиться в наличии файла, например с помощью блока «Проверить существование файла/папки».

Параметры блока:

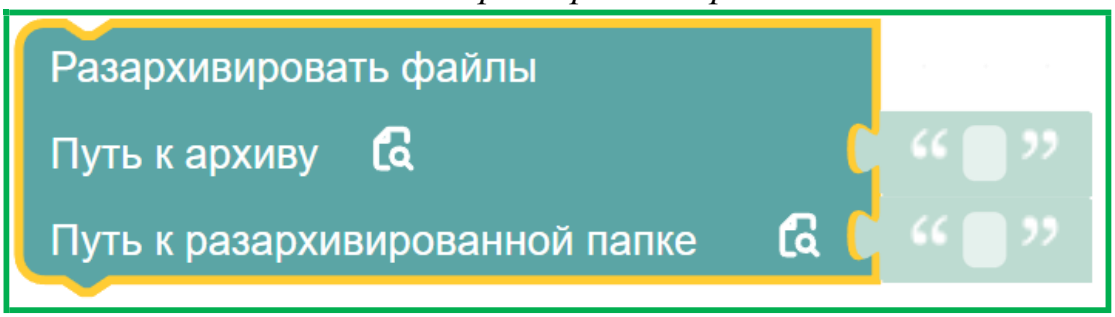
- Получить … файла – выпадающий список, который определяет, какой именно метапараметр будет возвращён. Тип данных: dropdown (список). Доступны следующие значения:
  - ✓ размер – размер файла в байтах;
  - ✓ дата изменения – дата и время последнего изменения файла;
  - ✓ время создания – дата и время создания файла;
  - ✓ имя файла – полное имя файла с расширением (например, report.txt);
  - ✓ расширение – расширение файла без точки (например, txt).
- Путь к файлу – полный путь к файлу, для которого нужно получить метадан-ные. Тип данных: str (строка). Блок проверяет существование файла по этому пути; если файл отсутствует – генерируется ошибка. Пример: C:\Temp\report.txt.
- Возвращаемое значение:

Тип и формат возвращаемого значения зависят от выбранного значения в параметре «Получить … файла»:

- ✓ Если выбрано размер – возвращается целое число (int) – размер файла в байтах. Пример: 2048.
- ✓ Если выбрано дата изменения или время создания – возвращается строка (str) в формате DD.MM.YYYY HH:MM:SS. Пример: 14.03.2025 18:30:22.
- ✓ Если выбрано имя файла или расширение – возвращается строка (str) с соответствующим значением. Пример для имени файла: report.txt, для расширения: txt.

Полученное значение можно сохранить в переменную, использовать в условии или записать в лог для дальнейшего анализа.

Блок «Разархивировать файлы»



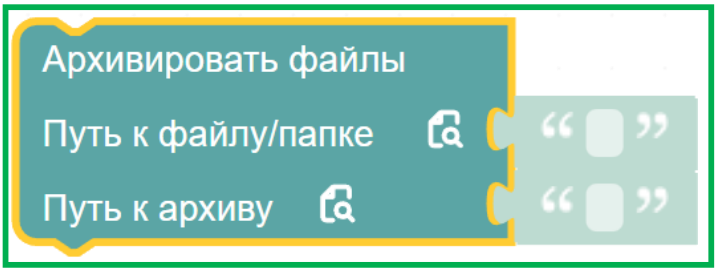
Блок предназначен для извлечения файлов и папок из архивов форматов ZIP, RAR и 7z и сохранения их в указанную целевую папку. Блок полезен для автоматической подготовки исходных данных перед обработкой (например, перед анализом входящих документов, загруженных в архиве), для распаковки инсталляционных пакетов или для доступа к содержимому архивированных отчётов.

Параметры блока:

- Путь к архиву – обязательный параметр. Указывает полный путь к файлу архива (ZIP, RAR или 7z), который необходимо разархивировать. Блок проверяет существование файла по указанному пути; если архив не найден, генерируется ошибка. Тип данных: str (строка). Пример: C:\Downloads\report_2025.zip.
- Папка назначения – обязательный параметр. Указывает путь к папке, в которую будут извлечены файлы и папки из архива. Если папка не существует, блок создаёт её автоматически. Тип данных: str (строка). Пример: C:\Extracted\report_2025.
- Возвращаемое значение:

Блок возвращает список строк – полных путей ко всем файлам, которые были успешно извлечены из архива. Это позволяет в дальнейшем обработать каждый распакованный файл (например, передать пути в блок «Получить метаданные файла» или «Вычислить хэш-сумму файла/папки»). Если архив был пустым или все файлы были пропущены из-за параметра перезаписи, возвращается пустой список.

Блок «Архивировать файлы»

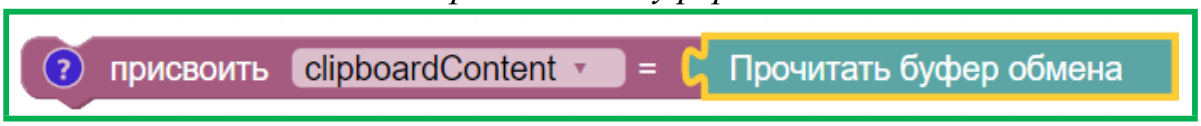


Блок предназначен для сжатия указанных файлов или целой папки в ZIP-архив. Это полезно для уменьшения объёма данных перед передачей по сети, для создания резервных копий, для упаковки группы файлов в один контейнер или для подготовки данных к отправке по электронной почте.

Параметры блока:

- Что архивировать – обязательный параметр. Указывает, какие объекты необходимо сжать. Можно передать путь к одному файлу, путь к одной папке или список путей (несколько файлов и/или папок). Тип данных: str (строка) или list (список строк). Пример для одного файла: C:\Docs\contract.pdf. Пример для папки: D:\Invoices\2024.
- Путь к архиву – обязательный параметр. Полный путь к создаваемому ZIP-архиву, включая имя файла с расширением .zip. Если указанная папка не существует, блок создаёт её автоматически. Тип данных: str (строка). Пример: C:\Backups\archive_20250422.zip.
- Возвращаемое значение: Блок возвращает строку (str) – полный путь к созданному ZIP-архиву. Это значение удобно использовать в последующих блоках, например для передачи в блок «Отправить файл по почте» или для логирова-ния.

Блок «Прочитать буфер обмена»



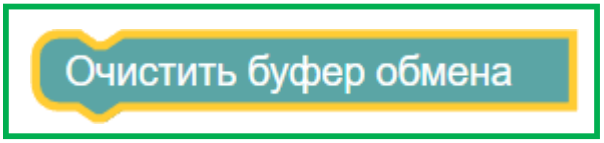
Блок предназначен для считывания текстового содержимого буфера обмена операционной системы и возврата его в виде строки. Это полезно для автоматизации задач, связанных с переносом данных между приложениями: например, можно скопировать текст из электронной почты или веб-страницы, затем считать его роботом для дальнейшей обработки (парсинга, поиска ключевых слов, записи в файл). Также блок удобен для интеграции с действиями пользователя, когда данные подготавливаются вручную и помещаются в буфер обмена.

Параметры блока:

- У блока отсутствуют обязательные или настраиваемые параметры. Он просто выполняет чтение буфера обмена в момент своего выполнения.
- Возвращаемое значение:Блок возвращает строку (str), содержащую текстовое содержимое буфера обмена. Если буфер обмена содержит нетекстовые данные или пуст, возвращается пустая строка ("").

Полученную строку можно сохранить в переменную, проверить на пустоту, записать в лог, сохранить в файл или передать в блоки обработки текста (например, «Найти и заменить», «Регулярное выражение»).

Блок «Очистить буфер обмена»



Блок предназначен для полной очистки содержимого буфера обмена операционной системы. После выполнения блока буфер обмена становится пустым. Это полезно в сценариях, где необходимо удалить конфиденциальные данные (пароли, личную информацию, номера документов) после их использования, что-бы предотвратить случайную вставку в другое приложение или несанкционированный доступ к данным из буфера обмена. Также очистка помогает избежать ложных срабатываний в последующих блоках чтения буфера обмена, если важно гарантировать, что предыдущие данные не повлияют на логику робота.

Параметры блока:

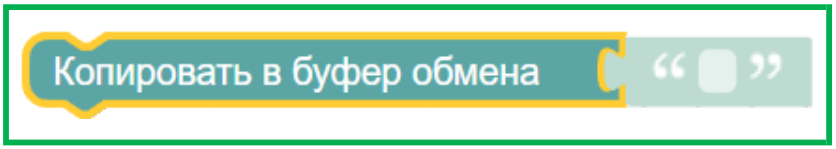
- У блока отсутствуют какие-либо параметры. Он выполняет единственное действие – очищает буфер обмена.
- Возвращаемое значение: Блок не возвращает никакого значения. После его выполнения буфер обмена становится пустым. Блок всегда завершается успешно, за исключением редких случаев системных ошибок (например, если операционная система не позволяет получить доступ к буферу обмена).

Рекомендации по использованию:

Рекомендуется использовать блок «Очистить буфер обмена» после того, как робот считал конфиденциальные данные через блок «Прочитать буфер обмена» и

обработал их. Это снижает риск утечки информации, особенно если робот работает на общем компьютере или в среде с несколькими пользователями.

Блок «Копировать в буфер обмена»



Блок предназначен для копирования текстового значения или содержимого указанной переменной в буфер обмена операционной системы. После выполнения блока скопированные данные становятся доступными для вставки (Ctrl+V) в любом другом приложении или внутри робота через блок «Прочитать буфер обме-на». Блок полезен для автоматизации переноса данных между системами, для подготовки данных к ручной вставке пользователем, для передачи информации из робота во внешние приложения (например, в текстовый редактор, электронную таблицу или поле ввода веб-формы), а также для временного хранения промежуточных результатов.

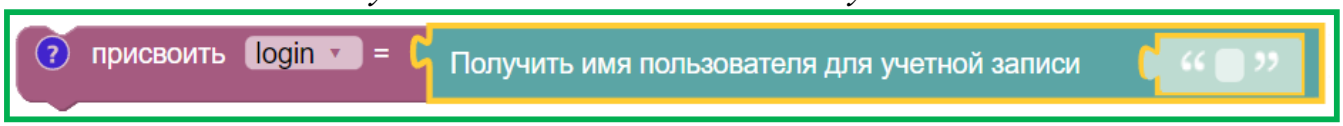
Параметры блока:

- Текст для копирования – обязательный параметр. Указывает текст или переменную, содержимое которой необходимо скопировать в буфер обмена. Тип данных: str (строка) или любой другой тип, который будет автоматически приведён к строке.
- Возвращаемое значение: Блок не возвращает никакого значения. Его единственная задача – поместить указанный текст в буфер обмена. После успешного выполнения блок завершается без ошибок. При неудаче (например, если операционная система не позволяет получить доступ к буферу обмена) генерируется ошибка с соответствующим сообщением.

Рекомендации по использованию:

Рекомендуется использовать блок «Копировать в буфер обмена» совместно с блоком «Прочитать буфер обмена» для передачи данных между различными частями робота или между роботом и внешними приложениями. Если копируются конфиденциальные данные (пароли, персональные сведения), следует после использования вызвать блок «Очистить буфер обмена», чтобы избежать утечки информации.

Блок «Получить имя пользователя для учетной записи»



Блок предназначен для извлечения имени пользователя (логина) из диспетчера учётных данных операционной системы по указанному идентификатору сервиса (названию учётной записи). Это полезно для автоматизации процессов, где требуется получить сохранённые учётные данные (например, логин для доступа к корпоративному порталу, базе данных или сетевой папке), не запрашивая их каждый раз у пользователя и не храня в открытом виде в скрипте. Блок работает с защищённым хранилищем учётных данных ОС.

Параметры блока:

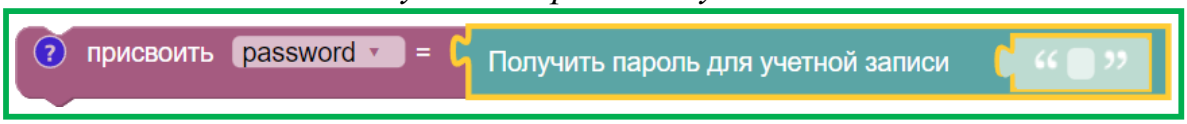
- Сервис (название учётной записи) – обязательный параметр. Указывает идентификатор сервиса, для которого необходимо получить имя пользователя. Тип данных: str (строка).

Значение должно точно соответствовать тому, которое использовалось при сохранении учётных данных в диспетчере ОС.

- Возвращаемое значение: Блок возвращает строку (str), содержащую имя пользователя (логин), связанное с указанным сервисом в диспетчере учётных данных. Если имя пользователя не найдено или запись отсутствует, блок завершается ошибкой с сообщением о том, что учётные данные не обнаружены.

Полученное имя пользователя можно сохранить в переменную, использовать для авторизации в приложениях, для формирования строк подключения к базе данных или для логирования.

Блок «Получить пароль для учетной записи»



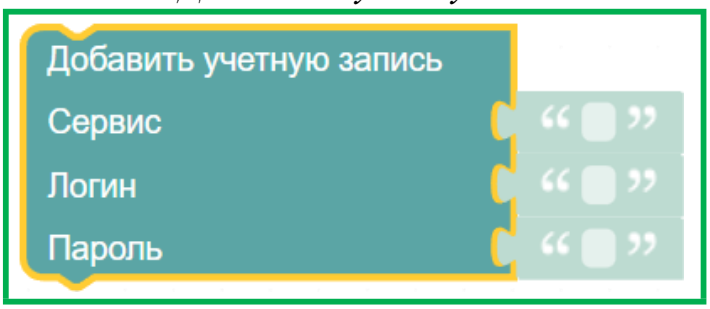
Блок предназначен для извлечения пароля из диспетчера учётных данных операционной системы по указанному идентификатору сервиса (названию учётной записи). Это необходимо для автоматизации процессов, где требуется получить сохранённый пароль (например, для подключения к базе данных, авторизации в приложении, доступа к сетевой папке или API) без его явного хранения в коде робота. Использование защищённого хранилища ОС повышает безопас-ность: пароль не отображается в логах и не сохраняется в открытом виде в пере-менных.

Параметры блока:

- Сервис (название учётной записи) – обязательный параметр. Указывает идентификатор сервиса, для которого необходимо получить пароль. Тип данных: str (строка). Пример: CorporatePortal, DatabaseBackup, VPN_Connection. Значение должно точно совпадать с тем, которое использовалось при сохранении учёт-ных данных в диспетчере ОС.
- Возвращаемое значение: Блок возвращает строку (str), содержащую пароль, связанный с указанным сервисом в диспетчере учётных данных. Если пароль не найден или запись отсутствует, блок завершается ошибкой с сообщением, что учётные данные не обнаружены.

Полученный пароль можно сохранить в переменную и использовать для авторизации, формирования строки подключения или передачи в другие блоки (например, «Архивировать файлы» с паролем или «Выполнить команду»).

Блок «Добавить учетную запись»



Блок предназначен для добавления новой записи с учётными данными (логин и пароль) в диспетчер учётных данных операционной системы. После успешного выполнения блока указанные имя пользователя и пароль сохраняются под уникальным именем сервиса и могут быть в дальнейшем извлечены блоками «Получить имя пользователя для учетной записи» и «Получить пароль для учетной записи». Это удобно для однократной настройки учётных записей, которые робот будет использовать многократно, без жёсткого зашивания паролей в код и без необходимости вводить их при каждом запуске.

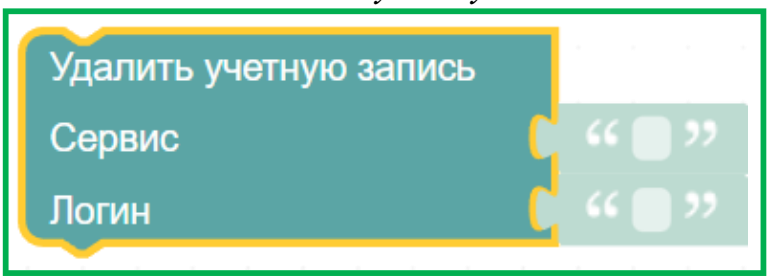
Параметры блока:

- Сервис – обязательный параметр. Уникальное название учётной записи, под которым она будет сохранена в диспетчере учётных данных. Тип данных: str (строка). Значение не должно совпадать с уже существующими именами сервисов на данном компьютере для текущего пользователя. Пример: CorporatePortal, DbConnection, BackupService.
- Имя пользователя – обязательный параметр. Логин или имя пользователя, которое необходимо сохранить. Тип данных: str (строка). Пример: ivan.ivanov или admin@example.com.
- Пароль – обязательный параметр. Пароль, соответствующий указанному имени пользователя. Тип данных: str (строка). Пример: P@ssw0rd123. Пароль сохраняется в защищённом хранилище и не отображается в открытом виде в логах выполнения робота (при корректной настройке платформы).
- Возвращаемое значение: Блок не возвращает значения. При успешном добавлении учётной записи блок завершается без ошибок. При возникновении ошибки (например, сервис с таким именем уже существует, отсутствуют права на за-пись в диспетчер учётных данных, некорректные параметры) блок генерирует исключение с соответствующим сообщением.

Рекомендации по использованию:

Перед добавлением новой учётной записи рекомендуется проверить, не существует ли уже сервис с таким именем.

Блок «Удалить учетную запись»



Блок предназначен для удаления ранее сохранённой учётной записи (логина и пароля) из диспетчера учётных данных операционной системы. Удаление производится по уникальному имени сервиса, которое было указано при добавлении записи. Блок полезен для очистки устаревших или более неиспользуемых учётных данных, для ротации паролей (когда старую запись нужно удалить перед добавлением новой) или для обеспечения безопасности при завершении работы робота, если учётные данные требовались только на время сеанса.

Параметры блока:

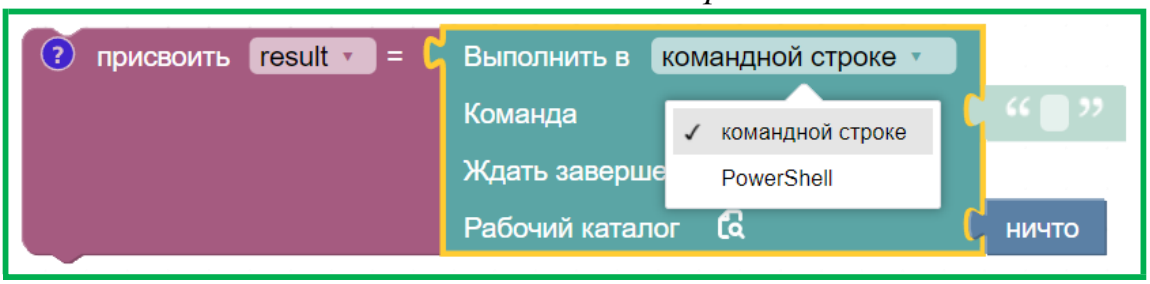
- Сервис – обязательный параметр. Уникальное название учётной записи, которую необходимо удалить. Тип данных: str (строка). Значение должно точно совпадать с именем сервиса, указанным при добавлении учётной записи. Пример: CorporatePortal, DbConnection, BackupService.
- Возвращаемое значение: Блок не возвращает значения. При успешном удале-нии учётной записи блок завершается без ошибок. Если запись с указанным именем сервиса не найдена, блок генерирует ошибку с сообщением, например, «Учётная запись с указанным именем сервиса не обнаружена».

Блок «Получить имя пользователя операционной системы»



Блок предназначен для получения имени текущего пользователя операционной системы, под которым запущен робот. Возвращаемое значение соответствует имени учётной записи, используемой в системе (например, в Windows – это имя доменного или локального пользователя, в Linux/macOS – имя пользователя). Блок полезен для логирования (фиксация того, от чьего имени выполнялись дей-ствия), для формирования динамических путей к папкам пользователя, для раз-граничения доступа или для передачи имени пользователя в другие системы.

Блок «Командная строка»



Блок предназначен для выполнения указанной команды в командной строке (cmd.exe) или в среде PowerShell. Это позволяет автоматизировать широкий

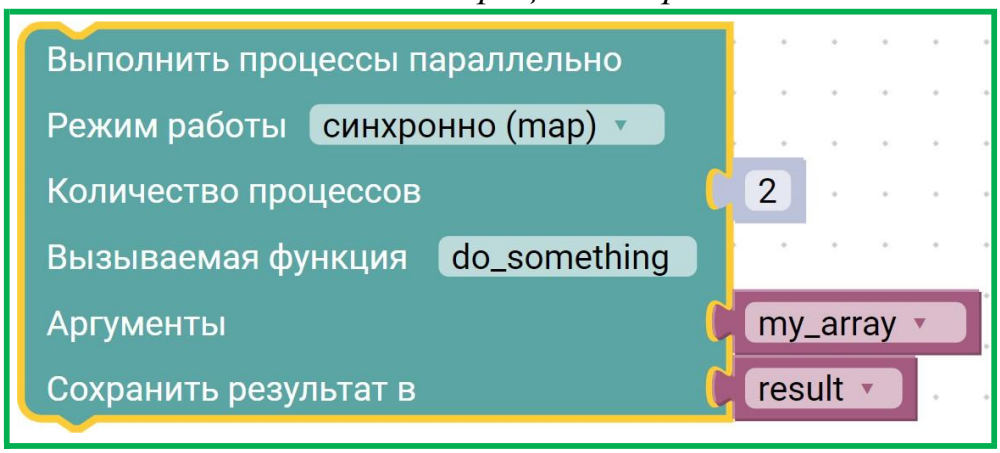
спектр системных операций: запуск сторонних утилит, выполнение скриптов, управление файлами и процессами, работу с сетью, реестром и другими компонентами операционной системы, для которых нет специализированных блоков в RPA-платформе. Блок даёт возможность гибко расширять функциональность робота за счёт любых консольных команд.

Параметры блока:

- Команда – обязательный параметр. Строка, содержащая команду (или путь к исполняемому файлу с аргументами), которую необходимо выполнить. Тип данных: str (строка). Можно использовать как обычные команды cmd (напри-мер, dir C:\Temp), так и команды PowerShell (например, Get-ChildItem -Path C:\Temp). Если команда требует прав администратора, блок может не срабо-тать, если робот запущен без соответствующих прав.
- Ждать завершения команды – необязательный параметр. Логический флажок (checkbox), определяющий, должен ли робот ожидать окончания выполнения команды или запустить её асинхронно. Тип данных: bool (логический) или dropdown (Да / Нет). Возможные значения: Да (истина, флажок установлен) – робот ждёт завершения команды, после чего получает результат. Нет (ложь, флажок снят) – команда запускается в фоновом режиме, робот продолжает выполнение следующих блоков немедленно. При асинхронном запуске возвращаемое значение обычно отсутствует или ограничено.
- Рабочий каталог – необязательный параметр. Указывает путь к папке, которая станет текущей рабочей директорией для выполняемой команды. Если пара-метр не задан, используется рабочий каталог самого робота (обычно папка с проектом или исполняемым файлом). Тип данных: str (строка). Пример: C:\Projects\Data.
- Возвращаемое значение: Возвращаемое значение зависит от того, установлен ли флажок «Ждать завершения команды» и от конкретной реализации плат-формы. Обычно блок возвращает структурированный объект или несколько значений: Вывод команды (stdout) – строка (str) или список строк (list), содер-жащая всё, что команда напечатала в стандартный вывод. Если вывода не было, возвращается пустая строка или пустой список. Вывод ошибок (stderr) – строка (str) или список строк (list), содержащая сообщения об ошибках, если они возникли. Может быть полезен для диагностики. Код возврата – целое число (int), возвращаемое завершённым процессом. Ноль обычно означает успех, любое другое значение – ошибку или особое состояние (код зависит от команды).

Полученные данные можно сохранить в переменную, проанализировать с помощью регулярных выражений, записать в лог или передать в другие блоки для дальнейшей обработки.

Блок «Выполнить процессы параллельно»



Блок предназначен для запуска пула процессов с целью выполнения параллельных вычислений. Он используется в сценариях, где необходимо обработать множество независимых задач одновременно (например, пакетная обработка файлов, массовые запросы к API, параллельный парсинг страниц, вычисления на разных наборах данных). Блок позволяет значительно ускорить выполнение работы за счёт использования нескольких ядер процессора.

Параметры блока:

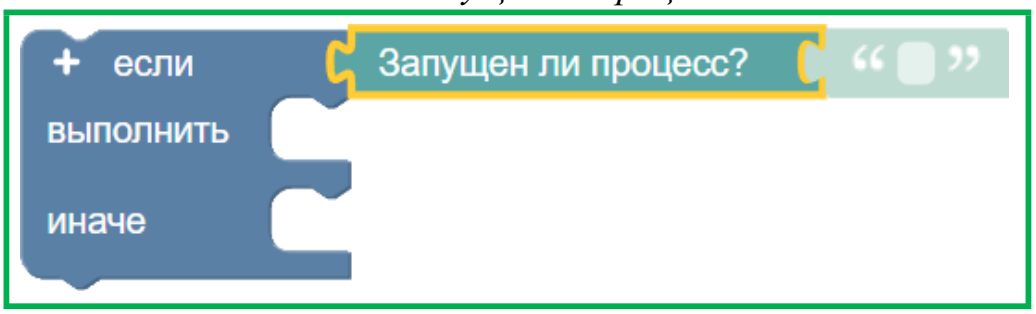
- Режим работы – определяет, будет ли блок ожидать завершения всех параллельных процессов. Тип данных: dropdown (список). Возможные значения: Синхронно – блок ждёт окончания всех процессов и возвращает результаты. Асинхронно – блок запускает процессы и продолжает выполнение без ожидания.
- Количество процессов – максимальное число процессов, запускаемых одновременно. Тип данных: int (целое число). Рекомендуется указывать значение от 1 до количества ядер процессора.
- Вызываемая функция – имя функции (в виде строки), которая будет выполняться в каждом процессе. Функция должна быть определена в алгоритме или импортирована.
- Аргументы – список значений, каждый элемент которого будет передан в один вызов вызываемой функции. Если в списке 10 элементов, а процессов 3, то сначала запустятся 3 процесса с первыми тремя аргументами, затем, по мере завершения, – следующие.
- Сохранить результат в – переменная, в которую будут записаны возвращаемые значения из вызванных функций. Доступна только при синхронном режиме. Тип данных: var (переменная). Пример: result.
- Возвращаемое значение: В режиме работы Синхронно блок возвращает список (list) результатов, полученных от всех вызовов вызываемой функции. Порядок элементов списка соответствует порядку переданных аргументов. Если функ-ция не возвращает значения, в списке будут элементы None. В режиме работы Асинхронно блок возвращает объект пула процессов (аналог multiprocessing.Pool). Этот объект можно использовать для дальнейшего управления (например, принудительного завершения, получения результатов позже)

– но в рамках визуального алгоритма обычно возвращается для служебных целей или не используется.

Особенности кодогенерации:

При сохранении проекта в виде исполняемого файла (exe) или Python-скрипта, в сгенерированный код автоматически добавляется вызов freeze_support(). Это необходимо для корректной работы многопроцессорности в собранных приложениях (особенно на Windows). При запуске алгоритма непо-средственно из студии разработки эта строка не генерируется. Разработчик может не заботиться об этом – платформа добавляет freeze_support() автоматически.

Блок «Запущен ли процесс»



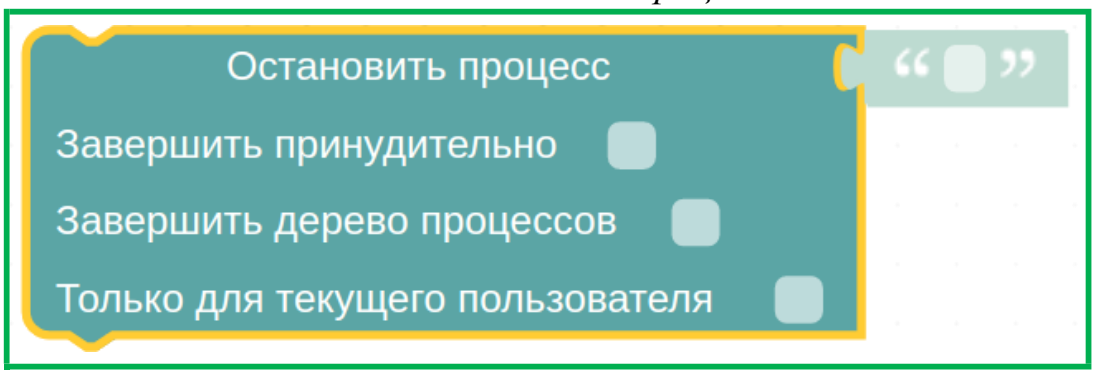
Блок предназначен для проверки, запущен ли указанный процесс в операционной системе. Он анализирует список текущих процессов и возвращает логическое значение: «истина», если обнаружен хотя бы один экземпляр процесса с заданным именем, и «ложь» – в противном случае. Блок полезен для контроля состояния системы перед выполнением критических операций (например, перед запуском нового экземпляра приложения, которое не должно дублироваться), для ожидания завершения процесса, для проверки, что требуемый сервис или программа активны, а также для диагностики и логирования.

Параметры блока:

- Имя процесса – обязательный параметр. Указывает имя процесса, наличие которого необходимо проверить. Тип данных: str (строка). Имя обычно вводится без пути к исполняемому файлу и без расширения.
- Возвращаемое значение: Блок возвращает логическое значение (bool – «истина» или «ложь»): Истина (True) – если хотя бы один экземпляр указанного процесса запущен в системе. Ложь (False) – если ни одного экземпляра указанного процесса не обнаружено.

Полученное значение можно использовать в блоке «Условие» (если процесс запущен – выполнить одни действия, если нет – другие), сохранить в переменную для логирования или передать в другие блоки.

Блок «Остановить процесс»



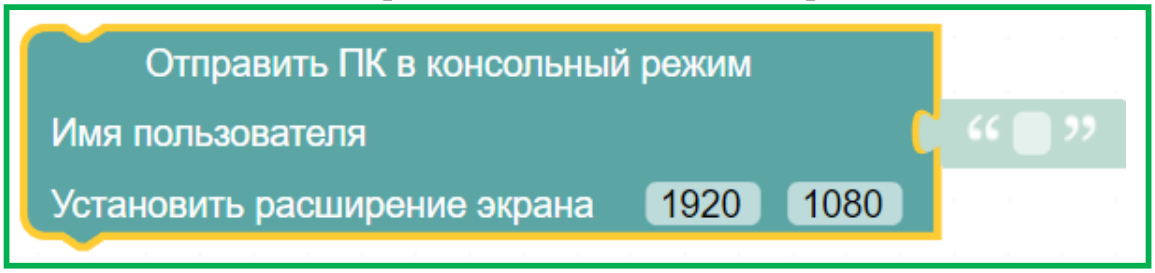
Блок предназначен для принудительной остановки (завершения) указанного процесса в операционной системе. Если запущено несколько экземпляров процесса с одинаковым именем, блок завершает все обнаруженные экземпляры. Блок полезен для освобождения системных ресурсов, перезапуска зависших приложений, закрытия нежелательных программ перед выполнением критических операций (например, перед заменой файла, который может быть занят процессом), а также для управления автоматизируемыми приложениями.

Параметры блока:

- Имя процесса – обязательный параметр. Указывает имя процесса, который необходимо остановить. Тип данных: str (строка). Имя обычно указывается без пути к исполняемому файлу и без расширения .exe.
- Только для текущего пользователя – необязательный параметр. Логический флажок (checkbox), определяющий, завершать ли процессы только текущего пользователя или всех пользователей системы. Тип данных: bool (логический). Возможные значения: Да (флажок установлен) – завершаются только экземпляры процесса, запущенные от имени текущего пользователя ОС. Нет (флажок снят) – завершаются все экземпляры процесса (требуются права администратора для чужих процессов).

Возвращаемое значение: Блок обычно не возвращает значения. При успешном завершении процессов (или если процессы не были найдены) блок завершается без ошибок. При возникновении ошибки (недостаточно прав, процесс защищён, имя процесса указано неверно) генерируется исключение с соответствую-щим сообщением.

Блок «Отправить ПК в консольный режим»



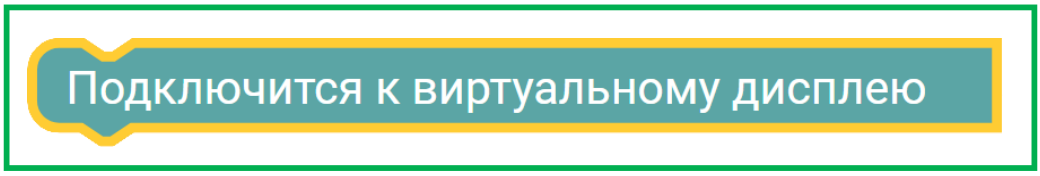
Блок предназначен для перевода персонального компьютера в консольный режим (режим локальной консоли). После выполнения блока текущий сеанс удалённого рабочего стола (RDP) завершается, окно подключения закрывается, а компьютер переходит в состояние, аналогичное локальному входу пользователя.

Этот блок особенно важен для роботов, которые выполняют операции, требую-щие наличия активного графического сеанса (UI-автоматизация, работа с окнами, эмуляция ввода с клавиатуры и мыши).

Параметры блока:

- Параметр «Имя пользователя» позволяет указать учётную запись, от имени ко-торой выполняется переход в консольный режим. Это может быть полезно, ес-ли на ПК одновременно работают несколько пользователей (например, через разные RDP-сеансы), и требуется завершить сеанс конкретного пользователя, а не текущего.
- Параметр «Установить расширение экрана» задаёт желаемое разрешение гра-фического интерфейса после перехода в консольный режим. Значения 1920 и 1080 соответствуют ширине и высоте экрана в пикселях. Это влияет на отобра-жение окон и элементов интерфейса при выполнении UI-действий.
- Возвращаемое значение: Блок не возвращает значения. При успешном выпол-нении (или если ПК уже был в консольном режиме) блок завершается без оши-бок. В случае возникновения ошибки (например, отсутствие прав, проблемы с завершением RDP-сеанса) генерируется исключение.

Блок «Подключиться к виртуальному дисплею»

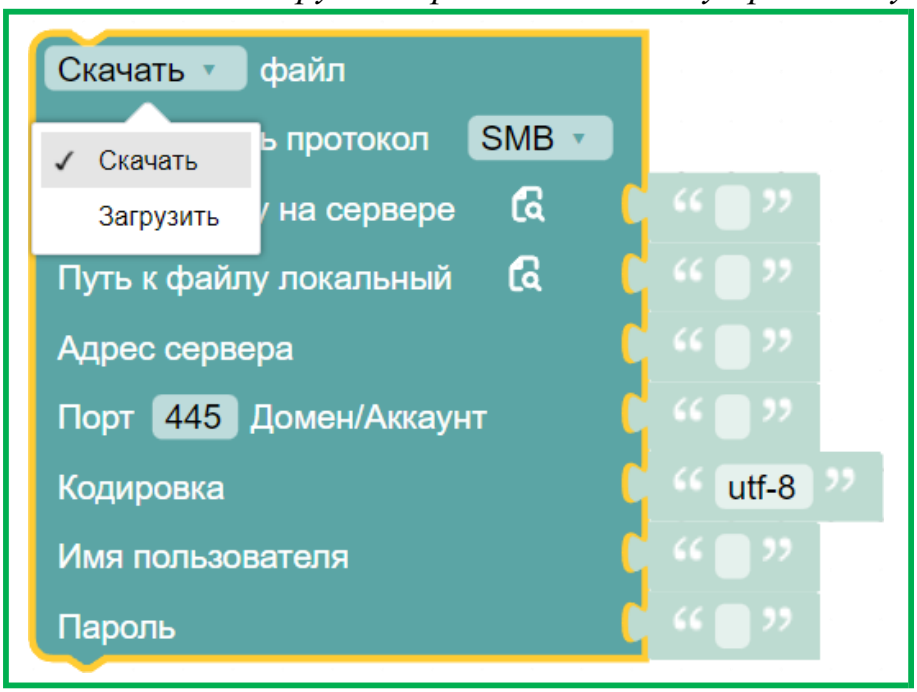


Блок предназначен для подключения и использования виртуального дисплея в операционной системе Linux. Он создаёт виртуальную графическую среду, ко-торая позволяет роботу выполнять действия, требующие наличия активного гра-фического интерфейса (UI-автоматизация, работа с десктопными приложениями, запуск браузера в видимом режиме), даже если на сервере физически отсутствует монитор или графическая подсистема. Блок особенно актуален при запуске робо-тов через оркестратор (например, Airflow) на серверах Linux без графического окружения.

Параметры блока:

- Блок не имеет параметров. Он автоматически создаёт виртуальный дисплей с настройками по умолчанию (например, разрешение 1920x1080, цветовая глуби-на 24 бита).
- Возвращаемое значение: Блок не возвращает значения. При успешном подклю-чении к виртуальному дисплею блок завершается без ошибок. В случае невоз-можности запуска виртуального дисплея (например, отсутствуют необходимые библиотеки Xvfb, недостаточно прав, уже занят указанный номер дисплея) ге-нерируется ошибка с соответствующим сообщением.

Блок «Скачать загрузить файл по сетевому протоколу»



Блок предназначен для обмена файлами с удалённым сервером по сетевым протоколам SMB (Server Message Block) или FTP (File Transfer Protocol). Он позволяет как скачивать файлы с сервера на локальный компьютер, так и загружать файлы с локального компьютера на сервер. Блок полезен для интеграции с корпоративными файловыми хранилищами (сетевые папки Windows), FTP-серверами, автоматического получения входящих документов или сохранения результатов обработки.

Параметры блока:

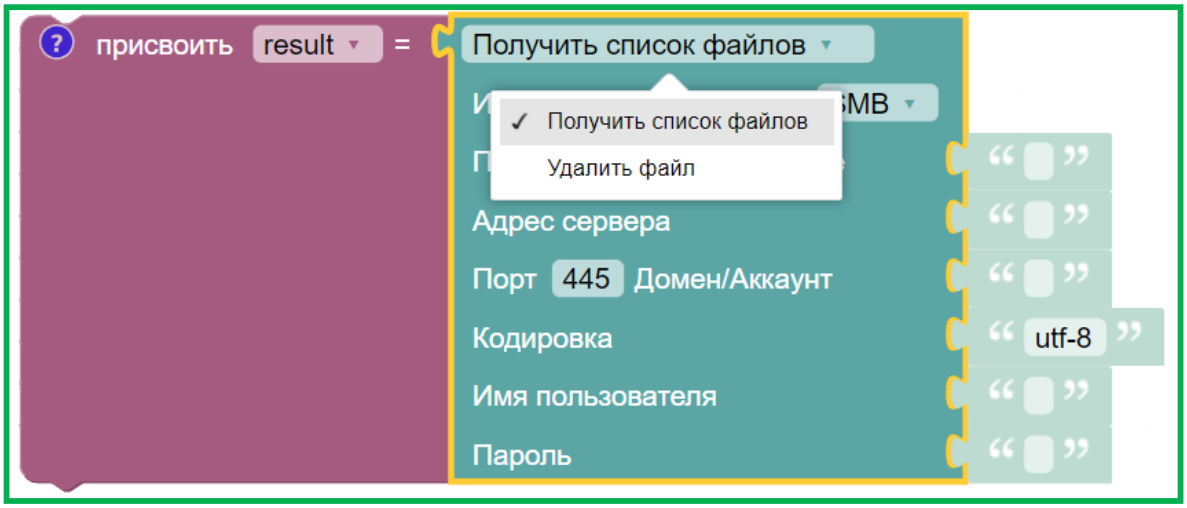
- Тип взаимодействия с файлами – выпадающий список, определяющий направление передачи.
- Тип данных: dropdown. Возможные значения: Скачать, Загрузить. Пример: Скачать.
- Протокол – выпадающий список, выбирающий протокол соединения.
- Тип данных: dropdown. Возможные значения: SMB, FTP. Пример: SMB.
- Путь к файлу локальный – локальный путь к файлу. При скачивании – куда сохранить (с именем файла). При загрузке – откуда взять файл для отправки. Тип данных: str.
- Адрес сервера – IP-адрес или доменное имя сервера. Тип данных: str. Пример: 192.168.1.100 или ftp.example.com.
- Порт – порт для подключения. Для SMB по умолчанию 445, для FTP – 21. Тип данных: int. Пример: 21.
- Домен/Аккаунт – (только для SMB) домен Windows или имя учётной записи. Тип данных: str. Пример: CORP или CORP\ivanov.
- Кодировка – кодировка для имён файлов и путей (актуально для SMB и FTP при работе с не-ASCII именами). Тип данных: str. Пример: UTF-8, windows-1251.

- Имя пользователя – имя пользователя для аутентификации. Для FTP может быть anonymous при анонимном доступе. Тип данных: str. Пример: ivan.ivanov.
- Пароль – пароль для аутентификации. Тип данных: str. Рекомендуется использовать защищённые переменные или хранилище учётных данных.

Дополнительные параметры (в зависимости от реализации):

- Пассивный режим (для FTP) – флажок, включающий пассивный режим работы. Обычно включён по умолчанию. Тип данных: bool.
- Шифрование (для FTP) – выбор типа шифрования (явное/неявное TLS), если поддерживается.
- Возвращаемое значение: Блок не возвращает значения. При успешной операции блок завершается без ошибок. При ошибке (недоступность сервера, неверные учётные данные, отсутствие файла, недостаток прав) генерируется исключение.

Блок «Получить список файлов / Удалить файл на удаленном сервере»



Блок предназначен для выполнения операций с файлами на удалённом сервере по сетевым протоколам SMB или FTP. В зависимости от выбранного режима блок позволяет либо получить список файлов в указанной папке, либо удалить указанный файл. Это полезно для автоматизации работы с корпоративными файловыми хранилищами (сетевые папки Windows) и FTP-серверами: мониторинг новых файлов, очистка устаревших данных, удаление временных файлов после обработки.

Параметры блока:

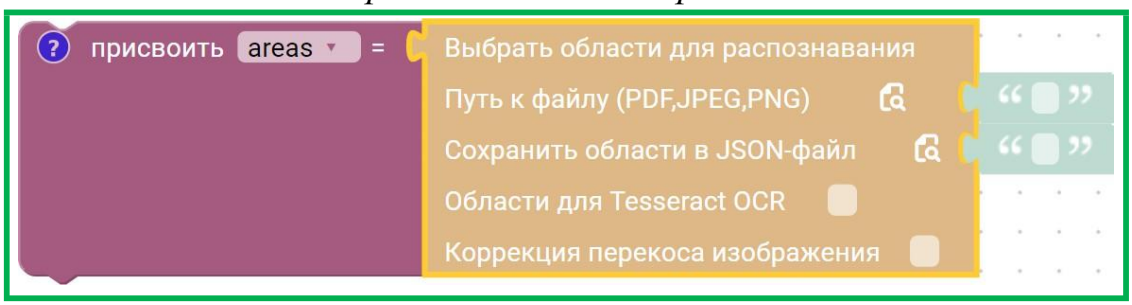
- Тип взаимодействия с файлами – выпадающий список, определяющий операцию.Тип данных: dropdown. Возможные значения: Получить список файлов, Удалить файл. Пример: Получить список файлов.
- Протокол – выпадающий список, выбирающий протокол соединения. Тип данных: dropdown. Возможные значения: SMB, FTP. Пример: SMB.
- Путь к папке / файлу на сервере – путь в формате, понятном для выбранного протокола. Для SMB может быть \\server\share\folder или относительный путь внутри шары. Для FTP – абсолютный путь от корня или относительный от домашней директории пользователя. Тип данных: str (строка). Пример (SMB): \\192.168.1.100\share\documents. Пример (FTP): /incoming/reports.

- Адрес сервера – IP-адрес или доменное имя сервера. Тип данных: str. Пример: 192.168.1.100 или ftp.example.com.
- Порт – порт для подключения. Для SMB по умолчанию 445, для FTP – 21. Тип данных: int. Пример: 445.
- Домен/Аккаунт – (только для SMB) домен Windows или имя учётной записи. Тип данных: str. Пример: CORP или CORP\ivanov. Может быть пустым.
- Кодировка – кодировка для имён файлов и путей. Тип данных: str. Пример: UTF-8, windows-1251.
- Имя пользователя – имя пользователя для аутентификации. Тип данных: str. Для анонимного FTP можно указать anonymous. Пример: ivan.ivanov.
- Пароль – пароль для аутентификации. Тип данных: str. Рекомендуется использовать защищённые переменные.
- Возвращаемое значение: В режиме «Получить список файлов» – блок возвращает список строк (list), содержащий имена файлов (и, возможно, папок) в указанной директории. Если папка пуста, возвращается пустой список. В режиме «Удалить файл» – блок обычно не возвращает значения. При успешном удалении завершается без ошибок. Если файл не найден или недостаточно прав, генерируется ошибка.

Теперь, после того как мы подробно разобрали блоки для работы с файловой системой, буфером обмена, учётными записями, процессами, параллельными вычислениями и сетевыми протоколами (SMB/FTP), мы переходим к следующему крупному разделу – «Обработка документов».

В этом разделе будут рассмотрены блоки, предназначенные для автоматизации работы с документами различных форматов: извлечение текста и таблиц, чтение и запись метаданных, конвертация, создание и модификация файлов. Эти блоки являются основой для роботов, которые обрабатывают входящие докумен-ты, формируют отчёты, извлекают структурированные данные из неструктурированных источников и выполняют другие задачи, связанные с корпоративным документооборотом.

Блок «Выбрать области для распознавания»



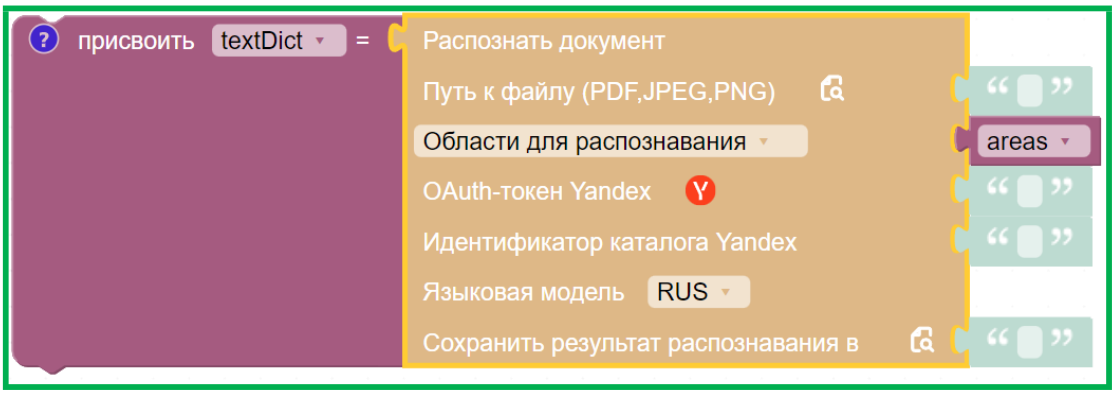
Блок предназначен для интерактивного или программного выбора областей на документе (изображении или PDF-файле) с целью последующего оптического распознавания символов (OCR). Выбранные области могут быть сохранены в JSON-файл для повторного использования (например, при массовой обработке однотипных документов). Блок также позволяет включить коррекцию перекоса изображения для повышения качества распознавания и отобрать области, которые

будут передаваться в Tesseract OCR. Это полезно для автоматизации извлечения данных из счёт-фактур, накладных, анкет и других структурированных документов, где заранее известны зоны расположения нужных полей (сумма, дата, номер документа и т.п.).

Параметры блока:

- Путь к файлу (PDF, JPG, PNG) – обязательный параметр. Указывает полный путь к документу, в котором необходимо выбрать области для распознавания. Тип данных: str (строка). Поддерживаются форматы PDF, JPEG, PNG.
- Сохранить области в JSON-файл – необязательный параметр. Указывает путь к JSON-файлу, в который будут сохранены координаты и параметры выбранных областей. Если файл уже существует, он будет перезаписан. Если параметр не указан, области не сохраняются, но могут быть возвращены как результат работы блока. Тип данных: str (строка). Пример: C:\docs\invoice.json.
- Области для Tesseract OCR – необязательный параметр. Чекбокс, который определяет, следует ли отмечать выбранные области как предназначенные для распознавания через Tesseract OCR. Если флажок установлен, блок добавляет соответствующую метку к каждой области. Тип данных: bool (логический), визуально – чекбокс.
- Коррекция перекоса изображения – необязательный параметр. Чекбокс, включающий автоматическое выравнивание документа перед выбором областей. Рекомендуется включать для сканированных копий, особенно если они могли быть положены под углом. Тип данных: bool (логический), визуально – чекбокс.
- Возвращаемое значение: Блок возвращает список областей (list), каждая об-ласть представлена в виде словаря или структуры, содержащей координаты (например, x, y, width, height), возможно номер страницы (для PDF) и флаг use_for_tesseract (если был установлен соответствующий чекбокс). Этот список может быть использован в последующих блоках распознавания. Если области не были выбраны (пользователь закрыл окно без выбора), возвращается пустой список.

Блок «Распознать области»



Блок предназначен для автоматического извлечения текстовой информации из документов (изображений и PDF) с использованием облачных технологий компьютерного зрения Yandex Vision. Он позволяет гибко настраивать процесс

распознавания: можно либо заранее указать конкретные прямоугольные области документа, которые необходимо обработать, либо задать ключевые слова-якоря, по которым блок сам определит нужные фрагменты текста.

Параметры блока:

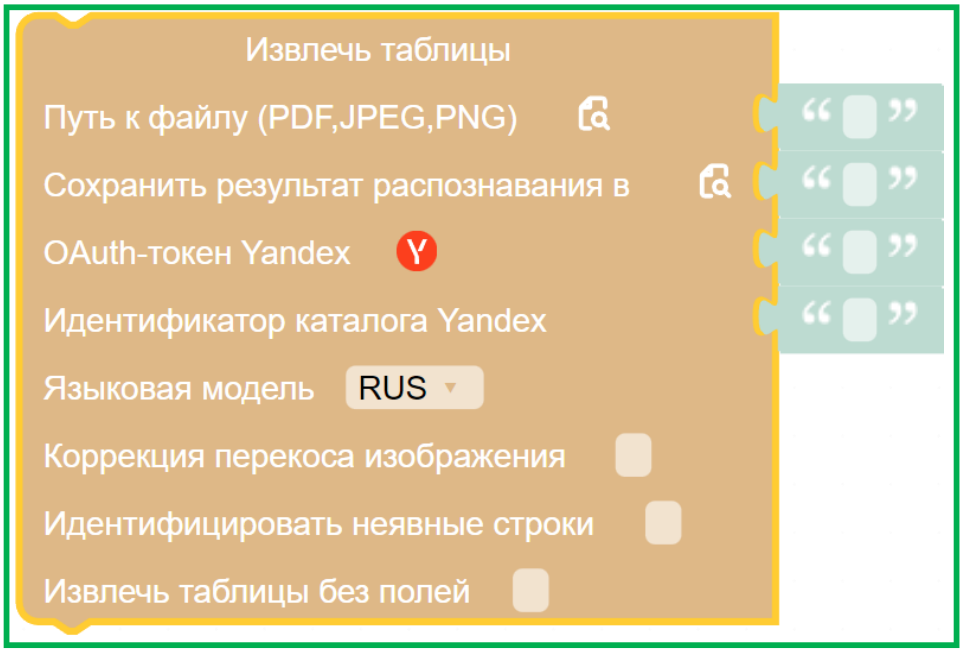
- Файл документа – обязательный параметр. Путь к входному файлу (поддерживаются форматы PDF, JPG, PNG). Тип данных: str (строка). Пример: C:\docs\invoice.jpg.
- Режим распознавания – обязательный параметр. Выпадающий список, определяющий способ извлечения данных. Возможные значения: области для распознавания, ключевые слова. Тип данных: str (строка).
- Параметры поиска (области для распознавания) – заполняется, если выбран режим «области для распознавания». Представляет собой словарь (dict), где ключом является номер страницы документа (для PDF), а значением – список координат прямоугольных областей в формате [(x1, y1), (x2, y2)]. Тип данных: dict (словарь).
- Параметры поиска (ключевые слова) – заполняется, если выбран режим «ключевые слова». Список ключевых слов (list) для поиска в документе. Каждый элемент списка может быть строкой (извлечение текста от этого слова до конца строки) или списком из двух строк (извлечение текста между этими словами). Тип данных: list (список).
- Языковая модель – обязательный параметр. Выпадающий список, определяющий язык распознавания. Возможные значения: RUS, ENG, RUS+ENG (многоязычный режим). Тип данных: str (строка).
- Путь к файлу результата – обязательный параметр. Путь для сохранения результата распознавания в формате JSON. Тип данных: str (строка).
- OAuth-токен Yandex – обязательный параметр. Токен доступа к API Yandex Vision. Тип данных: str (строка).
- Идентификатор каталога Yandex – обязательный параметр. ID каталога в Yandex Cloud, на который у вашего аккаунта есть роль ai.vision.user или выше. Тип данных: str (строка).
- Возвращаемое значение: Блок возвращает результат распознавания в виде структурированных данных. Формат возвращаемого значения зависит от режима распознавания, но в любом случае он может быть сохранён в JSON-файл по указанному пути.

Для работы блока требуется доступ к Yandex Vision. Необходимо:

- Зарегистрироваться в Yandex Cloud и создать платёжный аккаунт (доступен пробный период с грантом).
- Получить OAuth-токен Yandex.
- Получить идентификатор каталога в консоли управления Yandex Cloud.

Подробная инструкция по регистрации приведена в официальной документации Puzzle RPA.

Блок «Извлечь таблицы»



Блок предназначен для автоматического обнаружения и извлечения таблиц из документов (PDF, JPG, PNG) с последующим сохранением их в Excel-файл. Каждая распознанная таблица помещается на отдельный лист книги Excel. Блок использует облачные технологии компьютерного зрения Yandex Vision, что поз-воляет распознавать таблицы даже в отсканированных документах, фотографиях и PDF-файлах без текстового слоя. Это незаменимый инструмент для автоматиза-ции ввода табличных данных из накладных, актов, счетов-фактур, отчётов и дру-гих документов, где информация представлена в виде таблиц.

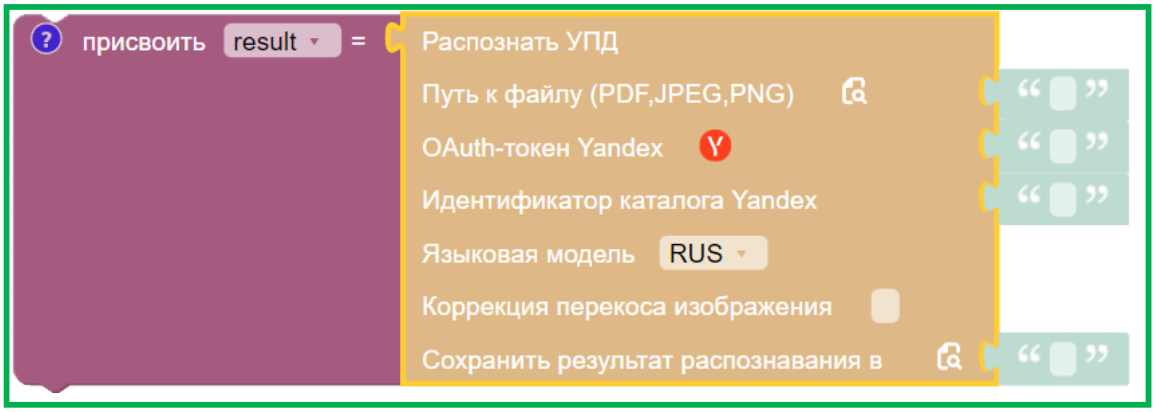
Параметры блока:

- Путь к файлу (PDF, JPG, PNG) – обязательный параметр. Указывает полный путь к входному документу, из которого необходимо извлечь таблицы. Тип данных: str (строка).
- Сохранить результат распознавания в – обязательный параметр. Путь к Excel-файлу, в который будут сохранены извлечённые таблицы. Каждая таблица за-писывается на отдельный лист. Тип данных: str (строка).
- OAuth-токен Yandex – обязательный параметр. Токен доступа к API Yandex Vision. Тип данных: str (строка).
- Идентификатор каталога Yandex – обязательный параметр. ID каталога в Yandex Cloud, на который ваш аккаунт имеет права ai.vision.user или выше. Тип данных: str (строка).
- Языковая модель – обязательный параметр. Выпадающий список, определяю-щий язык распознавания текста внутри таблиц. Возможные значения: RUS (только русский), ENG (только английский), RUS+ENG (автоматическое опре-деление языка для каждого фрагмента). Тип данных: str (строка).
- Коррекция перекоса изображения – необязательный параметр. Чекбокс, вклю-чающий автоматическое выравнивание документа перед распознаванием. Ре-комендуется устанавливать для сканированных или сфотографированных до-кументов. Тип данных: bool (логический).

- Идентифицировать неявные строки – необязательный параметр. Чекбокс, включающий режим выделения строк таблицы при отсутствии явных разделителей (границ). Тип данных: bool (логический).
- Извлекать таблицы без полей – необязательный параметр. Чекбокс, включающий режим распознавания таблиц, не имеющих явной разметки (без рамок и сетки). Тип данных: bool (логический).
- Возвращаемое значение: Блок обычно не возвращает значения в переменную, либо возвращает путь к созданному Excel-файлу (как подтверждение успешного выполнения). Основной результат – это сохранённый Excel-файл с извлечёнными таблицами. В случае ошибки (недоступность сервера, неверный токен, отсутствие входного файла, проблемы с распознаванием) генерируется исключение.

Для работы блока также требуется доступ к Yandex Vision.

Блок «Распознать УПД»



Блок предназначен для автоматического распознавания таблиц из универсального передаточного документа (УПД) – документа, который объединяет функции счёт-фактуры и первичного учётного документа (накладной, акта). Блок использует облачный сервис Yandex Vision для извлечения табличных данных (позиции товаров/услуг, количество, цена, сумма и т.д.) и сохраняет результат в Excel-файл. Это специализированный инструмент для бухгалтерской и складской автоматизации, позволяющий быстро вводить данные из УПД в учётные системы без ручного набора.

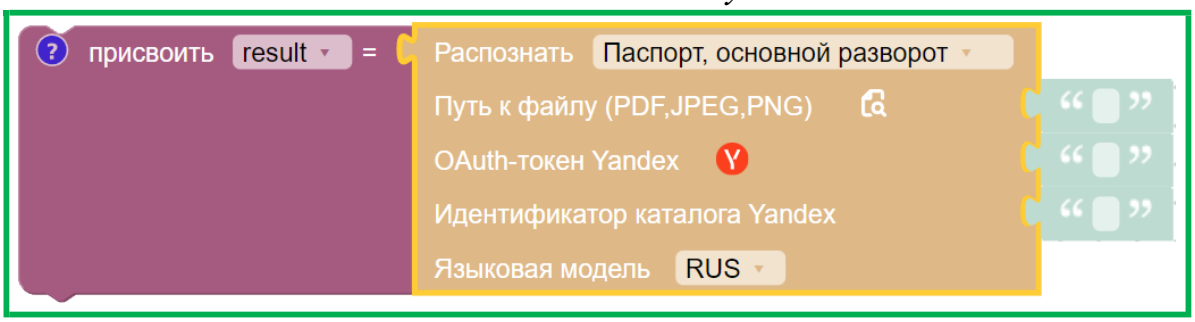
Параметры блока:

- Файл документа – обязательный параметр. Путь к входному файлу УПД (JPEG, PNG или PDF). Тип данных: str (строка). Пример: C:\docs\upd.pdf.
- Путь для сохранения результата распознавания в формате Excel (XLSX). Тип данных: str (строка). Пример: C:\docs\upd_result.xlsx.
- Коррекция перекоса изображения – необязательный параметр. Чекбокс, включающий автоматическое выравнивание документа. Тип данных: bool (логический).
- Языковая модель – обязательный параметр. Выпадающий список для выбора языка распознавания. Возможные значения: RUS, ENG, RUS+ENG. Тип дан-ных: str (строка). Для УПД обычно RUS.

- OAuth-токен Yandex – обязательный параметр. Токен доступа к API Yandex Vision. Тип данных: str (строка).
- Идентификатор каталога Yandex – обязательный параметр. ID каталога в Yandex Cloud, на который ваш аккаунт имеет права. Тип данных: str (строка).
- Возвращаемое значение: Блок возвращает словарь (dict), содержащий результат распознавания. Обычно в словаре есть ключ "tables", значение которого – список распознанных таблиц (каждая таблица представлена в виде структуры данных, например, списка строк и колонок).

Для работы блока также требуется доступ к Yandex Vision.

Блок «Распознать документ»



Блок предназначен для автоматического распознавания структурированных документов удостоверительного типа с использованием облачного сервиса Yandex Vision. Он анализирует изображение или PDF-файл, извлекает ключевые поля (серия, номер, ФИО, дата выдачи и т.д.) и возвращает результат в виде структурированного словаря. Блок поддерживает три типа документов: паспорт РФ (основной разворот), водительское удостоверение (лицевая и оборотная сто-роны) и свидетельство о регистрации транспортного средства (лицевая и оборот-ная стороны). Это незаменимый инструмент для автоматизации процессов верификации личности, регистрации клиентов, заполнения анкет и других задач, требующих быстрого ввода данных из удостоверяющих документов.

Параметры блока:

- Тип документа – обязательный параметр. Выпадающий список, определяющий шаблон распознавания. Тип данных: str (строка). Возможные значения: Паспорт РФ (основной разворот), Водительское удостоверение, Свидетельство о регистрации ТС. Пример: Паспорт РФ (основной разворот).
- Файл документа – обязательный параметр. Полный путь к файлу изображения (JPEG, PNG) или PDF, подлежащему распознаванию. Тип данных: str (строка).
- OAuth-токен Yandex – обязательный параметр. Токен доступа к API Yandex Vision. Тип данных: str (строка).
- Идентификатор каталога Yandex – обязательный параметр. ID каталога в Yandex Cloud, на который ваш аккаунт имеет права. Тип данных: str (строка).
- Языковая модель – обязательный параметр. Выпадающий список для выбора языка распознавания. Тип данных: str (строка). Возможные значения: RUS, ENG, RUS+ENG. Пример: RUS.

- Возвращаемое значение: Блок возвращает словарь (dict) с результатом распознавания. Основной ключ – "fields", содержащий вложенный словарь с извлечёнными полями. Набор полей зависит от выбранного типа документа.

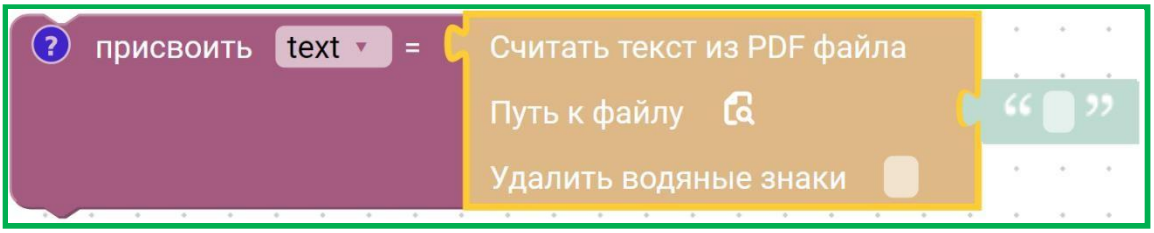
Пример для паспорта РФ:

```
{"fields": {"Фамилия": "ИВАНОВ", "Имя": "ИВАН", "Отчество": "ИВАНОВИЧ", "Серия": "1234", "Номер": "567890", "Дата выдачи": "01.01.2010", "Код подразделения": "123-456"}}
```

Пример для водительского удостоверения (условный):

```
{"fields": {"Фамилия": "ПЕТРОВ", "Имя": "АЛЕКСЕЙ", "Дата рождения": "15.05.1985", "Серия и номер": "12 34 567890", "Дата выдачи": "10.10.2015", "Срок действия": "10.10.2025", "Категории": "B,C"}}
```

Блок «Считать текст из PDF файла»



Блок предназначен для извлечения текстового содержимого из PDF-файла. Он считывает все текстовые данные, содержащиеся в документе, и возвращает их в виде единой строки. Дополнительно блок может удалять повторяющиеся водя-ные знаки, которые часто мешают при дальнейшей обработке текста. Это полезно для автоматизации анализа текстовых документов (договоров, отчётов, статей), поиска ключевых слов, извлечения фактов, а также для подготовки текста к обра-ботке другими блоками (например, «Найти и заменить», «Регулярное выраже-ние»).

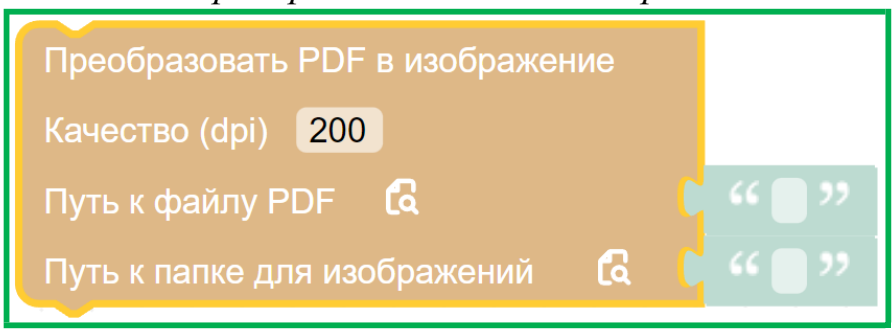
Параметры блока:

- Путь к файлу PDF – обязательный параметр. Указывает полный путь к PDF-файлу, из которого необходимо извлечь текст. Тип данных: str (строка). При-мер: C:\Users\User\Desktop\document.pdf.
- Удалить водяные знаки – необязательный параметр. Чекбокс, включающий удаление повторяющихся водяных знаков из извлечённого текста. Тип данных: bool (логический). Возможные значения: установлен (истина) – водяные знаки удаляются; снят (ложь) – текст возвращается без изменений.

Заметка: Если параметр отключён, блок возвращает исходный извлечённый текст без дополнительной очистки.

- Возвращаемое значение: Блок возвращает строку (str), содержащую весь извлечённый из PDF-файла текст. Если документ пуст или не содержит текстового слоя, возвращается пустая строка (""). При включённом удалении водяных знаков возвращается очищенный текст. Пример возвращаемого значения: "Уважаемые клиенты! Настоящий договор регулирует отношения между сторонами... (и далее текст документа)"

Блок «Преобразовать PDF в изображение»

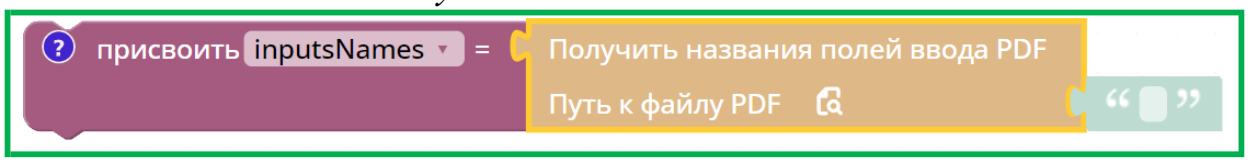


Блок предназначен для конвертации страниц PDF-документа в растровые изображения формата PNG. Каждая страница исходного PDF-файла сохраняется в отдельный PNG-файл в указанной пользователем папке. Качество (разрешение) выходных изображений задаётся в DPI (точек на дюйм). Этот блок незаменим в сценариях, где требуется дальнейшая обработка страниц PDF с помощью инструментов компьютерного зрения (например, распознавание областей, извлечение таблиц, распознавание документов), которые работают только с изображениями, а не с PDF напрямую.

Параметры блока:

- DPI (качество / разрешение) – обязательный параметр. Разрешение выходных изображений в точках на дюйм. Тип данных: int (целое число). Диапазон значений: от 72 до 3000. Значение по умолчанию: 200. Пример: 200.
- Путь к файлу PDF – обязательный параметр. Полный путь к исходному PDF-файлу, который необходимо конвертировать. Тип данных: str (строка). Пример: C:\Users\User\Desktop\source.pdf.
- Путь к папке для изображений – обязательный параметр. Путь к каталогу, в который будут сохранены полученные PNG-файлы. Если папка не существует, блок создаёт её автоматически. Тип данных: str (строка). Пример: C:\Users\User\Desktop\pdf_pages_png.
- Возвращаемое значение: Блок возвращает список строк (list), содержащий полные пути ко всем созданным PNG-файлам. Порядок элементов списка соответствует порядку страниц в исходном PDF. Если PDF не содержит страниц (пустой документ), возвращается пустой список. Это удобно для последующей обработки: например, можно передать полученные пути в блок «Распознать области» или «Извлечь таблицы» для каждой страницы.

Блок «Получить названия полей ввода PDF»



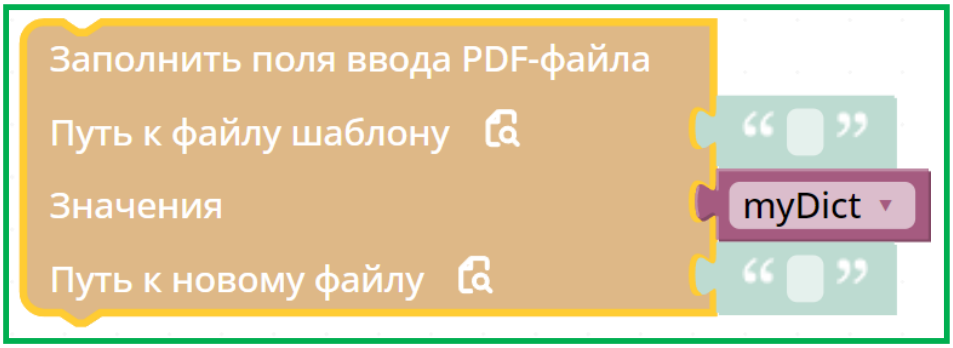
Блок предназначен для извлечения информации из интерактивных PDF-форм (форм с полями ввода, такими как текстовые поля, чекбоксы, радиокнопки, выпадающие списки). Он возвращает словарь, в котором ключами являются имена полей, а значениями – текущее содержимое этих полей в файле. Это полезно для автоматизации обработки заполненных анкет, опросных листов, заявок, налоговых

деклараций и других документов, где данные введены в интерактивные поля PDF. Робот может считать введённые пользователем значения и передать их в учётную систему, базу данных или для дальнейшего анализа.

Параметры блока:

- Путь к файлу PDF – обязательный параметр. Полный путь к PDF-файлу, содержащему интерактивную форму. Тип данных: str (строка). Пример: C:\Users\User\Desktop\form.pdf.
- Возвращаемое значение: Блок возвращает словарь (dict), где каждый ключ – имя поля ввода (строка), а значение – текущее содержимое поля (строка, число или булево значение в зависимости от типа поля). Если в PDF нет полей формы, возвращается пустой словарь {}.

Блок «Заполнить поля ввода PDF-файла»



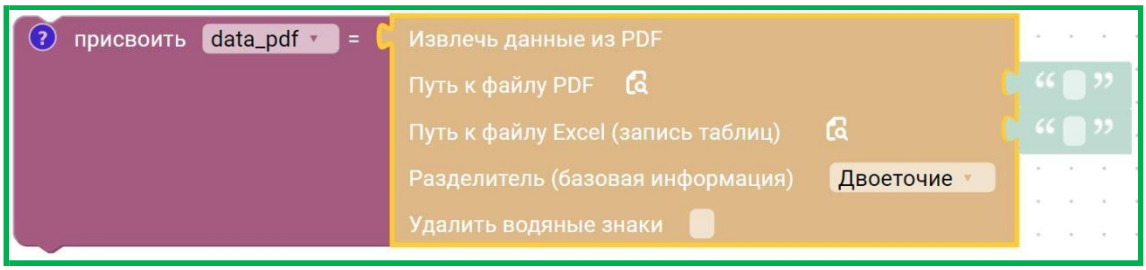
Блок предназначен для автоматического заполнения интерактивных PDF-форм. Он принимает шаблон PDF-файла с полями ввода и словарь, содержащий пары «имя поля → значение», а затем сохраняет заполненную копию в новый файл. Это незаменимый инструмент для автоматизации создания типовых доку-ментов: договоров, заявлений, анкет, налоговых деклараций, счёт-фактур и дру-гих форм, где требуется подставить данные из учётной системы (1С, CRM, базы данных) в готовый бланк.

Параметры блока:

- Путь к файлу шаблону – обязательный параметр. Полный путь к исходному PDF-файлу, содержащему интерактивные поля формы. Тип данных: str (стро-ка). Пример: C:\Users\User\Desktop\form.pdf.
- Значения полей (словарь) – обязательный параметр. Словарь, в котором ключи – имена полей (строки), значения – данные для заполнения (строки, числа или булевы значения, которые будут приведены к строке). Тип данных: dict (сло-варь).
- Путь к новому файлу – обязательный параметр. Полный путь, по которому бу-дет сохранена заполненная копия PDF-файла. Если файл с таким именем уже существует, он будет перезаписан (поведение может зависеть от платформы). Тип данных: str (строка). Пример: C:\Users\User\Desktop\form_filled.pdf.
- Возвращаемое значение: Блок обычно не возвращает значения (либо возвраща-ет путь к созданному файлу в виде строки). При успешном заполнении и сохра-нении блок завершается без ошибок. В случае ошибки (неверный путь, отсут-

ствие полей, несовпадение имён, проблемы с записью) генерируется исключение.

Блок «Извлечь данные из PDF»

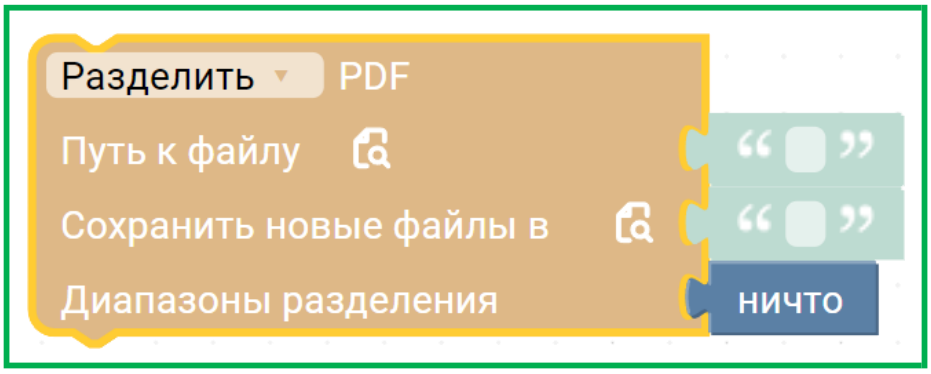


Блок предназначен для автоматического извлечения структурированных данных из PDF-файлов, содержащих текстовый слой. Он выполняет две основные задачи: во-первых, извлекает базовую текстовую информацию, разделяя её на пары «реквизит – значение» по заданному разделителю (двоеточие или тире); во-вторых, обнаруживает и извлекает таблицы, сохраняя их в Excel-файл (каждая таблица на отдельном листе, имя листа соответствует имени таблицы). Это мощ-ный инструмент для обработки счетов, накладных, актов, отчётов и других доку-ментов, где присутствуют как ключевые поля (сумма, дата, номер), так и таблич-ные данные (перечень товаров, услуг).

Параметры блока:

- Путь к файлу PDF – обязательный параметр. Полный путь к исходному PDF-документу, из которого требуется извлечь данные. Тип данных: str (строка). Пример: C:\Users\User\Desktop\test.pdf.
- Путь к файлу Excel (запись таблиц) – обязательный параметр. Путь для сохранения извлечённых таблиц в формате Excel (XLSX). Если файл уже существует, он будет перезаписан. Тип данных: str (строка). Пример: C:\Users\User\Desktop\test.xlsx.
- Разделитель (базовая информация) – обязательный параметр. Символ, исполь-зуемый для разделения реквизита и значения в тексте. Возможные значения: двоеточие (:) или тире (-). Тип данных: str (строка).
- Удалить водяные знаки – необязательный параметр. Чекбокс, включающий удаление повторяющихся водяных знаков из текста перед извлечением. Тип данных: bool (логический).
- Возвращаемое значение: Блок возвращает список (list), содержащий ровно два словаря (dict): Первый словарь – базовая информация. Ключи – извлечённые реквизиты (названия полей), значения – соответствующие им значения из документа. Если пар «реквизит-значение» не найдено, словарь будет пустым. Второй словарь – табличные данные. Ключи – имена таблиц (заголовки, определённые в документе), значения – таблицы данных. Формат таблицы зависит от реализации платформы, обычно это DataFrame (pandas) или аналогичная структура, которую можно далее обрабатывать.

Блок «Разделить / объединить PDF»

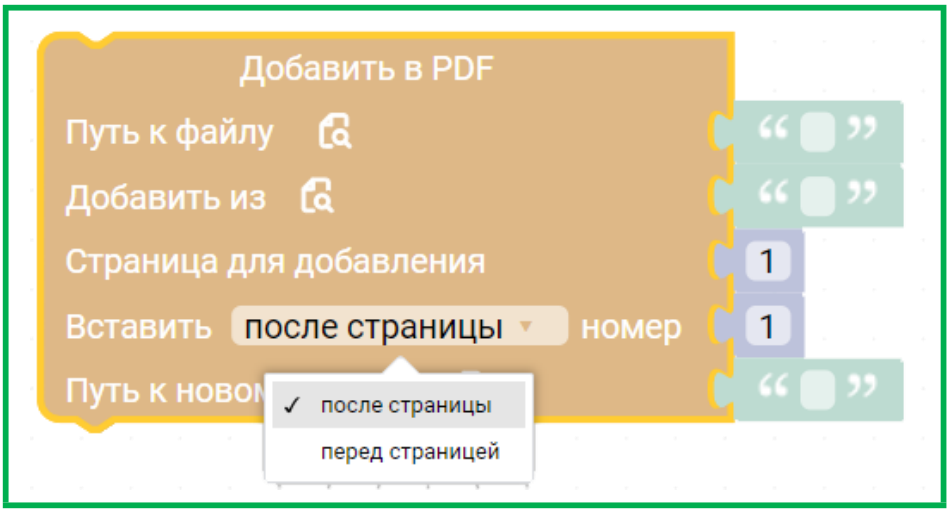


Блок предназначен для выполнения двух противоположных операций с PDF-документами: объединения нескольких PDF-файлов из одной папки в единый документ или разделения одного PDF-файла на части по заданным правилам (постранично, выделение отдельных страниц, диапазоны). Это полезно для подготовки документов к отправке (склеивание нескольких сканов в один отчёт), для извлечения отдельных страниц из большого документа (например, только подписанный лист из договора), для разделения многостраничных счетов-фактур по отдельным файлам или для удаления ненужных страниц.

Параметры блока:

- Режим – обязательный параметр. Выпадающий список, определяющий операцию. Тип данных: str (строка). Возможные значения: merge (объединить), split (разделить).
- Путь к файлу или папке – обязательный параметр. Зависит от режима: При merge – путь к папке, содержащей PDF-файлы для объединения. Тип данных: str. Пример: C:\Users\User\Desktop\Files. При split – путь к исходному PDF-файлу, который нужно разделить. Тип данных: str. Пример: C:\Users\User\Desktop\source.pdf.
- Путь к результату – обязательный параметр. Зависит от режима: При merge – полный путь к итоговому объединённому PDF-файлу (включая имя файла с расширением .pdf). Тип данных: str. Пример: C:\Users\User\Desktop\merged.pdf. При split – путь к папке, в которую будут сохранены разделённые части. Папка будет создана автоматически, если не существует.
- Диапазоны разделения – используется только в режиме разделения. Определяет, какие части создать. Тип данных: None (пустое значение), int (одно число) или list (список).
- Возвращаемое значение: Блок не возвращает значения. При успешном выполнении блок завершается без ошибок. При ошибке (неверный путь, отсутствие файлов, неверный формат диапазонов) генерируется исключение.

Блок «Добавить в PDF»



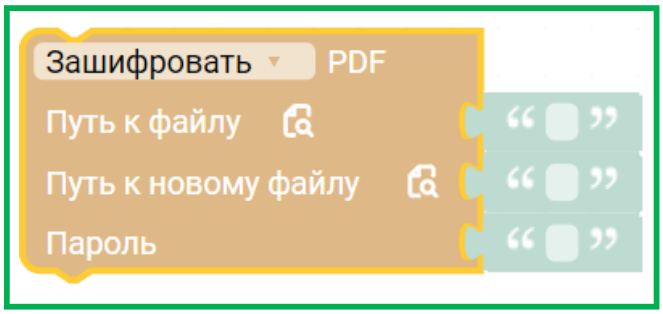
Блок предназначен для вставки страниц из одного PDF-документа в другой. Вы можете указать диапазон или отдельные страницы из файла-источника и вставить их до или после определённой страницы целевого документа. Результат сохраняется в новый PDF-файл, исходные документы остаются без изменений. Это полезно для сборки итоговых отчётов из нескольких частей, добавления приложений к основному документу, вставки отсутствующих страниц в сформированные ранее PDF или для объединения документов со сложной структурой (например, вставить приложение после первой страницы договора, но перед подписями).

Параметры блока:

- Путь к файлу – обязательный параметр. Полный путь к целевому PDF-документу, в который будут добавляться страницы. Тип данных: str (строка). Пример: C:\Users\User\Desktop\target.pdf.
- Добавить из – обязательный параметр. Полный путь к PDF-файлу-источнику, из которого берутся страницы для вставки. Тип данных: str (строка). Пример: C:\Users\User\Desktop\source.pdf.
- Страницы для добавления – обязательный параметр. Диапазон или номера страниц из источника, которые необходимо вставить. Формат ввода определяется студией (обычно строка). Тип данных: str (строка). Пример: 2-4 (страницы со 2 по 4 включительно) или 1,3,5 (страницы 1, 3 и 5). Пример: 2-4.
- Вставить после / перед – обязательный параметр. Выпадающий список, определяющий позицию вставки относительно страницы-ориентира. Тип данных: str (строка). Возможные значения: after (после), before (перед). Пример: after.
- Номер страницы (ориентир) – обязательный параметр. Номер страницы в целевом документе, относительно которой выполняется вставка. Тип данных: str (строка) или int (число) – в зависимости от реализации. Нумерация страниц начинается с 1.
- Путь к новому PDF – обязательный параметр. Полный путь для сохранения результирующего PDF-файла. Тип данных: str (строка). Пример: C:\Users\User\Desktop\result.pdf.
- Возвращаемое значение: Блок не возвращает значения. При успешной вставке и сохранении блок завершается без ошибок. В случае ошибки (неверные пути,

несуществующий файл, некорректный диапазон страниц, проблемы с записью) генерируется исключение.

Блок «Зашифровать / Дешифровать PDF»

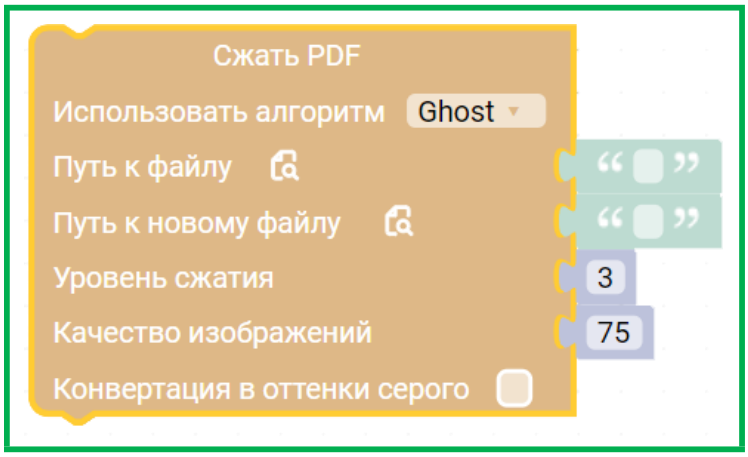


Блок предназначен для наложения парольной защиты на PDF-документ (шифрование) или для снятия пароля с зашифрованного PDF (дешифрование). Результат операции сохраняется в новый файл, исходный документ остаётся без изменений. Шифрование полезно для обеспечения конфиденциальности документов перед отправкой по электронной почте, загрузкой в облачное хранилище или передачей третьим лицам. Дешифрование необходимо для автоматизации работы с защищёнными документами, когда пароль известен и требуется извлечь содержимое (например, для последующего распознавания, извлечения таблиц или текста).

Параметры блока:

- Действие – обязательный параметр. Выпадающий список, определяющий операцию. Тип данных: str (строка). Возможные значения: encrypt (зашифровать), decrypt (расшифровать). Пример: encrypt.
- Путь к файлу – обязательный параметр. Полный путь к исходному PDF-файлу, который нужно зашифровать или расшифровать. Тип данных: str (строка). Пример: C:\Users\User\Desktop\doc.pdf.
- Путь к новому файлу – обязательный параметр. Полный путь для сохранения результата (зашифрованного или расшифрованного PDF). Тип данных: str (строка). Пример: C:\Users\User\Desktop\doc_secured.pdf.
- Пароль – обязательный параметр. При шифровании – пароль, который будет запрошен при открытии документа. При дешифровании – пароль для снятия защиты. Тип данных: str (строка). Пример: MySecretPassword.
- Возвращаемое значение: Блок не возвращает значения. При успешном выполнении блок завершается без ошибок. В случае ошибки (неверный пароль, повреждённый файл, неверный формат, проблемы с записью) генерируется исключение.

Блок «Сжать PDF»



Блок предназначен для уменьшения размера PDF-файла за счёт перекодирования изображений, изменения их разрешения и применения настроек качества. Доступны два алгоритма сжатия: Ghost (на основе Ghostscript) и Pypdf (на основе одноимённой библиотеки). Выбор алгоритма определяет набор доступных параметров. Блок полезен для оптимизации документов перед отправкой по электронной почте, загрузкой в облачные сервисы, публикацией на веб-сайтах или для экономии дискового пространства в архивах.

Параметры блока:

Общие для всех алгоритмов:

- Алгоритм сжатия – обязательный параметр. Выпадающий список, определяющий, какой инструмент использовать. Тип данных: str (строка). Возможные значения: Ghost, Pypdf. Пример: Ghost.
- Путь к исходному PDF – обязательный параметр. Полный путь к файлу, который нужно сжать. Тип данных: str (строка). Пример: C:\Users\User\Desktop\input.pdf.
- Путь к сжатому PDF – обязательный параметр. Полный путь для сохранения результата сжатия. Если файл с таким именем уже существует, он будет перезаписан. Тип данных: str (строка). Пример: C:\Users\User\Desktop\output.pdf.

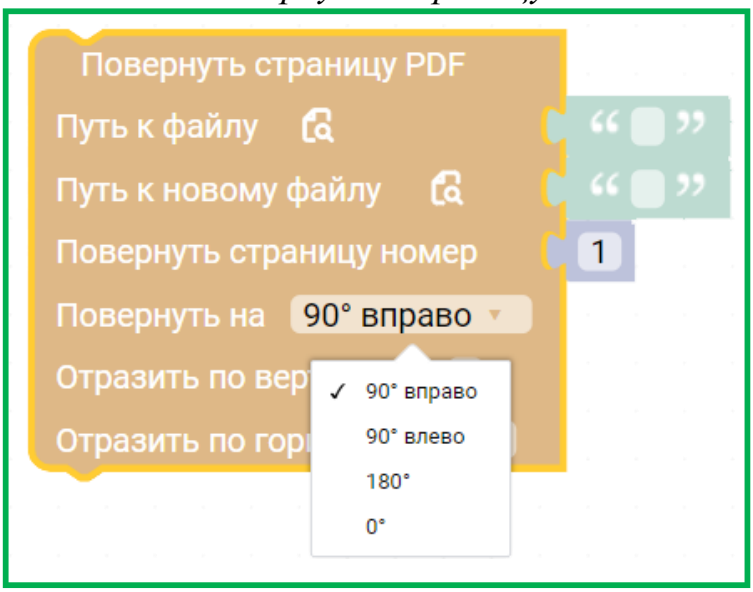
Параметры при выборе алгоритма Ghost:

- Уровень сжатия – степень сжатия от 0 (максимальное качество) до 3 (максимальное сжатие). Тип данных: int (целое число).
- Качество изображений – разрешение изображений в DPI. Тип данных: int (целое число).
- Оттенки серого – чекбокс, переводящий все иллюстрации в градации серого. Тип данных: bool (логический).

Параметры при выборе алгоритма Pypdf:

- Качество изображений – разрешение изображений в DPI. Тип данных: int (це-лое число).
- Возвращаемое значение: Блок обычно не возвращает. При успешном сжатии блок завершается без ошибок. В случае ошибки (неверный путь, отсутствие Ghostscript, повреждённый PDF) генерируется исключение.

Блок «Повернуть страницу PDF»



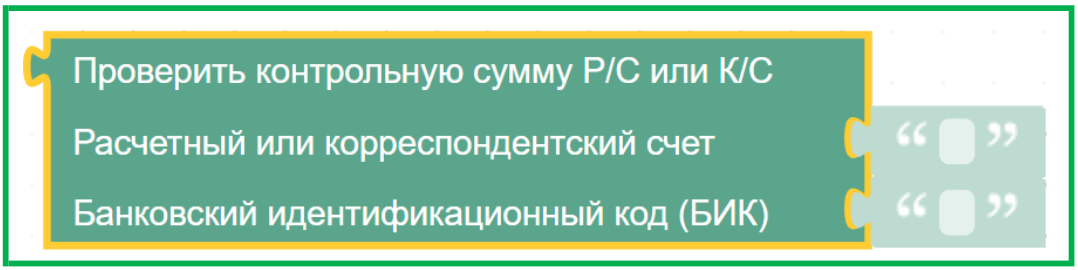
Блок предназначен для поворота и зеркального отражения выбранной страницы PDF-документа. Результат сохраняется в новый файл, исходный документ остаётся неизменным. Это полезно для исправления неправильно ориентированных страниц (например, при сканировании документа вверх ногами или боком), для приведения всех страниц к единой ориентации (книжной или альбомной), а также для зеркального отражения страниц с целью последующего корректного отображения или печати.

Параметры блока:

- Путь к файлу – обязательный параметр. Полный путь к исходному PDF-документу, в котором необходимо повернуть страницу. Тип данных: str (строка). Пример: C:\Users\User\Desktop\doc.pdf.
- Путь к новому файлу – обязательный параметр. Полный путь для сохранения результата (PDF с повёрнутой страницей). Тип данных: str (строка). Пример: C:\Users\User\Desktop\doc_rotated.pdf.
- Повернуть страницу номер – обязательный параметр. Номер страницы (начиная с 1), которую необходимо повернуть. Тип данных: str (строка) или int (число).
- Угол поворота – обязательный параметр. Выпадающий список, определяющий угол и направление поворота. Тип данных: str (строка). Возможные значения: 90 right (90° по часовой), 90 left (90° против часовой), 180 (180°), 0 (без поворо-та). Пример: 90 right.
- Отражение по вертикали – необязательный параметр. Чекбокс, зеркально отражающий страницу сверху вниз (относительно горизонтальной оси). Тип дан-ных: bool (логический).
- Отражение по горизонтали – необязательный параметр. Чекбокс, зеркально отражающий страницу слева направо (относительно вертикальной оси). Тип данных: bool (логический).
- Возвращаемое значение: Блок не возвращает значения. При успешном выполнении блок завершается без ошибок. В случае ошибки (неверный путь, несуще-

ствующий номер страницы, повреждённый PDF, проблемы с записью) генерируется исключение.

Блок «Проверить контрольную сумму Р/С или К/С»

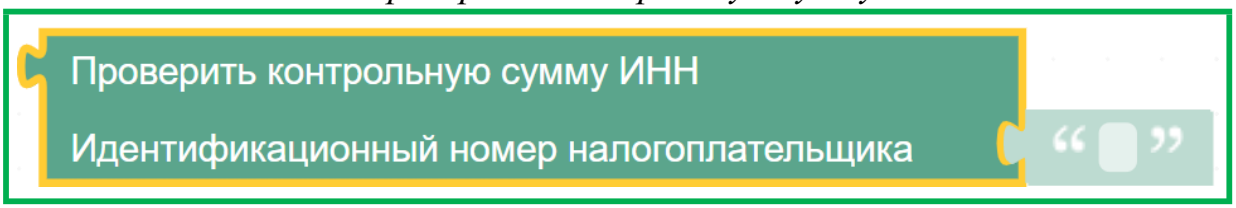


Блок предназначен для проверки контрольной суммы расчётного (р/с) или корреспондентского (к/с) счета на основе банковского идентификационного кода (БИК) банка. Он вычисляет контрольное значение по стандартному алгоритму, принятому в банковской системе Российской Федерации, и возвращает True, если номер счета корректен, или False в противном случае. Это важный инструмент для автоматизации финансовых операций: проверки вводимых реквизитов контрагентов, предотвращения ошибок при переводах, валидации счетов в заявках на оплату, интеграции с бухгалтерскими системами.

Параметры блока:

- Расчетный или корреспондентский счет – обязательный параметр. Номер счета (расчётного или корреспондентского), который требуется проверить. Тип данных: str (строка). Длина строки должна составлять 20 цифр. Пример: 40702810900000000001.
- Банковский идентификационный код (БИК) – обязательный параметр. Девятизначный БИК банка, в котором открыт счёт. Тип данных: str (строка). При-мер: 044525225.
- Возвращаемое значение: Блок возвращает логическое значение (bool): True – контрольная сумма счета корректна, номер счета структурно верен для указанного БИК. False – контрольная сумма не совпадает, счет введён с ошибкой или БИК не соответствует номеру счета.

Блок «Проверить контрольную сумму ИНН»



Блок предназначен для проверки корректности идентификационного номера налогоплательщика (ИНН) путём вычисления контрольных сумм по установленным алгоритмам. Он поддерживает как 10-значные ИНН для юридических лиц, так и 12-значные ИНН для физических лиц и индивидуальных предпринимателей. Блок возвращает True, если структура ИНН соответствует алгоритму, и False в противном случае. Это незаменимый инструмент для автоматизации проверки контрагентов, валидации вводимых данных в учётных системах, импорта рекви-

зитов из внешних источников и предотвращения ошибок при формировании налоговой отчётности.

Параметры блока:

- Идентификационный номер налогоплательщика – обязательный параметр. ИНН (10 или 12 цифр) для проверки. Тип данных: str (строка). Пример: 7725397028 (10-значный), 123456789012 (12-значный).
- Возвращаемое значение: Блок возвращает логическое значение (bool): True – контрольная сумма ИНН корректна, номер структурно верен. False – контрольная сумма не совпадает (ошибка в одной или нескольких цифрах) либо длина ИНН не равна 10 или 12, либо строка содержит недопустимые символы.

Подведение итогов практической работы

Мы завершили изучение раздела, посвящённого автоматизации операционной системы и обработке документов в среде Puzzle RPA. За время освоения этого раздела мы углубили свои знания и освоили инструменты, которые позволяют создавать быстрые, надёжные и масштабируемые сценарии для работы с файловой системой и документами любых форматов, полностью отказавшись от медленной и нестабильной UI-автоматизации там, где это возможно.

Мы освоили архитектуру взаимодействия робота с файловой системой. Поняли, как визуальные блоки Puzzle RPA транслируются в системные вызовы API, минуя графический интерфейс. Научились различать контекст выполнения (учётная запись, права доступа, сетевое окружение) и учитывать это при проектировании сценариев. Это знание позволяет нам прогнозировать поведение робота в разных средах и избегать типичных ошибок, связанных с правами и доступностью ресурсов.

Мы освоили полный набор базовых файловых блоков: копирование, перемещение, удаление, создание файлов и папок, проверка существования пути, получение списка файлов по маске. Особую ценность представляет возможность рекурсивного обхода вложенных папок и пакетной обработки тысяч документов в циклах. Мы научились использовать блок «Список файлов/папок» как основной инструмент для массовых операций.

Мы изучили работу с документами различных форматов. Научились читать текстовые файлы с учётом кодировок (UTF-8, windows-1251), извлекать тексто-вый слой из PDF, удалять водяные знаки, преобразовывать PDF-страницы в изображения для последующего OCR. Особое внимание уделили извлечению таблиц из PDF и сохранению их в Excel – как через прямой парсинг текстового слоя, так и через облачный сервис Yandex Vision для отсканированных документов.

Мы освоили интеграцию с облачным компьютерным зрением Yandex Vision. Научились настраивать блоки «Распознать области», «Извлечь таблицы», «Распознать УПД» и «Распознать документ»: выбирать языковую модель (RUS, ENG, RUS+ENG), режим распознавания (по областям или по ключевым словам), включать коррекцию перекоса. Поняли ограничения (размер файла до 1 МБ, угол перекоса до 45°) и требования к регистрации в Yandex Cloud. Это открыло возмож-

ность автоматической обработки сканов, фотографий документов и сложных PDF без текстового слоя.

Критически важной темой стало построение устойчивых к сбоям сценариев. Мы освоили стратегии обработки ошибок при работе с файловой системой: предварительная проверка существования файла (блок «Существует путь»), перехват исключений, логирование ошибок с сохранением имён проблемных файлов, изоляция неудачно обработанных документов в отдельные папки («Входящие» → «Успех» / «Ошибки»). Научились проектировать сценарии с повторными попытками для временных сетевых сбоев и работой с сетевыми дисками.

Мы расширили наши возможности за счёт выполнения системных команд и параллельной обработки. Научились запускать консольные команды (cmd, PowerShell) с ожиданием завершения или асинхронно, получать вывод и код возврата. Освоили блок «Выполнить процессы параллельно» для многопоточной обработки больших массивов файлов, что даёт десятки раз ускорение по сравнению с последовательным выполнением. Поняли разницу между синхронным и асинхронным режимами, а также особенности кодогенерации (freeze_support) для собранных приложений.

Мы изучили работу с буфером обмена и управление процессами. Научились читать, копировать и очищать буфер обмена для передачи данных между приложениями. Освоили проверку запущенных процессов и их принудительную остановку (всех экземпляров или только текущего пользователя). Это полезно для подготовки окружения перед выполнением критических операций.

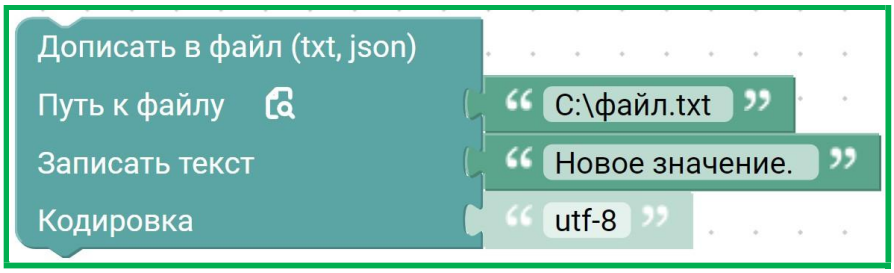
В ходе практической работы мы сформировали целостную систему знаний и навыков, необходимых для профессиональной автоматизации работы с операционной системой и документами. От простых операций копирования файлов мы перешли к сложным, отказоустойчивым решениям, способным обрабатывать тысячи документов за минуты, извлекать данные из отсканированных PDF, взаимодействовать с сетевыми хранилищами и параллельно выполнять тяжёлые вычисления.

Главный результат – мы научились не просто использовать готовые блоки, но и понимать внутреннюю логику работы файловой системы и форматов документов. Это позволяет нам выбирать оптимальные методы извлечения данных, проектировать устойчивые к сбоям сценарии и выстраивать архитектуру робота, которая не зависит от графического интерфейса и может выполняться на серверах без активного рабочего стола. Полученные компетенции являются фундаментом для реализации проектов любой сложности в области автоматизации документооборота и системной интеграции.

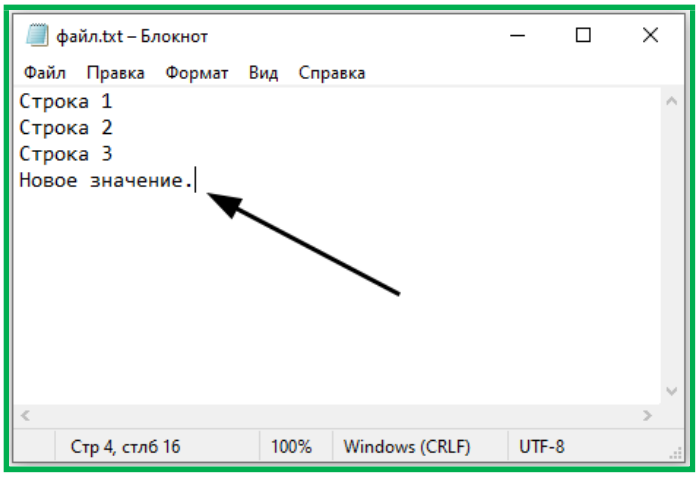
ПРИМЕРЫ ЗАДАЧ

Пример 1 – Дописать в файл

В данном примере происходит запись новых данных в существующий текстовый файл:

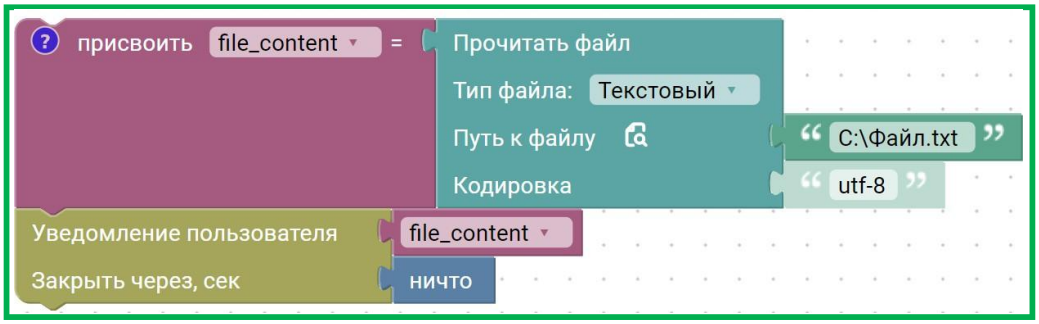


Обновленный текстовый файл:

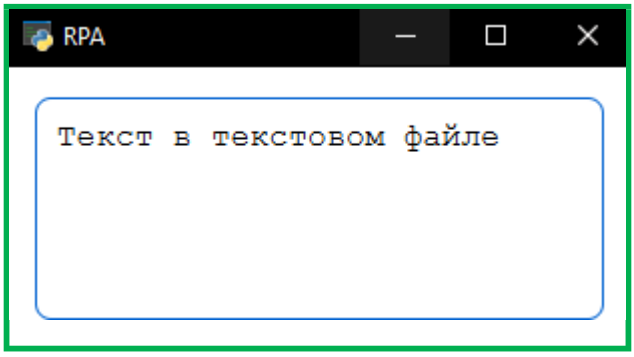


Пример 2 – Прочитать файл

В данном примере происходит считывание текста из текстового файла .txt:

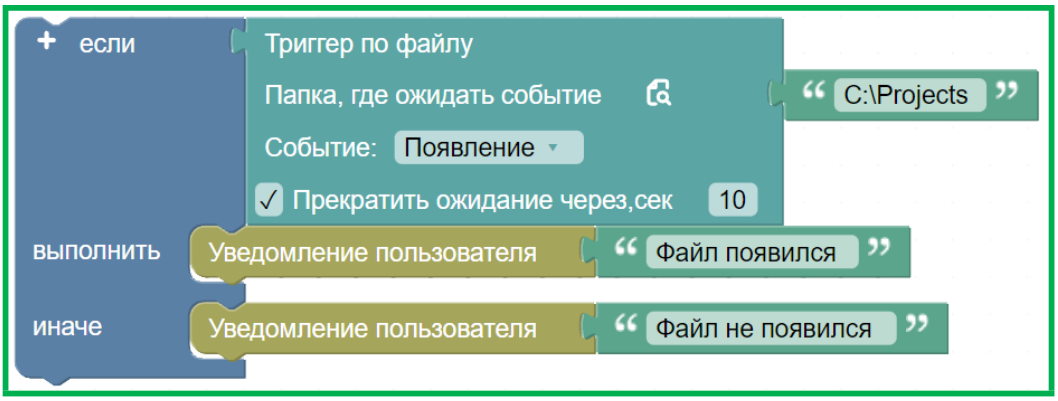


Результат выводится на экран:

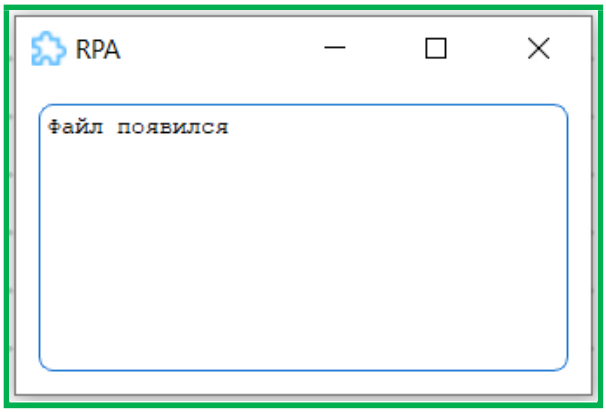


Пример 3 – Триггер по файлу

В данном примере блок ожидает появление нового файла в папке, в течение 10 секунд:



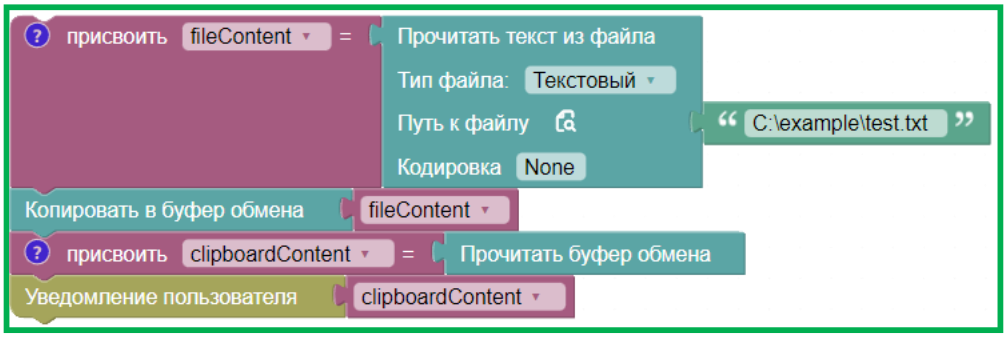
Вывод результата на экран:



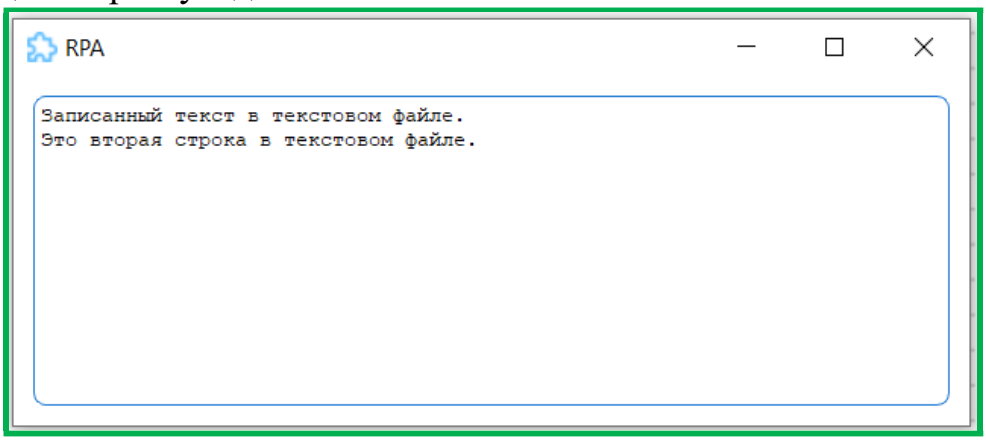
Пример 4 – Копировать в буфер обмена

В данном примере блок копирует прочитанный текст из файла в буфер обмена:

- Переменной **fileContent** присваивается содержимое текстового файла;
- Полученный текст копируется в буфер обмена.



Вывод на экран уведомления:

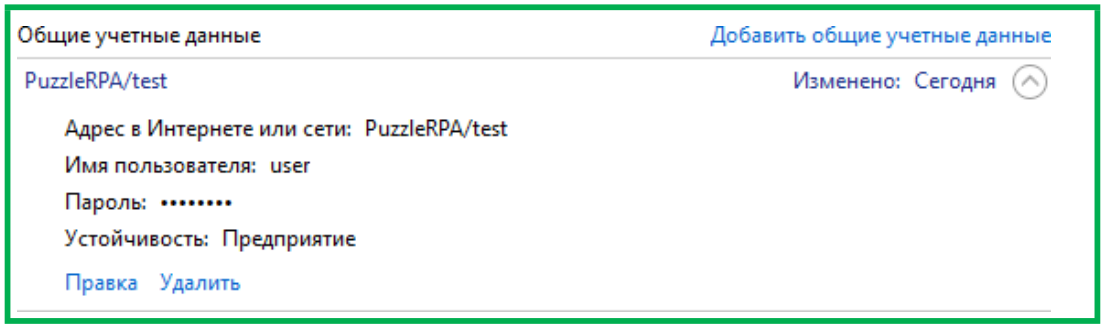


Пример 5 – Добавить учетную запись

В данном примере блок добавляет данные учетной записи в диспетчер учетных данных ОС.

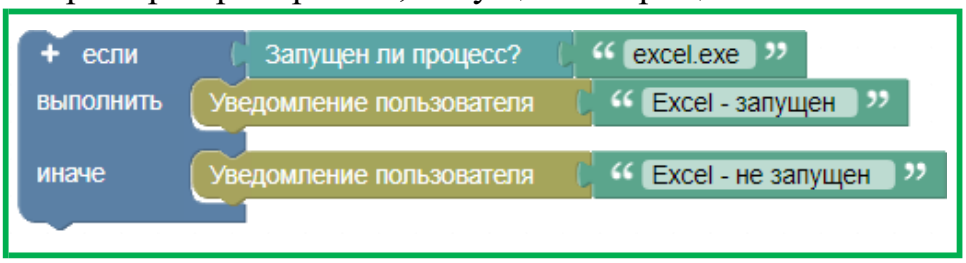


Созданная учетная запись в диспетчере учетных данных ОС:

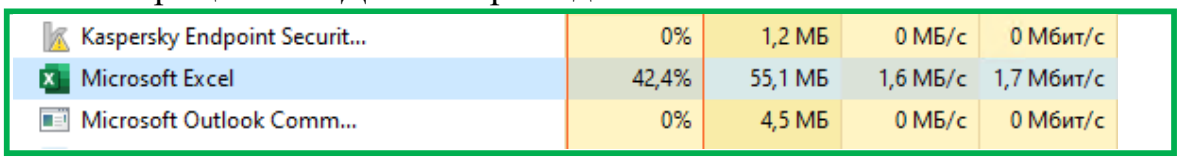


Пример 6 – Запущен ли процесс

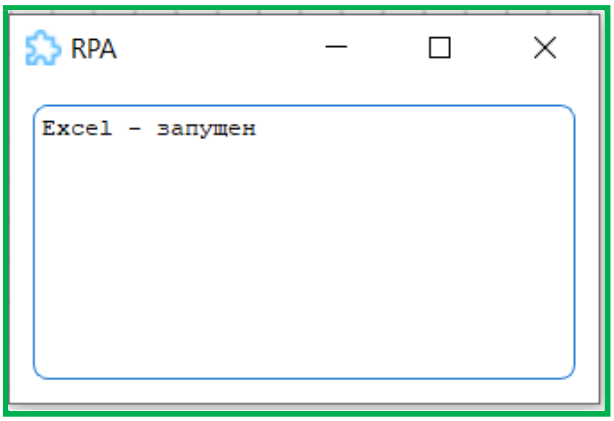
В данном примере проверяется, запущен ли процесс Excel.



Список процессов в Диспетчере задач:



Вывод на экран уведомления:

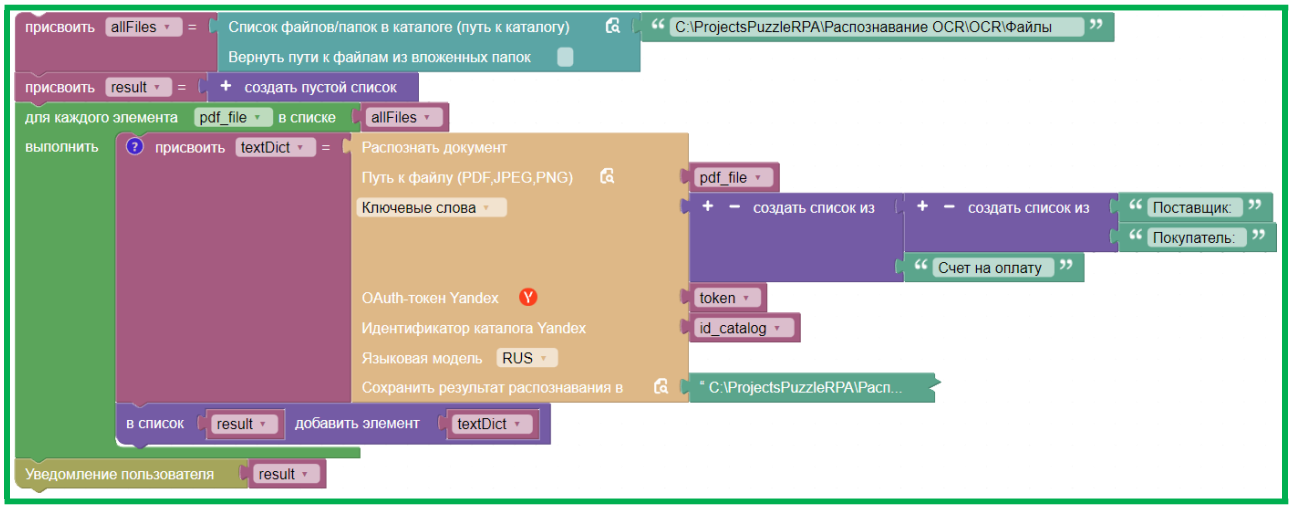


Пример 7 – Распознать области

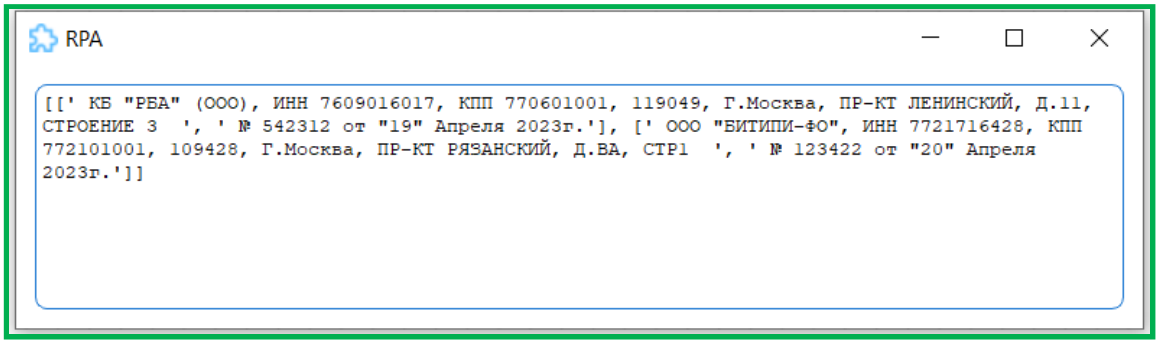
В данном примере распознаются данные из счетов на оплату по ключевым словам.

- Определяются пути ко всем файлам для обработки.

- Создается список для накопления результатов.
- В цикле выполняется распознавание по ключам Поставщик: и Покупатель:.
- Результаты добавляются в итоговый список и выводятся пользователю.

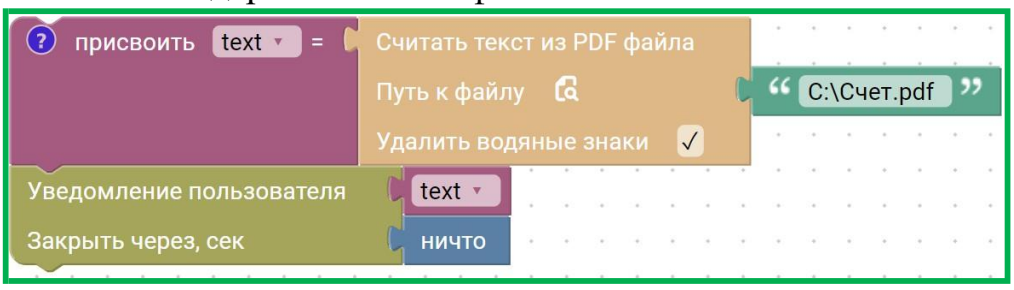


Вывод результата распознавания:

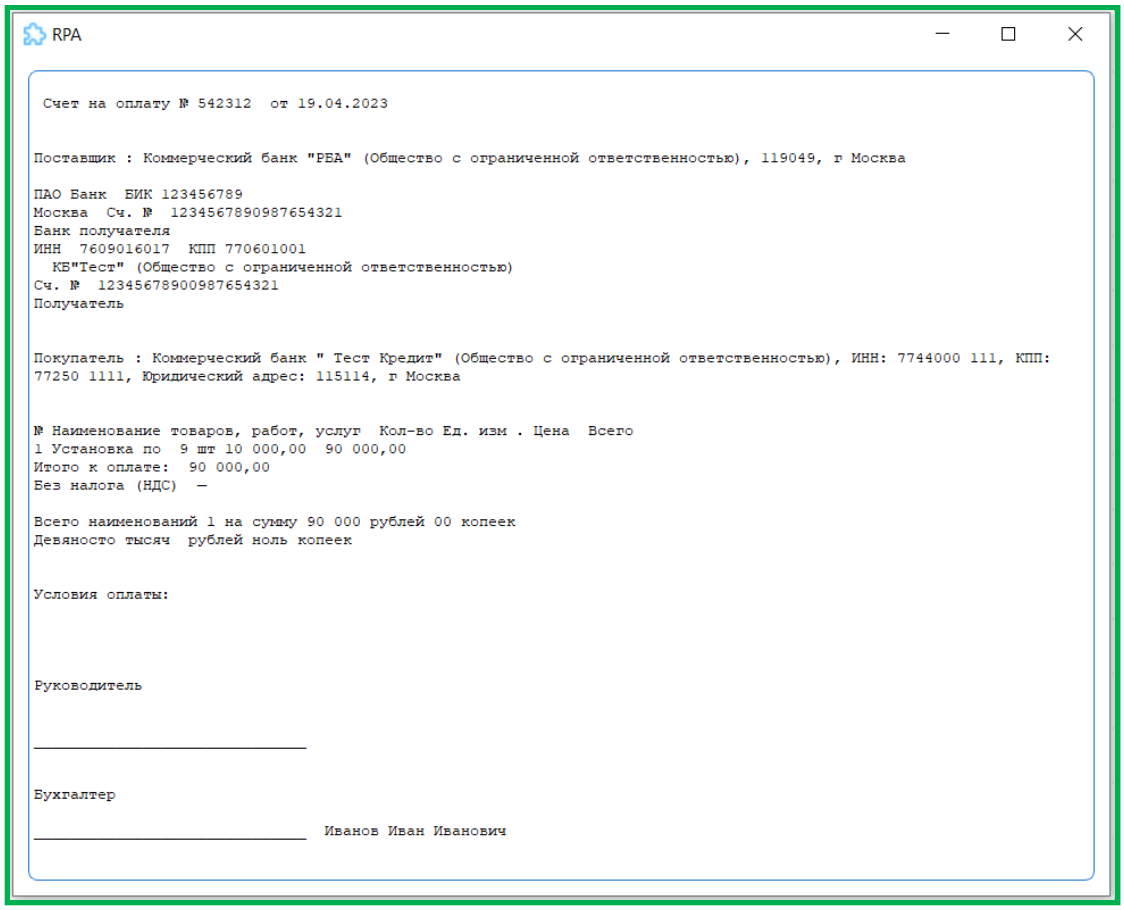


Пример 8 – Считать текст из PDF файла

Блок считывает содержимое PDF файла с текстовым слоем.



Вывод результата на экран:



Пример 9 – Преобразовать PDF в изображение

Блок преобразует PDF-файл в изображение PNG в выбранной директории.

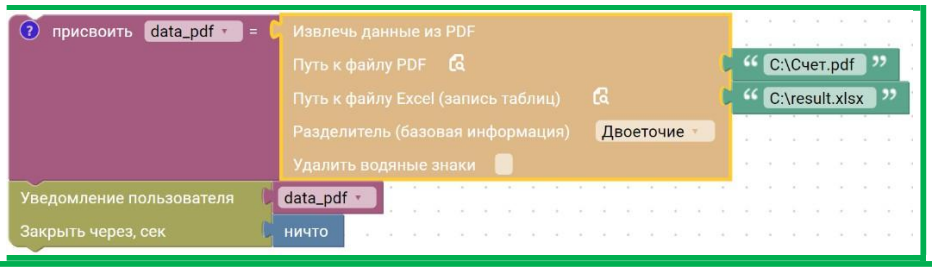


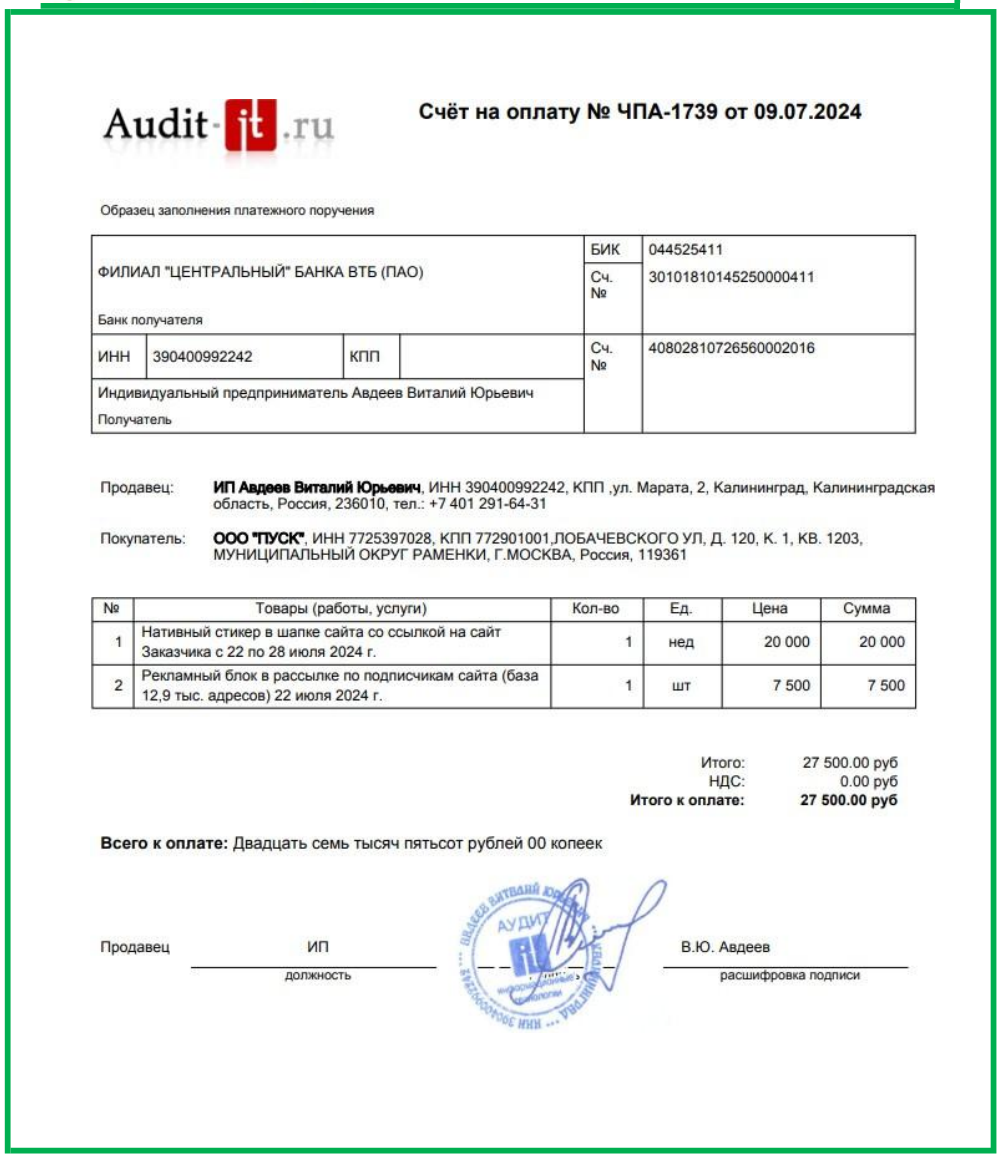
Результат. Файл изображения PNG в папке назначения:



Пример 10 – Извлечь данные из PDF

Блок извлекает таблицы из счёта на оплату.





Audit-it.ru

Счёт на оплату № ЧПА-1739 от 09.07.2024

Образец заполнения платежного поручения

ФИЛИАЛ "ЦЕНТРАЛЬНЫЙ" БАНКА ВТБ (ПАО) Банк получателя				БИК Сч. №	044525411 30101810145250000411
ИНН	390400992242	КПП		Сч. №	40802810726560002016
Индивидуальный предприниматель Авдеев Виталий Юрьевич Получатель					

Продавец: **ИП Авдеев Виталий Юрьевич**, ИНН 390400992242, КПП ,ул. Марата, 2, Калининград, Калининградская область, Россия, 236010, тел.: +7 401 291-64-31

Покупатель: **ООО "ПУСК"**, ИНН 7725397028, КПП 772901001,ЛОБАЧЕВСКОГО УЛ, Д. 120, К. 1, КВ. 1203, МУНИЦИПАЛЬНЫЙ ОКРУГ РАМЕНКИ, Г.МОСКВА, Россия, 119361

№	Товары (работы, услуги)	Кол-во	Ед.	Цена	Сумма
1	Нативный стикер в шапке сайта со ссылкой на сайт Заказчика с 22 по 28 июля 2024 г.	1	нед	20 000	20 000
2	Рекламный блок в рассылке по подписчикам сайта (база 12,9 тыс. адресов) 22 июля 2024 г.	1	шт	7 500	7 500

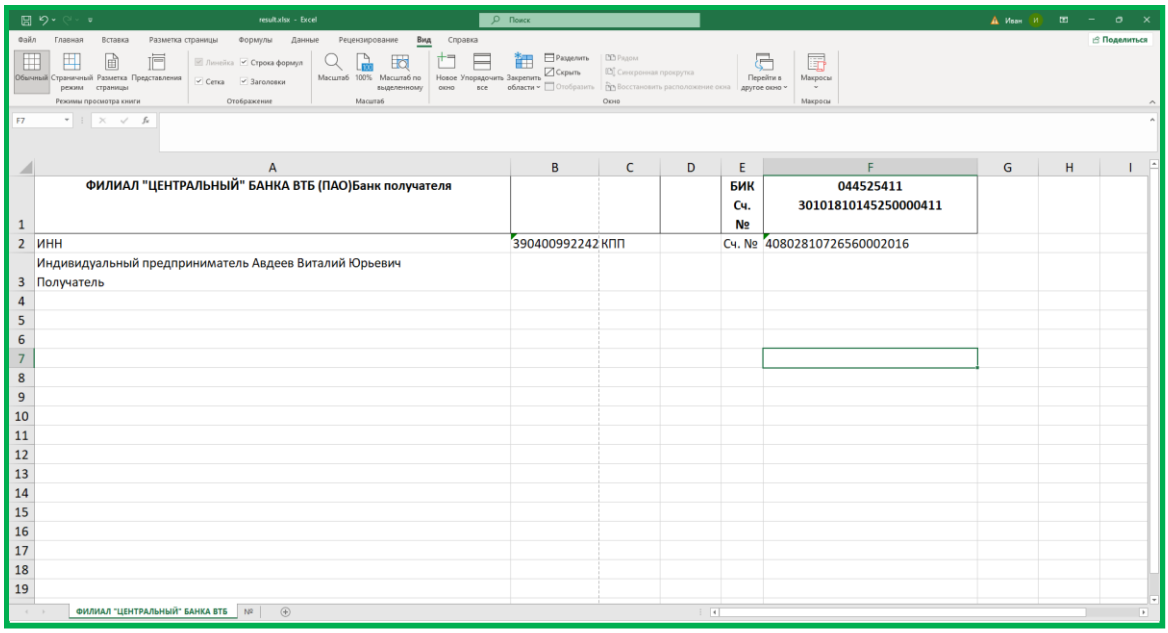
Итого: 27 500.00 руб
НДС: 0.00 руб
Итого к оплате: 27 500.00 руб

Всего к оплате: Двадцать семь тысяч пятьсот рублей 00 копеек

Продавец ИП _____ В.Ю. Авдеев
должность расшифровка подписи

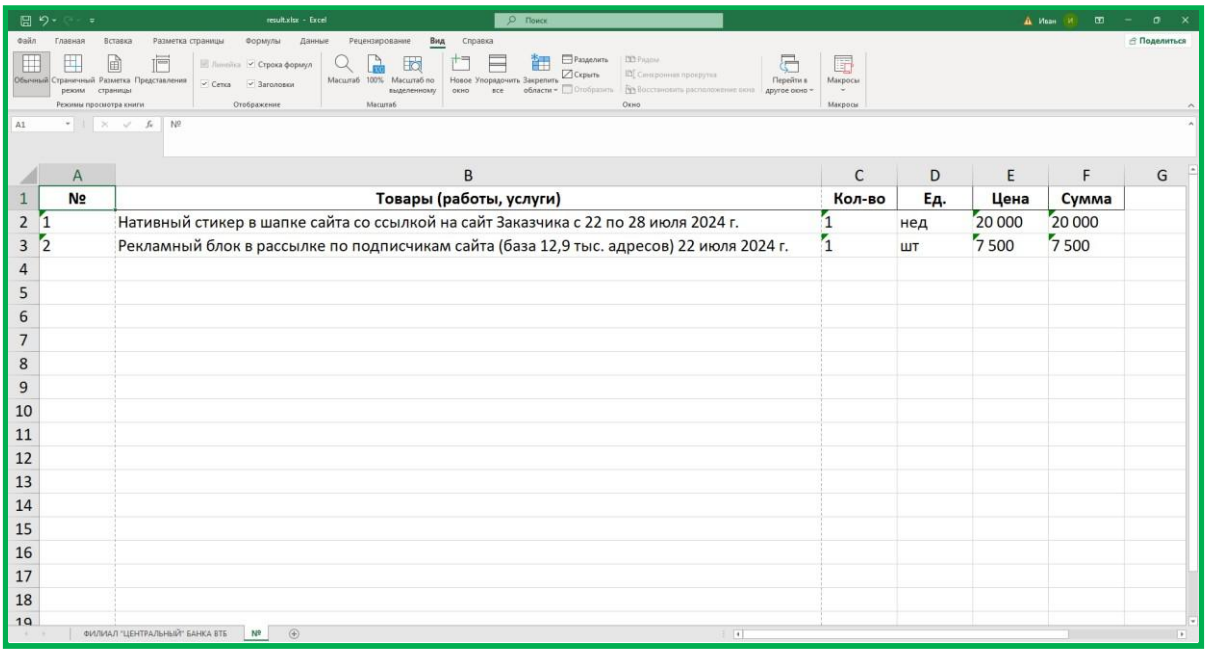
Каждая таблица автоматически записывается в файл.

- Таблица 1



	A	B	C	D	E	F	G	H	I
1	ФИЛИАЛ "ЦЕНТРАЛЬНЫЙ" БАНКА ВТБ (ПАО)Банк получателя				БИК Сч. № 	044525411 30101810145250000411			
2	ИНН	390400992242	КПП		Сч. №	40802810726560002016			
	Индивидуальный предприниматель Авдеев Виталий Юрьевич								
3	Получатель								
4									
5									
6									
7									
8									
9									
10									
11									
12									
13									
14									
15									
16									
17									
18									
19									

- Таблица 2



	A	B	C	D	E	F	G
1	№	Товары (работы, услуги)	Кол-во	Ед.	Цена	Сумма	
2	1	Нативный стикер в шапке сайта со ссылкой на сайт Заказчика с 22 по 28 июля 2024 г.	1	нед	20 000	20 000	
3	2	Рекламный блок в рассылке по подписчикам сайта (база 12,9 тыс. адресов) 22 июля 2024 г.	1	шт	7 500	7 500	
4							
5							
6							
7							
8							
9							
10							
11							
12							
13							
14							
15							
16							
17							
18							
19							

Кейс 1. Автоматизация обработки входящих документов с использованием системных блоков Puzzle RPA

В компании TechFlow организован входящий документооборот. Каждый день сотрудники присылают на общий сетевой диск пакеты документов в ZIP-архивах. Внутри каждого архива всегда находятся следующие файлы:

- счёт-фактура в формате «PDF с текстовым слоем» (`invoice.pdf`);
- акт выполненных работ в формате «PDF с текстовым слоем» (`act.pdf`);
- универсальный передаточный документ в формате «PDF с текстовым слоем» (`upd.pdf`);
- сопроводительный файл `request_info.txt` с данными о заявке.

Робот должен автоматически обрабатывать эти архивы: распаковывать, извлекать из PDF ключевые реквизиты (номер, дата, сумма), переименовывать фай-

лы по правилам компании, формировать итоговый отчёт и архивировать обработанные документы.

Важно: все PDF создаются с текстовым слоем (машиночитаемые).

Робот должен:

- Распаковать каждый архив.
- Извлечь ключевые данные из PDF (номер, дата, сумма).
- Переименовать PDF по шаблону, например Invoice_СФ-123_15.04.2026_45800.50.pdf.
- Прочитать request_info.txt.
- Сформировать текстовый отчёт (в C:\Reports\) со всей информацией по каждому архиву.
- Упаковать обработанные файлы в новый ZIP-архив (с пометкой _done) и поместить в C:\Archived\.
- Удалить исходный ZIP из папки C:\Incoming.

Что нужно перед запуском робота:

- Создать нужные папки: C:\Incoming, C:\Temp\Extracted, C:\Archived, C:\Reports.
- Создать тестовые данные: 3 ZIP-архива по инструкции из кейса. Внутри каждого – правильные PDF (с текстовым слоем) и request_info.txt.
- (Опционально) добавить учётную запись в хранилище, если требуется доступ к сетевым папкам.

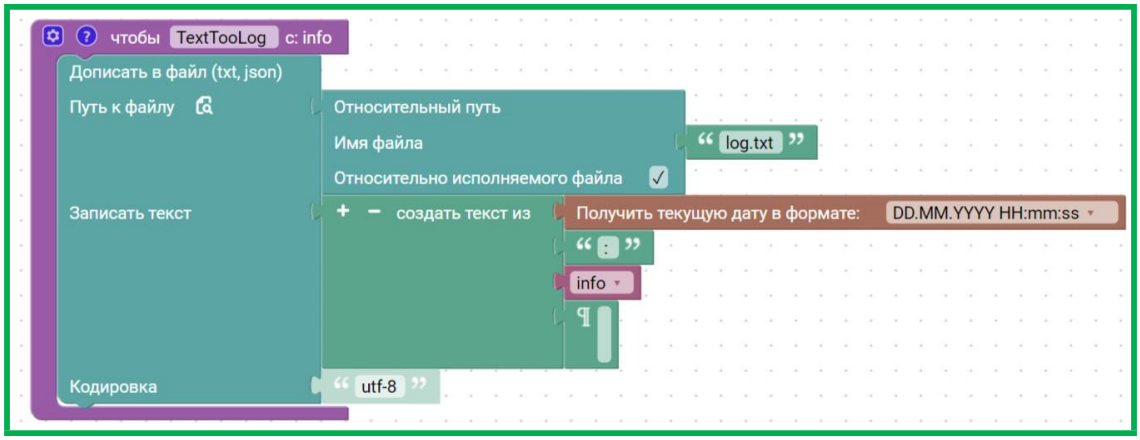
Ожидаемый результат:

- Рабочий сценарий в Puzzle RPA.
- Итоговый отчёт (текстовый файл) в C:\Reports\ со всеми данными.
- Архивы с суффиксом _done.zip в C:\Archived\.
- Исходные ZIP-архивы из C:\Incoming удалены.

Важные замечания:

- PDF должны быть машиночитаемыми (текстовый слой) – создавать из Word/Google Docs, а не сканировать.
- Для извлечения данных из PDF не нужно облачных сервисов – используйте локальный блок.
- Все пути и имена папок должны быть строго как в задании (или используйте переменные).

Прежде чем приступить к сборке основного сценария, создадим вспомогательный инструмент для записи логов.



Функция TextTooLog(info) делает следующее:

- Берёт текущую дату и время (блок «Получить текущую дату» с нужным нам-форматом).
- Дописывает к этой метке переданную строку info и перевод строки.
- Добавляет полученную строку в файл log.txt, который лежит рядом с исполняемым файлом робота (блок «Относительный путь» + «Дописать в файл»).

Таким образом, любой вызов TextTooLog('Сообщение') оставляет след в логе. Это поможет вам отлаживать робота и видеть ход выполнения каждого шага.

После того как мы создали вспомогательную функцию TextTooLog приступаем к главному сценарию обработки архивов. Все дальнейшие блоки мы размещаем в одной процедуре Main или вы можете дать любое понятное имя.

Шаг 1. Добавление учётной записи в защищённое хранилище

Сохранить пару «логин / пароль» в защищённом хранилище ОС. Даже если в локальном кейсе аутентификация не нужна, этот блок демонстрирует правильную работу с секретными данными.

Какой блок используем:

«Добавить учётную запись»

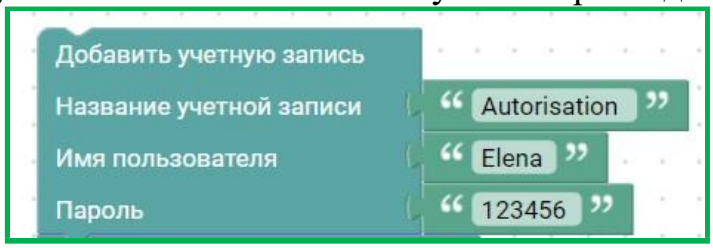
Какие данные передаём:

Параметр	Значение	Пояснение
Имя хранилища	Autorisation	Произвольный идентификатор учётки
Имя пользователя	Elena	Логин для доступа
Пароль	123456	Пароль

Что происходит после выполнения:

Пара Elena / 123456 сохраняется в защищённое хранилище (например, Windows Credential Manager).

В любой момент вы можете получить эти данные блоками «Получить имя пользователя для учётной записи» или «Получить пароль для учётной записи».

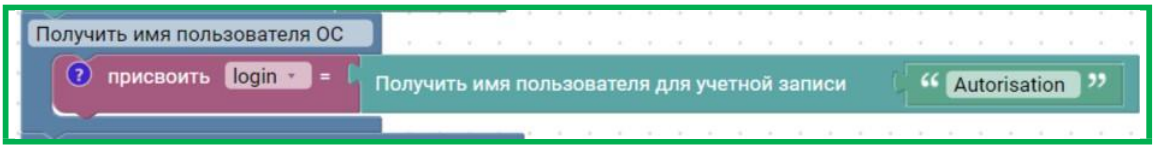


Шаг 2. Получаем имя пользователя операционной системы

Этот шаг не обязателен для работы робота, но полезен для логирования – мы узнаем, кто именно запустил обработку.

Берём блок «Получить имя пользователя операционной системы».

Результат сохраняем в переменную login.



Шаг 3. Проверяем, существует ли папка C:\Incoming

Чтобы робот не упал с ошибкой, перед началом работы убеждаемся, что входная папка существует.

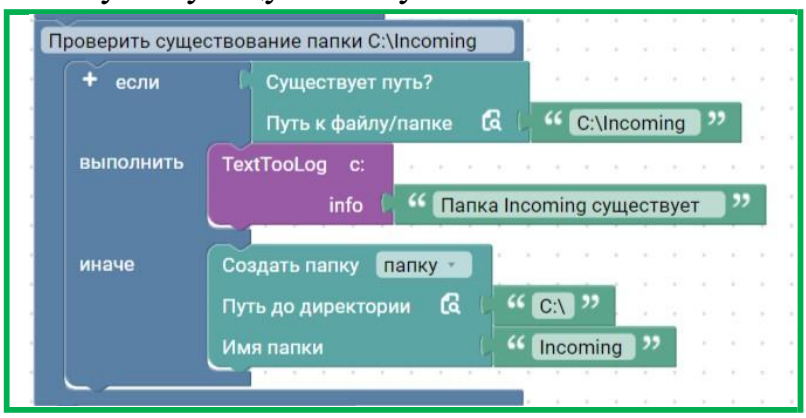
Используем блок «Существует путь?» из группы «Файлы и папки».

В параметре «Путь» пишем C:\Incoming.

Теперь нам нужно выполнить разные действия в зависимости от результата. Для этого используем блок «Если – выполнить». Он автоматически подхватывает результат предыдущего блока.

В ветку «Истина» помещаем блок «Вывести сообщение» (или используем нашу функцию TextTooLog с текстом «Папка Incoming существует»).

В ветку «Ложь» помещаем блок «Создать папку» из группы «Файлы и папки». В настройках указываем: родительский путь C:\, имя новой папки Incoming. Этот блок создаст отсутствующую папку.



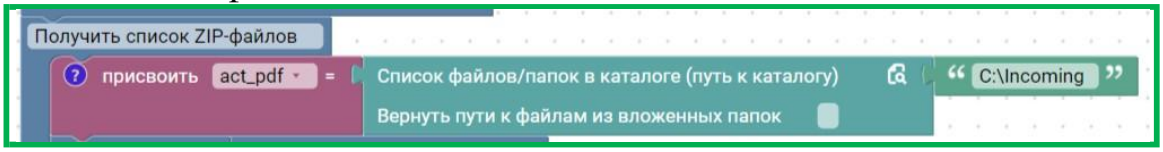
Шаг 4. Получаем список ZIP-файлов в папке C:\Incoming

Теперь робот должен узнать, какие архивы предстоит обработать. Берём блок «Список файлов/папок».

В параметре «Директория» указываем C:\Incoming.

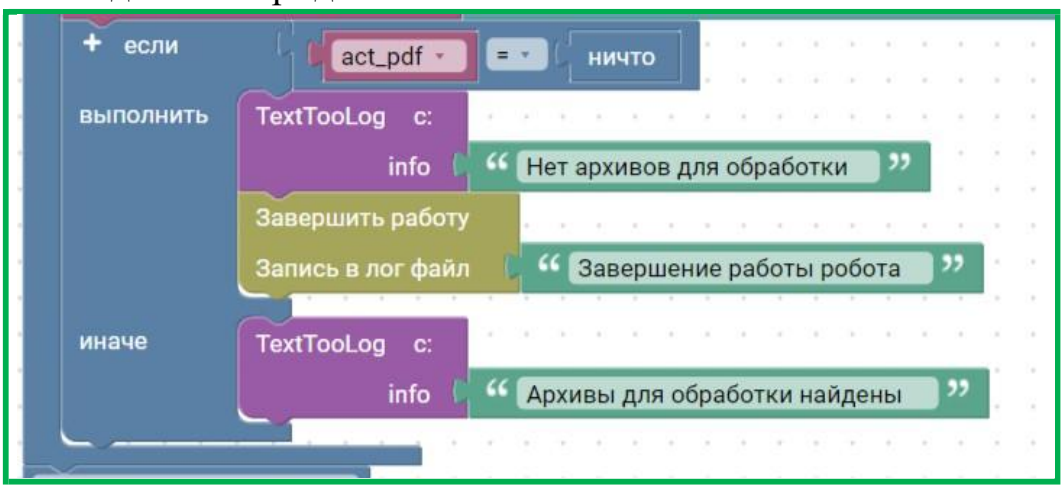
Флаг «Обход подпапок» оставляем выключенным, потому что архивы лежат прямо в Incoming, без вложенных папок.

Сохраняем результат в переменную, назовём её act_pdf. Блок вернёт список путей ко всем ZIP-файлам.



Сразу после этого проверяем, есть ли вообще архивы. Снова используем блок «Если – выполнить». Условием будет сравнение: act_pdf = None. Если список пуст, то в ветке «Истина» вызываем TextTooLog с сообщением «Нет архивов для обработки», а затем ставим блок «Завершить работу» из группы «Управление роботом». Этот блок немедленно остановит выполнение.

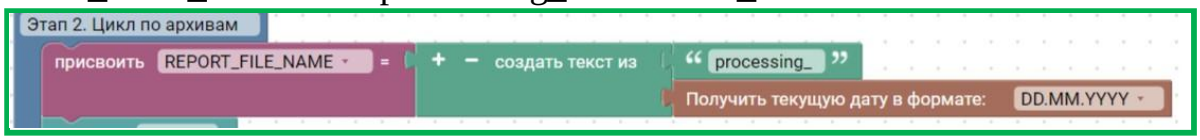
Если же список не пуст, в ветке «Ложь» выводим сообщение «Архивы для обработки найдены» и продолжаем.



Шаг 5. Готовим файл итогового отчёта

Перед основным циклом создаём текстовый файл, куда будем записывать результаты по каждому архиву.

Сначала получаем текущую дату. Берём блок «Получить текущую дату» из группы «Дата и время». В параметре «Формат» задаём (день.месяц.год). Сохраняем результат в переменную, например current_date. Теперь формируем имя файла отчёта с помощью блока «Присвоить значение переменной»: REPORT_FILE_NAME = "processing_" + current_date.

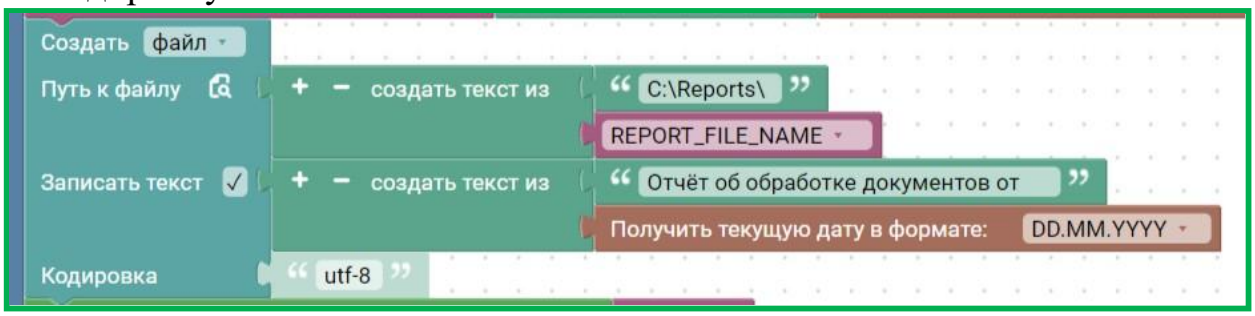


Далее создаём сам файл. Берём блок «Создать файл» (группа «Файлы и папки»).

Полный путь к файлу указываем как C:\Reports\ + значение переменной REPORT_FILE_NAME.

В параметре «Содержимое файла» пишем "Отчёт об обработке документов от " + current_date.

Кодировку ставим utf-8.



Шаг 6. Запускаем цикл обработки каждого ZIP-архива

Теперь начинаем главный цикл. Берём блок «Цикл для каждого элемента».

В качестве коллекции указываем переменную act_pdf – наш список архивов.

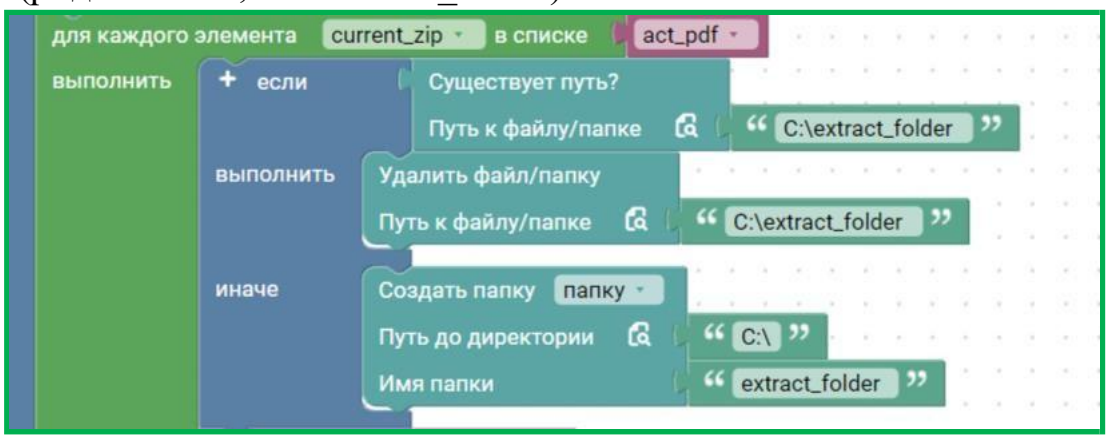
Для каждого элемента коллекции создаётся переменная current_zip. Внутрь цикла будем помещать все последующие шаги 6.1 – 6.12.

Шаг 6.1. Готовим временную папку для распаковки

Нам нужно место, куда разархивировать файлы. Лучше каждый раз начинать с чистой папки.

Сначала проверяем, не существует ли уже папка C:\extract_folder. Используем блок «Существует путь?» с путём C:\extract_folder.

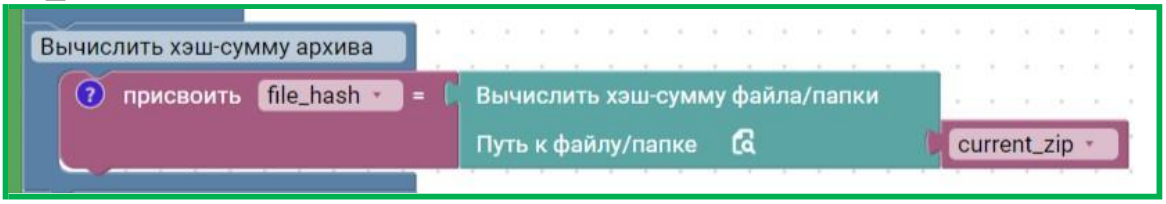
В блоке «Если – выполнить»: если папка существует, удаляем её блоком «Удалить файл/папку». Если не существует – создаём новую блоком «Создать папку» (родитель C:\, имя extract_folder).



Шаг 6.2. Вычисляем хэш-сумму архива

Для контроля целостности и для отчёта хорошо вычислить хэш. Берём блок «Вычислить хэш-сумму файла/папки».

В параметре «Путь» указываем current_zip. Результат сохраняем в переменную file_hash.



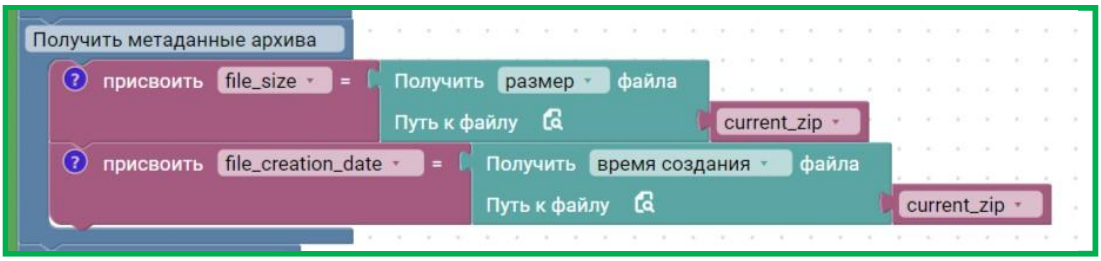
Шаг 6.3. Получаем метаданные архива (размер и дату создания)

Дважды используем блок «Получить метаданные файла» из той же группы.

Первый раз – в поле «Свойство» выбираем file_size, путь current_zip. Сохраняем в file_size.

Второй раз – свойство file_ctime (дата создания), сохраняем в file_creation_date.

Размер будет в байтах, дата – в формате, который выдаёт операционная система.



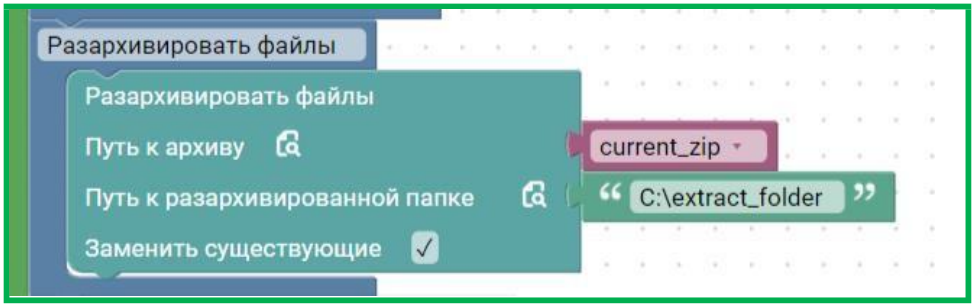
Шаг 6.4. Разархивируем файлы во временную папку

Берём блок «Разархивировать файлы».

Параметр «Исходный архив» – current_zip.

«Путь для разархивации» – C:\extract_folder.

Флаг «Перезаписывать» ставим TRUE, чтобы при повторных запусках всё обновлялось.

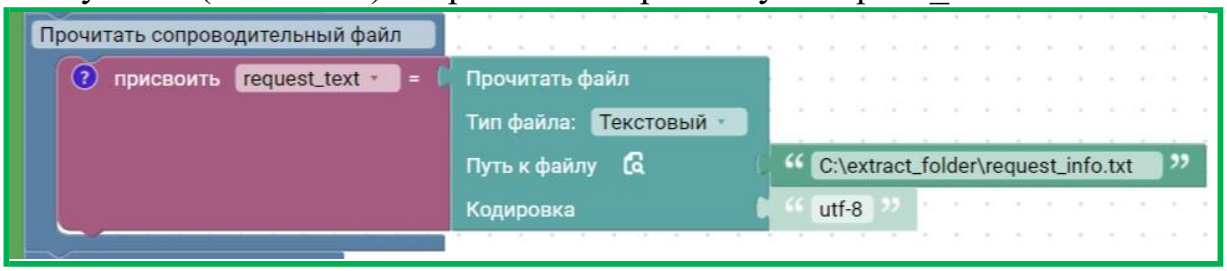


Шаг 6.5. Читаем сопроводительный файл request_info.txt

Внутри распакованной папки лежит текстовый файл с описанием заявки. Берём блок «Прочитать файл» из группы «Текстовые файлы».

Кодировка – utf-8. Полный путь к файлу: C:\extract_folder\request_info.txt.

Результат (весь текст) сохраняем в переменную request_text.



Шаг 6.6. Извлекаем отдельные поля из request_info.txt

Нам понадобятся RequestID, Employee, Department, Description, Priority. Для каждого поля используем блок «Получить часть текста».

Настраиваем блок так:

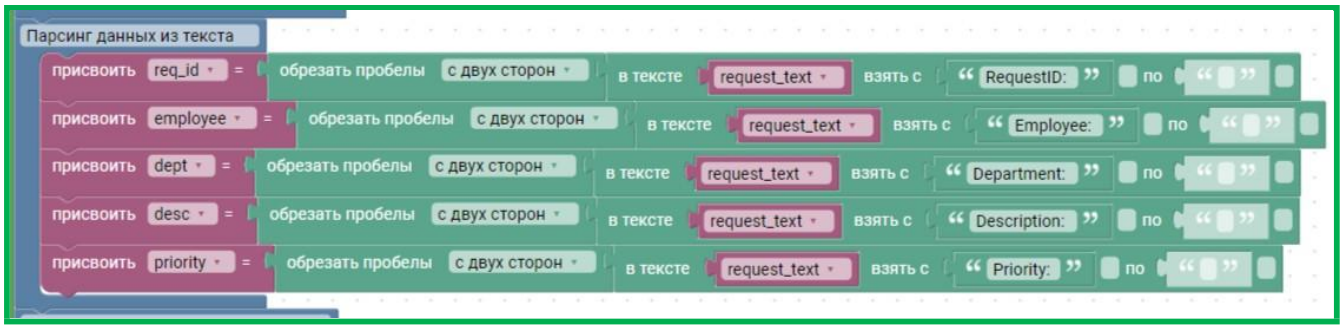
Исходный текст – request_text.

Искомая подстрока – например "RequestID:".

Параметр «До подстроки» – оставляем пустым, чтобы взять всё до конца.

Дополнительно после извлечения применяем блок убрать лишние пробелы и переводы строк.

Сохраняем результат в переменные: req_id, employee, dept, desc, priority.

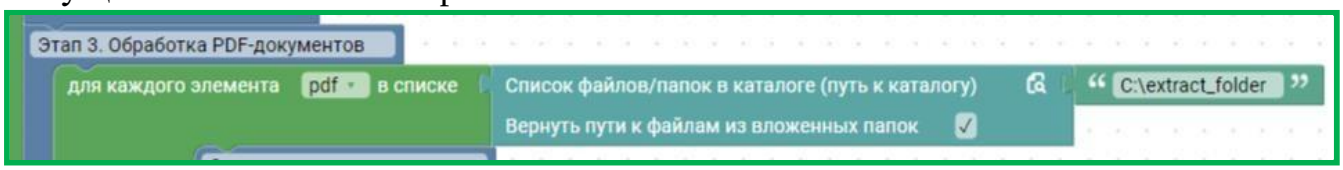


Шаг 7. Обрабатываем PDF-файлы (внутренний цикл)

Сначала нужно получить список всех файлов внутри C:\extract_folder, включая вложенные подпапки.

Берём блок «Список файлов/папок» с директорией C:\extract_folder, флаг «Обход подпапок» включаем. Сохраняем результат в переменную all_files.

Теперь запускаем ещё один цикл – «Цикл для каждого элемента» по all_files. Текущий элемент назовём pdf.



Шаг 7.1. Определяем тип документа по имени файла

Внутри цикла по файлам проверяем, какой именно PDF обрабатывается. Используем блок «Содержит ли строка подстроку».

Сравниваем pdf с "invoice.pdf", "act.pdf", "upd.pdf".

В зависимости от результата в блоке «Если – выполнить» присваиваем переменной doc_type значение "Invoice", "Act" или "UPD".

Шаг 7.2. Извлекаем текст из PDF

Для каждого типа документа берём блок «Считать текст из PDF-файла».

Путь к файлу – текущий pdf. Флаг «Удалить водяные знаки» оставляем FALSE.

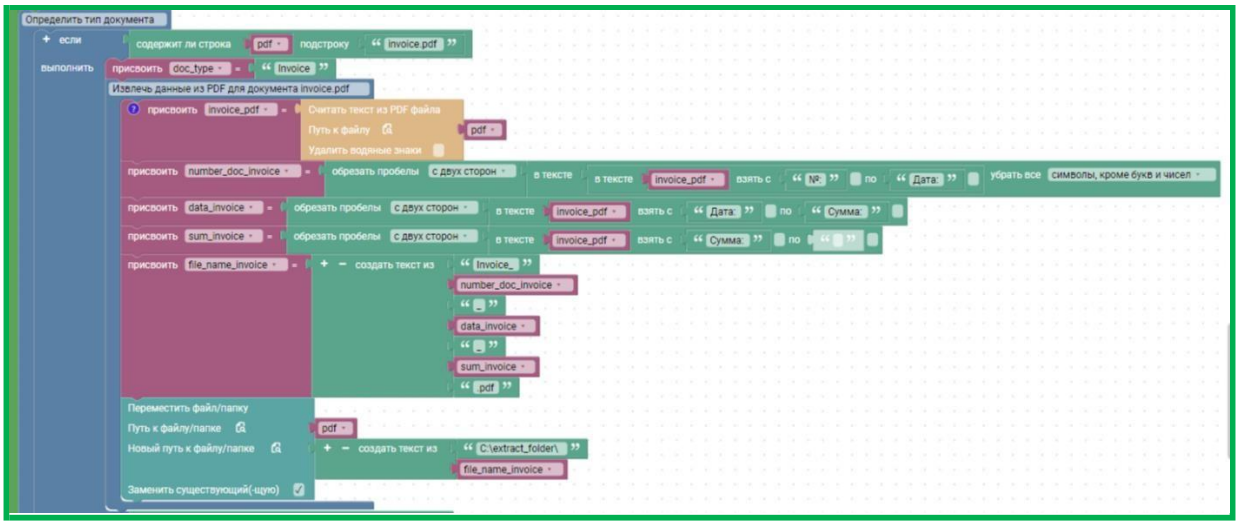
Сохраняем полученный текст в разные переменные: для счёта – invoice_pdf, для акта – act_doc_pdf, для УПД – upd_pdf. Удобно использовать условные конструкции в блоке «Если – выполнить», чтобы не выполнять лишние действия.

Шаг 7.3. Парсим реквизиты из текста PDF

Снова используем блок «Получить часть текста».

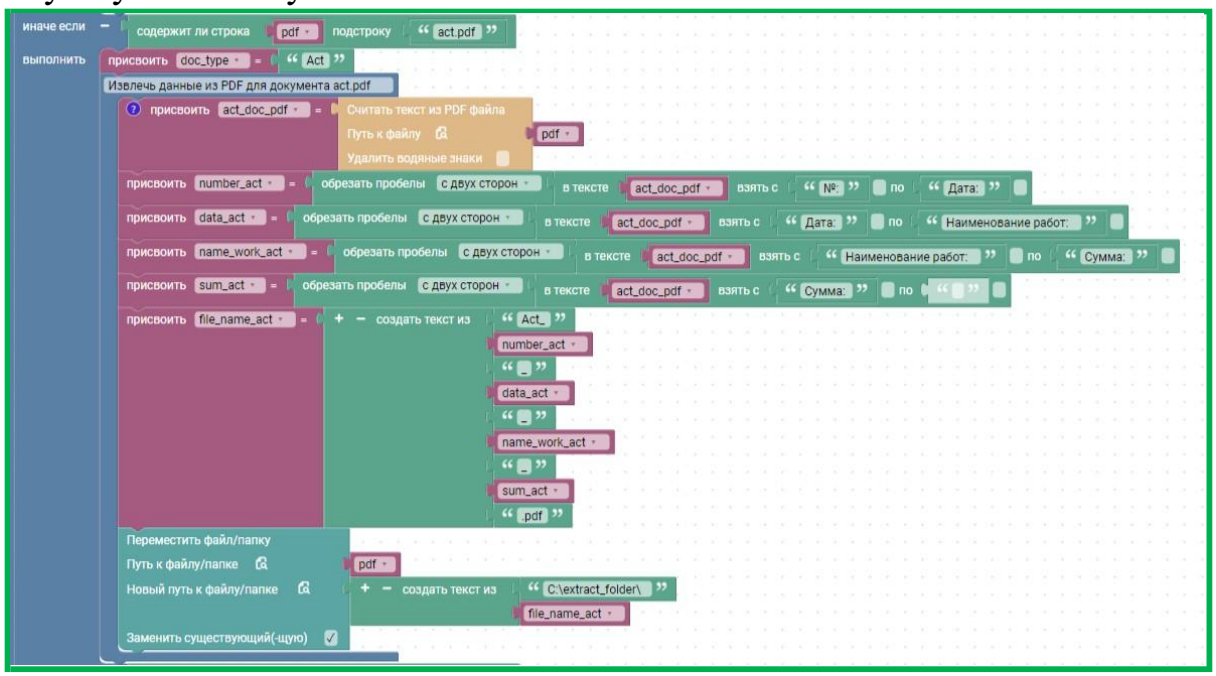
Для счёта-фактуры:

- ✓ Номер: ищем между "№:" и "Дата:".
- ✓ Дату: между "Дата:" и "Сумма:".
- ✓ Сумму: после "Сумма:" до конца строки.

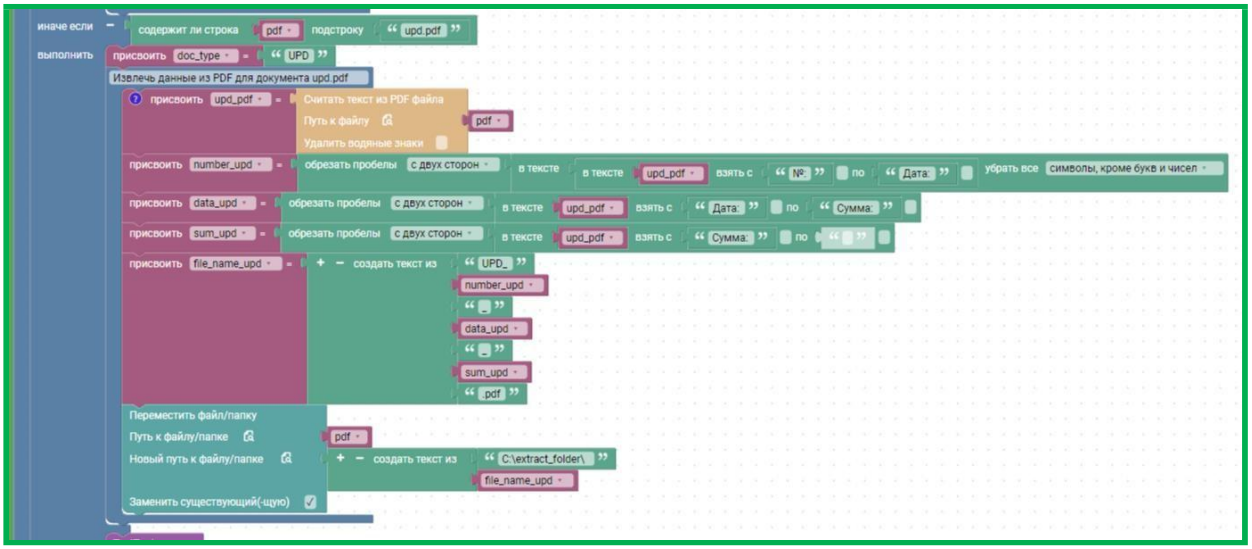


Для акта:

- ✓ Номер между "№:" и "Дата:".
- ✓ Дату между "Дата:" и "Наименование работ:".
- ✓ Наименование работ между "Наименование работ:" и "Сумма:".
- ✓ Сумму после "Сумма:".



Для УПД – аналогично счёту (номер, дата, сумма).



После извлечения желательно очистить значения от лишних символов. Для номера, чтобы оставить только буквы и цифры, можно использовать блок «Убрать все из строки» с регулярным выражением.

Шаг 7.4. Формируем новое имя файла

Используем блок «Присвоить значение переменной», собирая строку по шаблону.

Для счёта: file_name_invoice = "Invoice_" + number_invoice + "_" + data_invoice + "_" + sum_invoice + ".pdf".

Для акта: file_name_act = "Act_" + number_act + "_" + data_act + "_" + name_work_act + "_" + sum_act + ".pdf".

Для УПД: file_name_upd = "UPD_" + number_upd + "_" + data_upd + "_" + sum_upd + ".pdf".

Шаг 7.5. Переименовываем файл (перемещаем с новым именем)

Берём блок «Переместить файл/папку».

Исходный путь – текущий pdf.

Путь назначения – C:\extract_folder\ + новое имя (например, file_name_invoice).

Флаг «Заменить существующий» – TRUE, чтобы при конфликте имён файл перезаписался.

Этот блок фактически переименовывает файл, так как он остаётся в той же папке, но с новым именем.

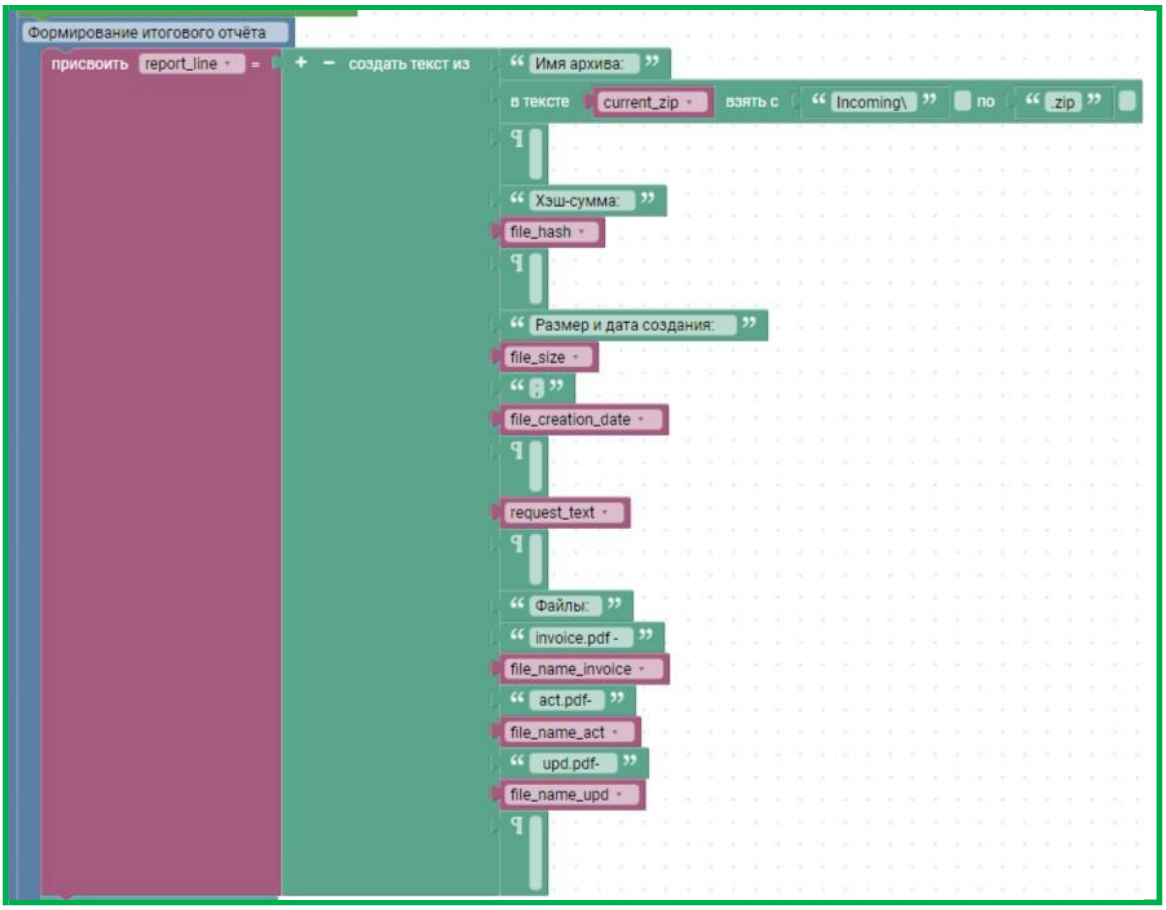
Шаг 8. Формируем строку отчёта по текущему архиву

После того как все PDF обработаны (цикл по all_files завершён), нам нужно добавить информацию об этом архиве в общий отчёт.

Создаём новую переменную report_line. С помощью блока «Присвоить значение переменной» составляем строку, в которую включаем:

- имя исходного архива (извлекаем из current_zip с помощью блока «Получить часть текста», вырезая между "Incoming\\" и ".zip");
- хэш-сумму (file_hash);
- размер и дату создания (file_size и file_creation_date);
- содержимое request_text;
- новые имена файлов: file_name_invoice, file_name_act, file_name_upd.

Вставляем между частями разделители (переводы строк, пробелы), чтобы отчёт читался легко.

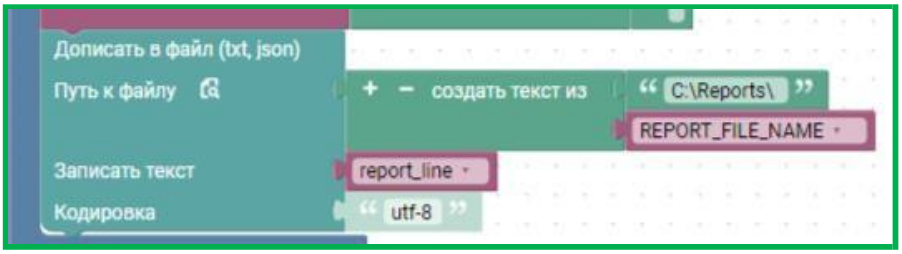


Затем берём блок «Дописать в файл».

Указываем путь к файлу отчёта: C:\Reports\ + REPORT_FILE_NAME.

В поле «Данные для вставки» передаём report_line. Кодировка utf-8.

Этот блок добавит новую строку в конец файла отчёта.



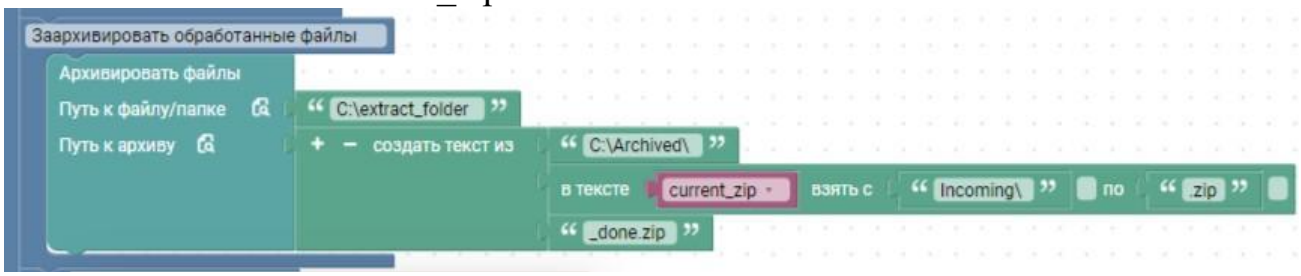
Шаг 9. Архивируем обработанные файлы и удаляем исходник

Чтобы сохранить результат, упаковываем всю временную папку C:\extract_folder в новый ZIP-архив с пометкой _done.

Используем блок «Архивировать файлы».

В параметре «Что архивировать» указываем C:\extract_folder.

В поле «Путь к архиву» формируем строку: "C:\Archived\" + имя исходного архива (без .zip) + "_done.zip". Имя исходного архива снова извлекаем через «Получить часть текста» из current_zip.



После успешного создания архива удаляем исходный ZIP-файл из папки C:\Incoming блоком «Удалить файл/папку», указав путь current_zip.



Мы завершили выполнение практической работы по автоматизации обработки входящих документов с использованием Puzzle RPA.

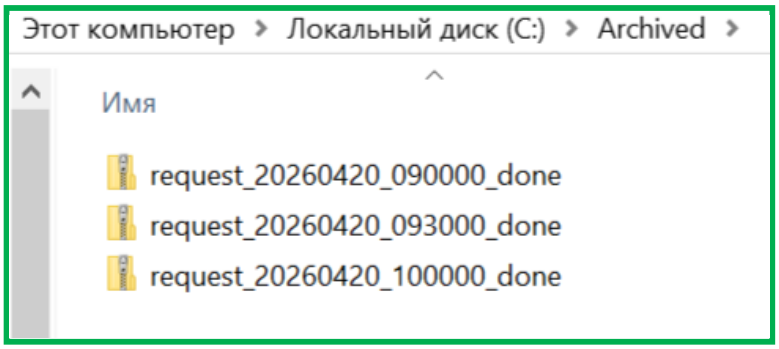
В ходе работы мы:

- Создали вспомогательную функцию логирования TextTooLog, которая записывает все ключевые события в файл log.txt.
- Разработали основной сценарий, который автоматически выполняет полный цикл обработки ZIP-архивов:
  - ✓ проверяет наличие входной папки и создаёт её при необходимости;
  - ✓ получает список архивов и обрабатывает каждый в цикле;
  - ✓ разархивирует файлы;
  - ✓ извлекает реквизиты из текстовых и PDF-документов;
  - ✓ переименовывает PDF по заданному шаблону;
  - ✓ формирует итоговый отчёт;
  - ✓ упаковывает обработанные файлы в новый архив и удаляет исходный.

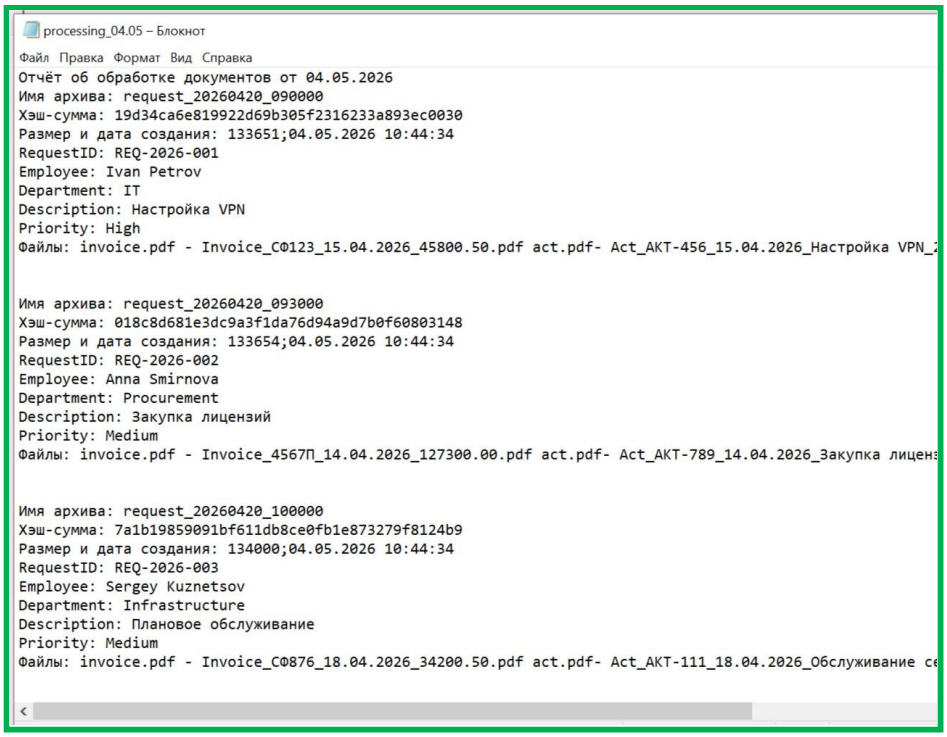
Все шаги реализованы с помощью стандартных блоков Puzzle RPA.

Результат работы робота можно проверить:

- В папке C:\Archived появляются ZIP-архивы с суффиксом _done.zip, внутри которых лежат переименованные PDF и файл request_info.txt.



- В папке C:\Reports создаётся текстовый отчёт со сводкой по каждому архиву.



- Исходные архивы из C:\Incoming удаляются после успешной обработки.

Данный сценарий полностью готов к использованию на реальных бизнес-кейсах. Студенты, выполнившие эту работу, научатся:

- проектировать RPA-процессы;
- применять основные системные блоки Puzzle RPA;
- обрабатывать документы без участия человека.

Теперь вы можете самостоятельно адаптировать робота под другие форматы документов или дополнительные источники данных.

Кейс 2. Автоматизация верификации контрагентов по PDF-документам (без использования Yandex Vision)

В бухгалтерию поступают заявки на открытие расчётных счетов. Каждая заявка представляет собой ZIP-архив, содержащий три PDF-файла (все с текстовым слоем):

- `passport.pdf` – данные паспорта (фамилия, имя, отчество, серия, номер);
- `inn.pdf` – номер ИНН (10 или 12 цифр);
- `bank_details.pdf` – БИК и расчётный счёт.

Робот должен автоматически:

- Распаковать архив.
- Извлечь текст из каждого PDF.
- Проверить ИНН на корректность (контрольная сумма).
- Проверить расчётный счёт по БИК (контрольная сумма).
- Сформировать Excel-отчёт с результатами проверки для всех архивов.
- Переместить исходный ZIP-архив в папку Успех (если все проверки пройдены) или Ошибки (если хоть одна проверка не пройдена).

Всё делается локальными блоками Puzzle RPA, без интернета.

Перед тем как собирать основного робота для проверки контрагентов, мы делаем тот же самый подготовительный шаг, что и в первом кейсе. Выносим запись логов в отдельную функцию, чтобы потом вызывать её одной строчкой в любом месте сценария.



Функция с именем TextTooLog принимает на вход строку info (например, «Начало обработки архива») и дописывает её в файл log.txt с меткой времени. Файл log.txt создаётся в той же папке, где лежит исполняемый файл робота (рядом со сценарием). Это очень удобно для отладки: вы всегда можете открыть log.txt и посмотреть, в каком порядке робот выполнял действия и не было ли ошибок.

Шаг 1. Проверить, существует ли входная папка C:\Incoming_Contracts

Блок: «Существует путь?»

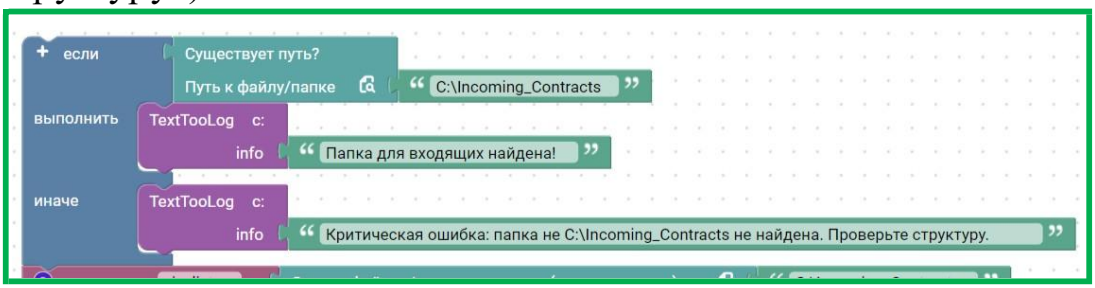
Путь: C:\Incoming_Contracts

Этот блок возвращает True или False. Его результат автоматически подхватывается следующим блоком «Если – выполнить».

Внутри «Если – выполнить»:

Ветка «Истина»: блок «Исполнить функцию». В поле указываете: TextTooLog('Папка для входящих найдена!')

Ветка «Ложь»: блок «Исполнить функцию» с вызовом TextTooLog('Критическая ошибка: папка C:\Incoming_Contracts не найдена. Проверьте структуру.')



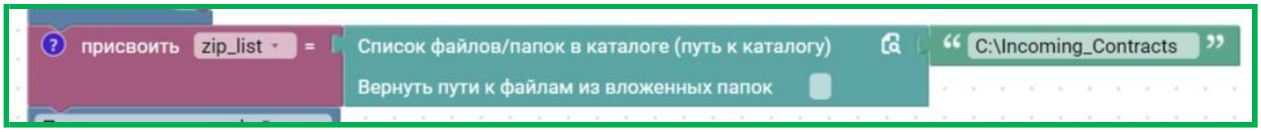
Шаг 2. Получить список ZIP-файлов в папке C:\Incoming_Contracts

Блок: «Список файлов/папок»

Директория: C:\Incoming_Contracts

Обход подпапок: FALSE

Результат сохраняем в переменную zip_list. Это список путей ко всем файлам в папке.



Шаг 3. Подсчитать количество файлов

Блок: «Присвоить значение переменной»

Выражение: len(zip_list)

Сохраняем в zip_count.

Шаг 4. Проверить, есть ли файлы

Блок: «Если – выполнить»

Условие: zip_count > 0

Ветка «Истина»:

Вызов TextTooLog('Найдены заявки в папке.')



Далее идёт цикл по каждому файлу (шаг 5).

Ветка «Ложь»:

Вызов TextTooLog('Нет заявок в папке, работа завершена.')

Шаг 5. Цикл по каждому архиву

Блок: «Цикл для каждого элемента»

Коллекция: zip_list

Переменная элемента: current_zip

Внутрь цикла помещаем шаги 5.1 – 5.12.

Шаг 5.1. Сформировать имя папки для распаковки

Блок: «Получить часть текста» (группа «Строковые функции»)

Исходный текст: current_zip

Искомая подстрока: 'contract' (левая граница)

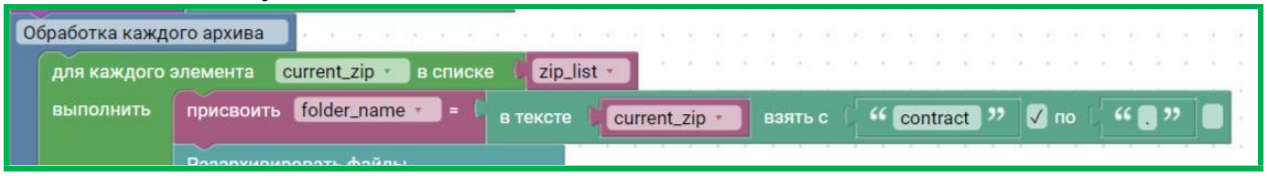
До подстроки: '.' (правая граница) – символ точки, до которого обрезать.

Флаг «Удалить найденную подстроку слева»: TRUE (чтобы отбросить 'contract' и всё, что до него)

Флаг «Удалить найденную подстроку справа»: FALSE

Результат сохраняем в folder_name.

Пояснение: из полного пути C:\Incoming_Contracts\contract_123.zip блок вырежет часть между 'contract' и '.'



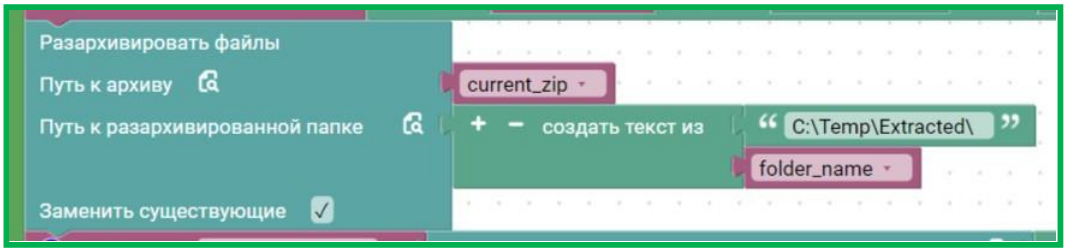
Шаг 5.2. Разархивировать файлы

Блок: «Разархивировать файлы» (группа «Файлы и папки»)

Исходный архив: current_zip

Папка для распаковки: 'C:\\Temp\\Extracted\\' + folder_name

Перезаписывать: TRUE



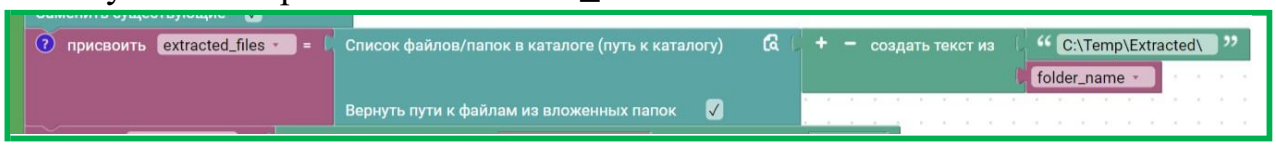
Шаг 5.3. Получить список всех распакованных файлов

Блок: «Список файлов/папок»

Директория: 'C:\\Temp\\Extracted\\' + folder_name

Обход подпапок: TRUE

Результат сохраняем в extracted_files.



Шаг 5.4. Превратить список файлов в строку (для удобной проверки)

Блок: «Сделать текст из списка»

Параметры:

Список: extracted_files

Разделитель: ;

Результат сохраняем в files_string. Теперь это одна строка вида "C:\Temp\Extracted\...\passport.pdf;C:\...\inn.pdf;...".



Шаг 5.5. Проверить наличие каждого обязательного PDF

Используем блок «Содержит ли строка подстроку» трижды:

- Для passport.pdf:
  Строка: files_string
  Подстрока: 'passport.pdf'
  Результат → has_passport
- Для inn.pdf:
  Подстрока: 'inn.pdf'
  Результат → has_inn
- Для bank_details.pdf:
  Подстрока: 'bank_details.pdf'
  Результат → has_bank

Все три результата – логические (True/False).

Шаг 5.6. Если все три файла присутствуют

Блок: «Если – выполнить» с условием has_passport and has_inn and has_bank.

Ветка «Истина» – здесь будет основная обработка (шаги 5.6.1 – 5.6.7).

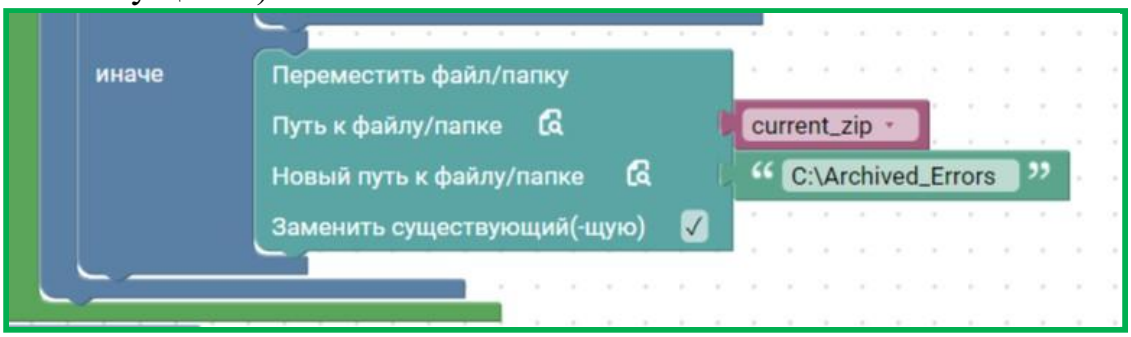


Ветка «Ложь» – перемещаем архив в папку ошибок и завершаем обработку этого архива (пропускаем остальные шаги).

В ложной ветке:

Блок «Переместить файл/папку»: исходный current_zip, назначение C:\Archived_Errors. Перед перемещением убедиться, что папка существует (или создать). В коде используется просто перемещение без проверки, но лучше проверить.

После перемещения – конец обработки текущего архива (переход к следующему элементу цикла).



Шаг 5.6.1. Извлечь текст из passport.pdf

Блок: «Считать текст из PDF-файла»

Путь: формируем как 'C:\\Temp\\Extracted\\' + folder_name + '\\passport.pdf' (можно через блок «Присвоить значение переменной» или прямо в поле).

Удалить водяные знаки: FALSE

Результат сохраняем в passport_text.

После этого вызываем TextTooLog('Данные из паспорта извлечены').

Шаг 5.6.2. Извлечь поля из паспорта (ФИО, серия, номер)

Используем блок «Получить часть текста» для каждого поля.

Все поля берутся из passport_text.

- Фамилия:

  Искомая подстрока: 'Фамилия:'

  До подстроки: ';' (символ точки с запятой)

  Очистить пробелы.

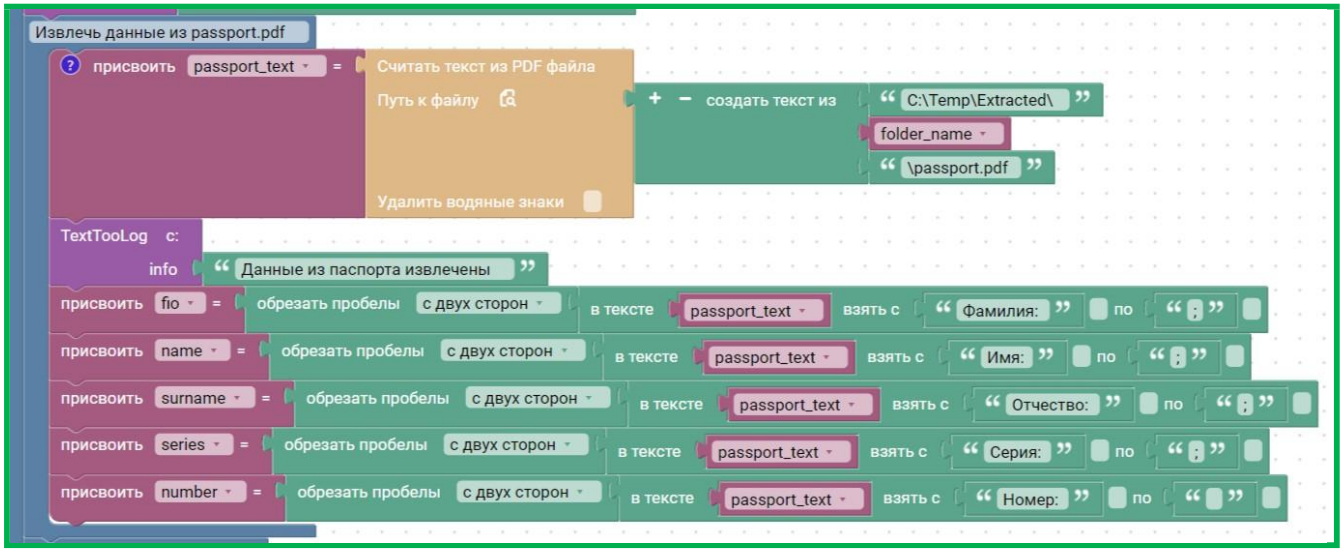
  Результат → fio

- Имя:

  Искомая подстрока: 'Имя:'

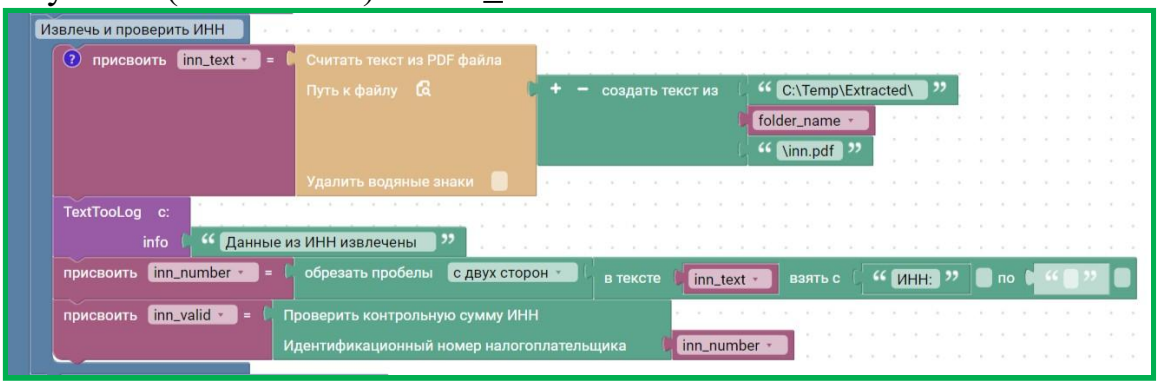
До: ';' → name

- Отчество:
  Искомая подстрока: 'Отчество:'
  До: ';' → surname
- Серия паспорта:
  Искомая подстрока: 'Серия:'
  До: ';' → series
- Номер паспорта:
  Искомая подстрока: 'Номер:'
  До подстроки: оставить пустым (до конца строки) → number



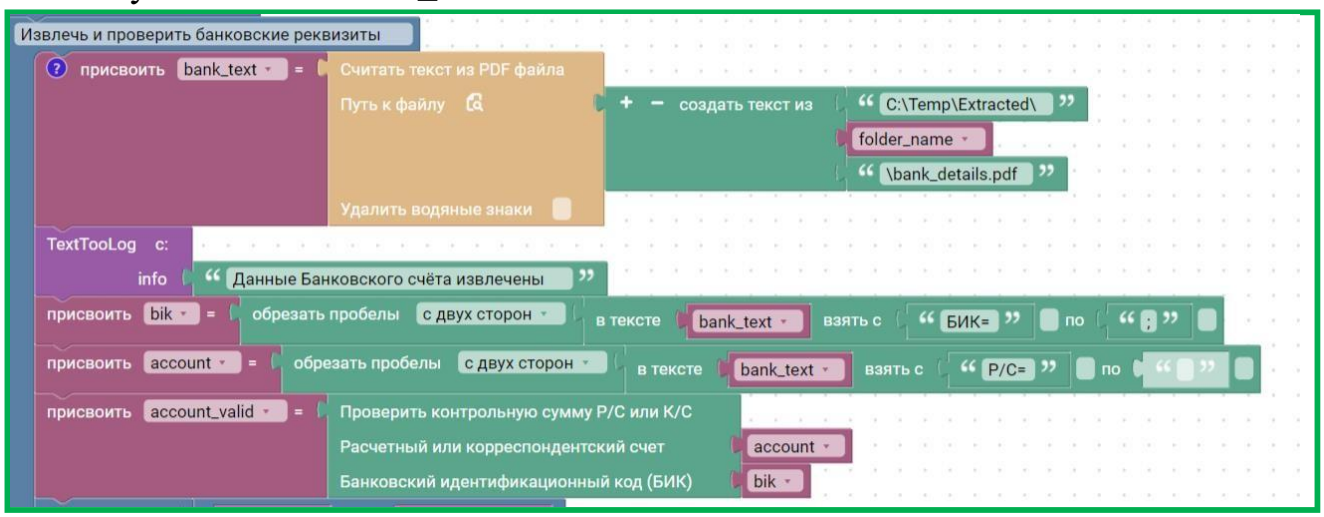
Шаг 5.6.3. Извлечь и проверить ИНН

- Блок: «Считать текст из PDF-файла»
  Путь: 'C:\\Temp\\Extracted\\' + folder_name + '\\inn.pdf'
  Результат → inn_text
  Вызов TextTooLog('Данные из ИНН извлечены')
- Блок: «Получить часть текста»
  Исходный текст: inn_text
  Искомая подстрока: 'ИНН:'
  До подстроки: пусто
  Результат → inn_number
- Блок: «Проверить контрольное число для ИНН» (группа «Валидация»)
  Параметр: inn_number
  Результат (логический) → inn_valid



Шаг 5.6.4. Извлечь и проверить банковские реквизиты

- Блок: «Считать текст из PDF-файла»
  Путь: 'C:\\Temp\\Extracted\\' + folder_name + '\\bank_details.pdf'
  Результат → bank_text
  Вызов TextTooLog('Данные Банковского счёта извлечены')
- Блок: «Получить часть текста» для БИК:
  Исходный: bank_text
  Искомая подстрока: 'БИК='
  До подстроки: ';'
  Результат → bik
- Блок: «Получить часть текста» для расчётного счёта:
  Исходный: bank_text
  Искомая подстрока: 'Р/С='
  До подстроки: пусто
  Результат → account
- Блок: «Проверить контрольное число для Р/С или К/С» (группа «Валидация»)
  Параметры: account, bik
  Результат → account_valid



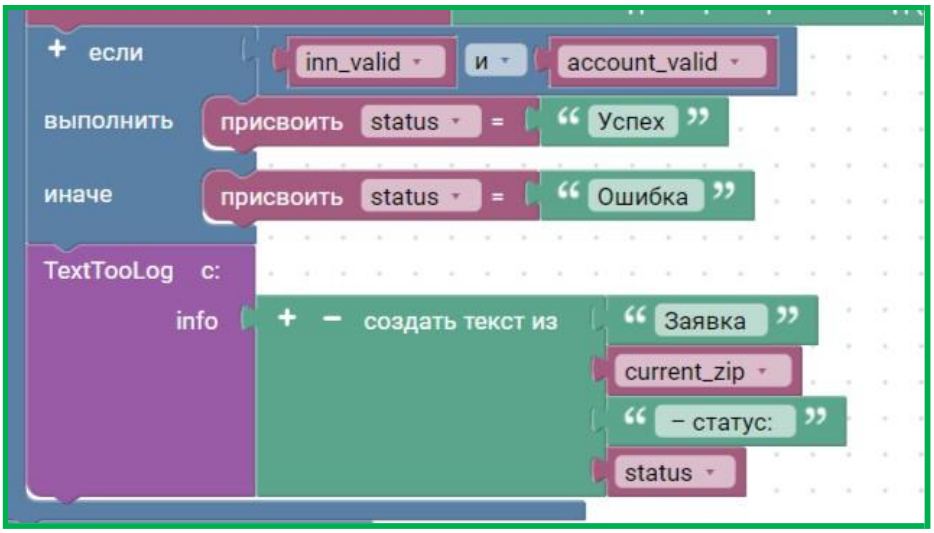
Шаг 5.6.5. Определить общий статус

Блок: «Если – выполнить» с условием inn_valid and account_valid

Ветка «Истина»: присвоить переменной status значение 'Успех'

Ветка «Ложь»: присвоить status = 'Ошибка'

После этого вызываем TextTooLog с сообщением 'Заявка ' + current_zip + ' – статус: ' + status.



Шаг 5.6.6. Запись в Excel-отчёт

Сначала проверяем, существует ли файл отчёта C:\Reports\verification.xlsx.

Если существует – добавляем строку. Если нет – создаём новый файл с заголовками, а потом добавляем строку.

Проверка существования файла:

- Блок: «Существует путь?» для 'C:\\Reports\\verification.xlsx'.

Если – выполнить:

Истина (файл есть): блок «Модифицировать строку в Excel» (группа «Excel»).

Путь к файлу: C:\Reports\verification.xlsx

Имя листа: 'Лист1'

Номер строки: -1 (означает добавить в конец)

Массив данных: [folder_name, fio, name, surname, series, number, inn_number, inn_valid, bik, account, account_valid, status]



Ложь (файла нет):

Сначала создаём шаблон словаря dict_excel с ключами (названия колонок) и пустыми значениями:

Имя архива, Фамилия, Имя, Отчество, Серия, Номер паспорта, ИНН, ИНН валиден, БИК, Расчётный счёт, Счёт валиден, Статус

Для этого используем блок «Присвоить значение переменной» и прямо пишем этот словарь.

Затем блок «Записать в Excel» с параметрами:

- Путь: C:\Reports\verification.xlsx
- Данные: dict_excel (так он создаст файл с заголовками)
- Имя листа: 'Лист1'

И наконец блок «Модифицировать строку в Excel» с row_id = -1, чтобы добавить первую строку данных.



Шаг 5.6.7. Переместить архив в папку успеха или ошибок

Блок: «Если – выполнить» по status == 'Успех'

Ветка «Успех»:

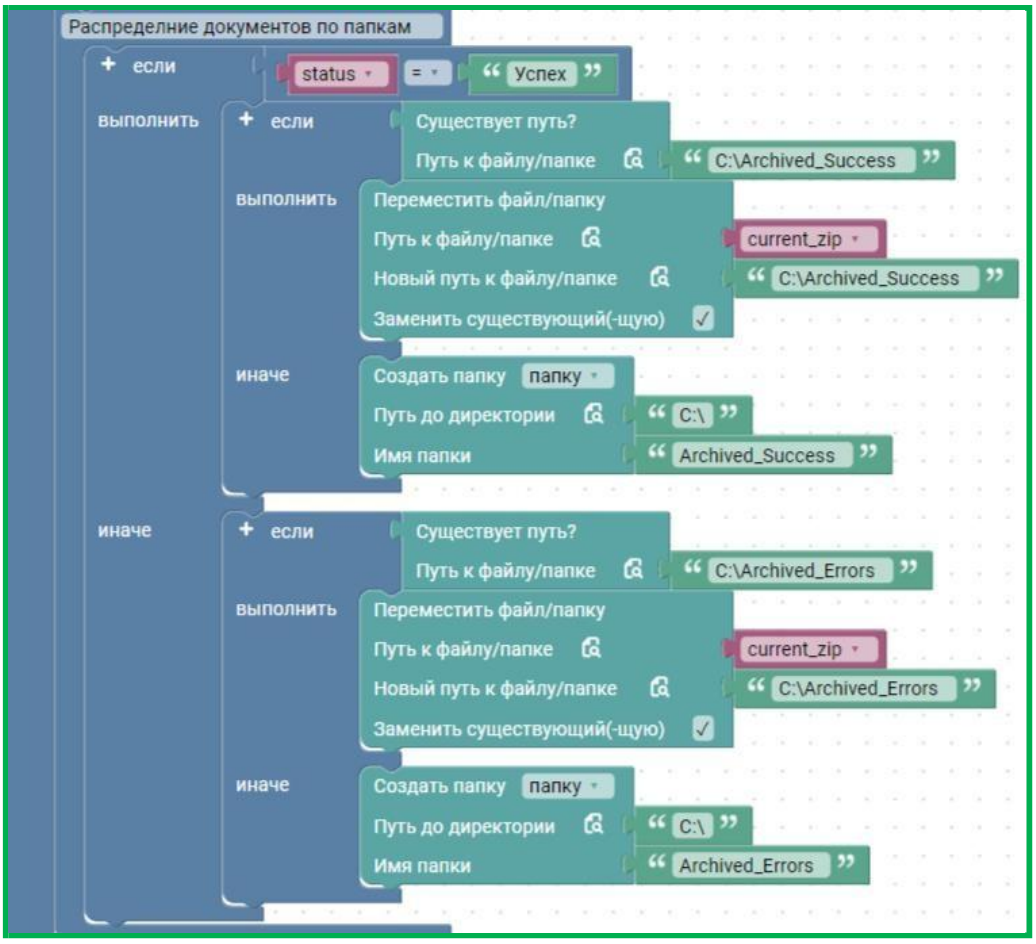
Проверить, существует ли C:\Archived_Success (блок «Существует путь?»).

Если нет – создать папку (блок «Создать папку»: родитель C:\, имя Archived_Success).

Переместить current_zip в C:\Archived_Success (блок «Переместить файл/папку»: исходный путь current_zip, назначение C:\Archived_Success, флаг замены TRUE).

Ветка «Ошибка»:

Аналогично для папки C:\Archived_Errors (создать, если нет, и переместить).



Мы завершили выполнение практической работы по автоматизации верификации контрагентов по PDF-документам с использованием Puzzle RPA.

В ходе работы мы:

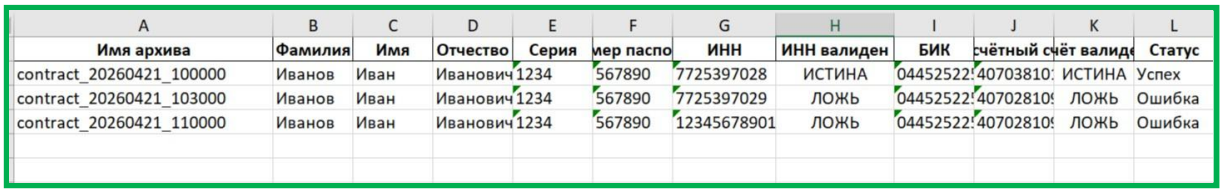
- Создали вспомогательную функцию логирования TextTooLog, которая записывает все ключевые события в файл log.txt для отладки и контроля работы робота.
- Разработали основной сценарий, который автоматически выполняет полный цикл проверки ZIP-архивов:
  - ✓ проверяет наличие входной папки C:\Incoming_Contracts и сигнализирует об ошибке при её отсутствии;
  - ✓ получает список архивов и обрабатывает каждый в цикле;
  - ✓ разархивирует файлы во временную папку;
  - ✓ проверяет наличие всех обязательных PDF-документов: passport.pdf, inn.pdf, bank_details.pdf;
  - ✓ извлекает из паспорта фамилию, имя, отчество, серию и номер паспорта;
  - ✓ извлекает номер ИНН и проверяет его контрольную сумму с помощью блока «Проверить контрольное число для ИНН»;
  - ✓ извлекает БИК и расчётный счёт, а затем проверяет контрольную сумму расчётного счёта по БИК с помощью блока «Проверить контрольное число для Р/С или К/С»;
  - ✓ формирует Excel-отчёт в папке C:\Reports с результатами проверок по каждой заявке (ФИО, паспортные данные, ИНН и его валидность, БИК, счёт и его валидность, общий статус);

✓ в зависимости от успешности проверок перемещает исходный ZIP-архив в папку C:\Archived_Success или C:\Archived_Errors.

Все шаги реализованы с помощью стандартных блоков Puzzle RPA.

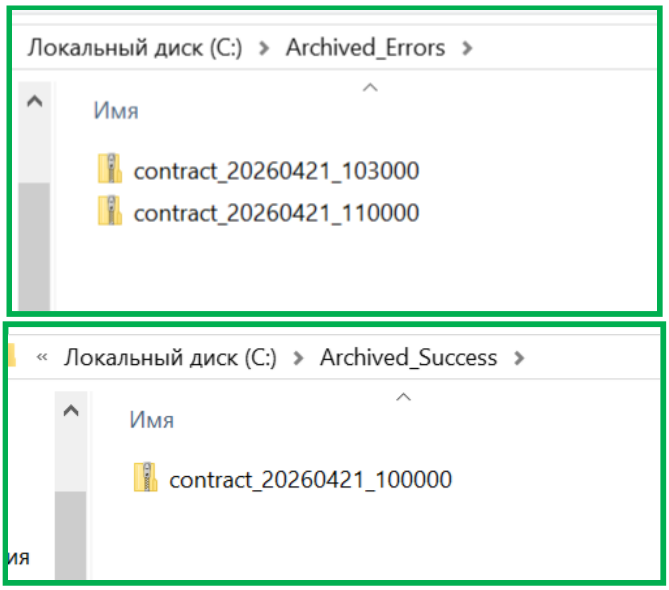
Результат работы робота можно проверить:

- В папке C:\Reports создаётся файл verification.xlsx, содержащий таблицу со всеми заявками, извлечёнными данными и результатами проверок (ИНН кор-ректен, счёт корректен, общий статус «Успех» или «Ошибка»).



A	B	C	D	E	F	G	H	I	J	K	L
Имя архива	Фамилия	Имя	Отчество	Серия	мер паспо	ИНН	ИНН валиден	БИК	счётный счёт валиде		Статус
contract_20260421_100000	Иванов	Иван	Иванович	1234	567890	7725397028	ИСТИНА	04452522	40703810	ИСТИНА	Успех
contract_20260421_103000	Иванов	Иван	Иванович	1234	567890	7725397029	ЛОЖЬ	04452522	40702810	ЛОЖЬ	Ошибка
contract_20260421_110000	Иванов	Иван	Иванович	1234	567890	12345678901	ЛОЖЬ	04452522	40702810	ЛОЖЬ	Ошибка

- В папках C:\Archived_Success и C:\Archived_Errors раскладываются исходные ZIP-архивы в зависимости от успешности верификации – успешные заявки отдельно, ошибочные отдельно.



Все действия робота записываются в log.txt благодаря функции TextTooLog, что упрощает отладку.

Данный сценарий полностью готов к использованию на реальных бизнес-кейсах, например, при автоматической проверке контрагентов перед открытием расчётного счёта или при приёме заявок на обслуживание. Студенты, выполнившие эту работу, научатся:

- проектировать RPA-процессы для обработки документооборота;
- применять локальные блоки валидации (ИНН, расчётный счёт) без подключения к внешним сервисам;
- работать с PDF и Excel, извлекать данные и формировать отчёты;
- распределять файлы по папкам в зависимости от результатов проверки;
- логировать и обрабатывать ошибки.

Теперь вы можете самостоятельно адаптировать робота под другие типы документов (например, добавить проверку ОГРН или КПП), подключить его к сетевым папкам или интегрировать с учётной системой.

Кейс 2. Автоматизация верификации контрагентов (с использованием Yandex Vision)

В компании «Финансовый контроль» ежедневно поступают заявки на открытие расчётных счетов для новых контрагентов. Вместе с заявкой в сетевую папку помещаются сканы или фотографии следующих документов:

- паспорт РФ (основной разворот) – файл passport.jpg или passport.png;
- ИНН (свидетельство) – файл inn.jpg или inn.png;
- файл bank_details.txt с текстовым описанием банковских реквизитов (БИК и расчётный счёт).

Робот должен автоматически:

- Распознать данные из паспорта (серия, номер, ФИО, дата выдачи) с помощью Yandex Vision;
- Распознать ИНН из изображения (10 или 12 цифр) и проверить его контрольную сумму;
- Извлечь из текстового файла БИК и номер расчётного счёта, проверить контрольную сумму счёта;
- Сформировать единый структурированный отчёт (Excel) и сохранить его в архивной папке;
- Переместить исходные файлы в папку «Успех» или «Ошибки» в зависимости от успешности проверки.

Важное примечание перед началом: облачный сервис Yandex Cloud

Для выполнения данного кейса требуется доступ к Yandex Vision – облачному сервису распознавания текста и документов от Yandex. Без подключения к этому сервису блоки «Распознать документ» и «Распознать области» работать не будут.

Что нужно сделать (до запуска робота):

- Зарегистрироваться в Yandex Cloud
- Перейдите на сайт cloud.yandex.ru. Нажмите «Начать бесплатно» или «Войти», используя свой Яндекс ID. Если у вас нет аккаунта, создайте его.
- Создать платёжный аккаунт. Yandex Cloud требует привязку банковской карты для активации аккаунта, даже для бесплатного периода. Не пугайтесь – на начальном этапе предоставляется стартовый грант (для резидентов РФ не менее 4000 ₽). В рамках учебного задания расходы составят копейки (одно распознавание стоит ≈ 0,01 ₽), и гранта хватит на тысячи таких операций.
- Получить OAuth-токен. OAuth-токен – это ключ, который подтверждает ваше право выполнять операции от имени вашего аккаунта. В консоли Yandex Cloud перейдите в раздел «Настройки» → «Токены». Нажмите «Создать токен». Скопируйте полученный длинный набор символов (начинается с y0_). Сохраните его в надёжном месте, он понадобится при настройке робота. **Никогда не публикуйте этот токен в открытом доступе.**
- Получить идентификатор каталога (Folder ID). Каталог – это контейнер для ресурсов в Yandex Cloud. На главной странице консоли вы увидите список ката-

логов (как правило, один каталог с именем default или созданный вами). Наведите курсор на имя каталога, появится всплывающая подсказка с Folder ID. Скопируйте его (выглядит как b1gxxxxxxxxxxxx).

- Сохранить токен и Folder ID в защищённом хранилище робота

В Puzzle RPA используйте блок «Добавить учётную запись» (группа «Учётные записи»). Создайте две записи: Имя хранилища: YandexToken, Пользователь: OAuth, Пароль: (вставьте ваш OAuth-токен). Имя хранилища: YandexFolder, Пользователь: FolderID, Пароль: (вставьте ваш Folder ID).

Это позволит роботу получать секретные данные без их прописывания в коде.

Что может пойти не так и как избежать ошибок:

- Нет доступа к интернету – Yandex Vision работает только онлайн. Убедитесь, что компьютер робота имеет выход в интернет.
- Истёк грант или закончились средства – один раз проверьте баланс в консоли. Для учебных целей обычно достаточно 100–200 операций распознавания, это копейки.
- Неверный токен или Folder ID – внимательно скопируйте их, не добавляйте лишних пробелов. Робот не сможет выполнить распознавание и упадёт с ошибкой авторизации.
- Ограничения на изображения – размер файла до 1 МБ, разрешение не более 20 Мп, угол перекоса не более 45°. При нарушениях распознавание может вернуть пустой результат или ошибку.

Что нужно создать для кейса:

- Необходимые папки на диске C.
- Подпапки с именами по шаблону.
- В каждой подпапке файлы:
- passport.jpg (изображение с паспортными данными)
- inn.jpg (изображение с номером ИНН)
- bank_details.txt (текстовый файл с БИК и расчётным счётом)

Перед тем как собирать основной сценарий для кейса, мы создадим вспомогательную функцию TextTooLog. Она будет записывать в файл log.txt (который создаётся рядом с исполняемым файлом робота) все важные сообщения с указанием даты и времени. Это необходимо для отладки и контроля работы робота.



Шаг 1. Проверка существования входной папки

Блок – «Существует путь?»

Путь: C:\Incoming_Contracts

«Если – выполнить» (подхватывает результат предыдущего блока)

Ветка «Истина»: «Исполнить функцию» (вызов TextTooLog): параметр info = 'Папка для входящих найдена!'

Ветка «Ложь»: «Исполнить функцию» (вызов TextTooLog): 'Критическая ошибка: папка C:\\Incoming_Contracts не найдена. Проверьте структуру.'



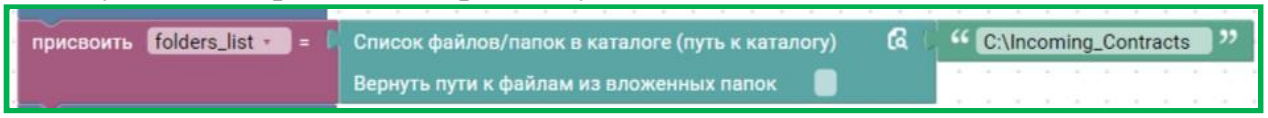
Шаг 2. Получение списка подпапок

Блок – «Список файлов/папок»

Директория: C:\Incoming_Contracts

Флаг «Обход подпапок»: FALSE

Результат сохранить в переменную folders_list



Шаг 3. Проверяем наличие файлов в папке

Блок – «Присвоить значение переменной»

Выражение: len(folders_list)

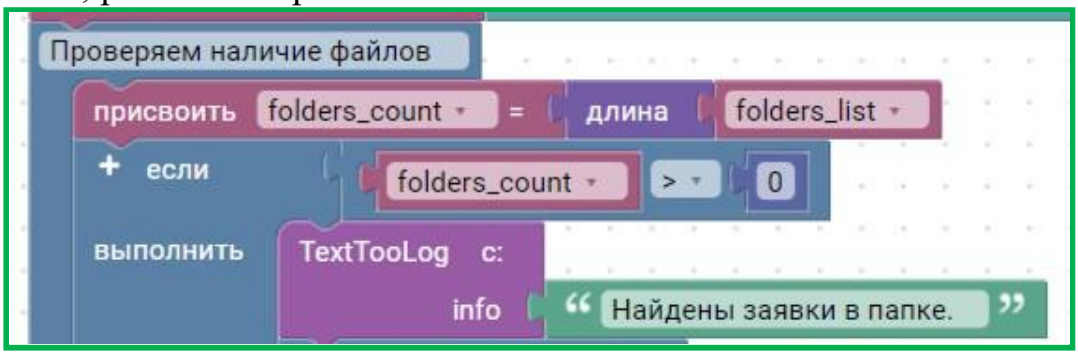
Результат сохранить в folders_count

Шаг 4. Проверка наличия заявок

Блок – «Если – выполнить» с условием folders_count > 0

Ветка «Истина»: «Исполнить функцию» (TextTooLog) с сообщением 'Найдены заявки в папке.' Далее внутри этой ветки размещаем весь цикл обработки (блоки 8–…).

Ветка «Ложь»: «Исполнить функцию» (TextTooLog) с сообщением 'Нет заявок в папке, работа завершена.'



Шаг 5. Цикл по каждой папке-заявке

Блок – «Цикл для каждого элемента»

Коллекция: folders_list

Переменная для текущего элемента: current_folder

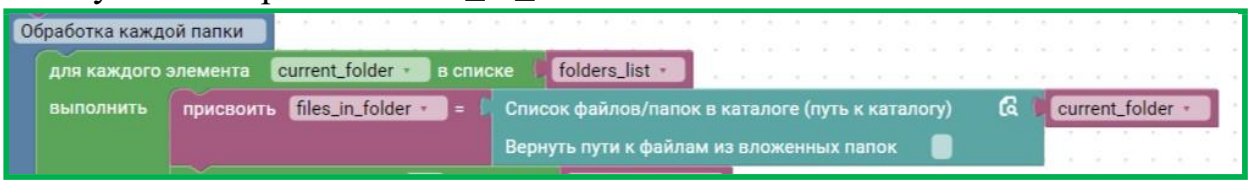
Внутри цикла выполняем шаги 5.1–5.12.

5.1. Получение списка файлов внутри текущей папки

Блок – «Список файлов/папок»

Директория: current_folder

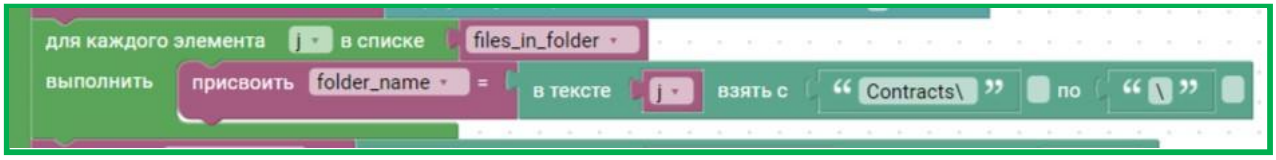
Результат сохранить в files_in_folder



5.2. Вложенный цикл для извлечения имени папки.

«Цикл для каждого элемента» по files_in_folder – переменная j

Внутри «Получить часть текста»: исходный текст j, искать между 'Contracts\\' и '\\', флаги удаления FALSE, результат folder_name.



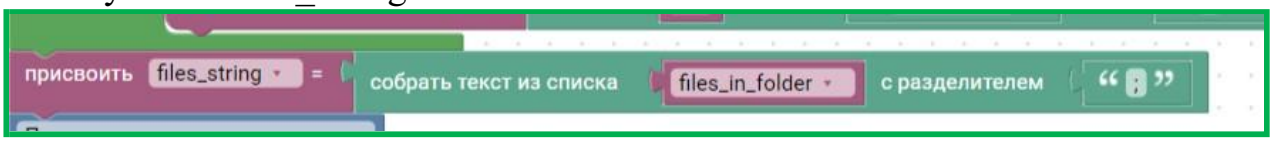
5.3. Преобразование списка файлов в строку

Блок – «Сделать текст из списка»

Список: files_in_folder

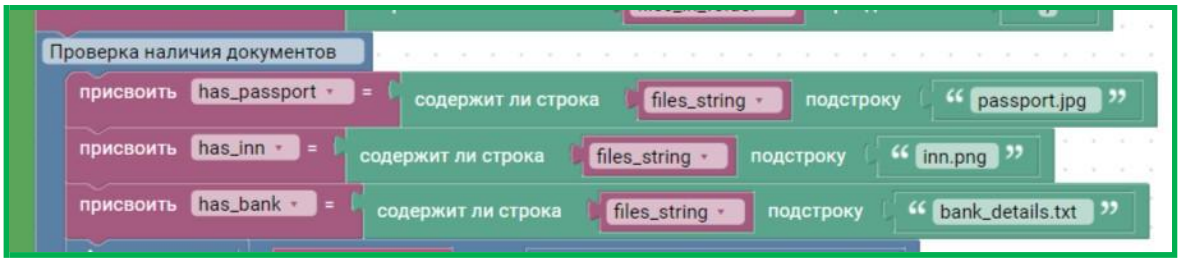
Разделитель: ; (точка с запятой)

Результат: files_string



5.4. Проверка наличия обязательных файлов

- Блок – «Содержит ли строка подстроку»
  Исходная строка: files_string
  Подстрока: 'passport.jpg'
  Результат: has_passport
- Блок – «Содержит ли строка подстроку»
  Подстрока: 'inn.png' (в коде именно .png)
  Результат: has_inn
- Блок – «Содержит ли строка подстроку»
  Подстрока: 'bank_details.txt'
  Результат: has_bank



Блок – «Если – выполнить» с условием has_passport and has_inn and has_bank

Ветка «Истина» (все файлы есть):

Вызов TextTooLog('Все файлы найдены. Продолжаем обработку.')

Далее блоки 5.5–5.9.

Ветка «Ложь» (нет какого-то файла):

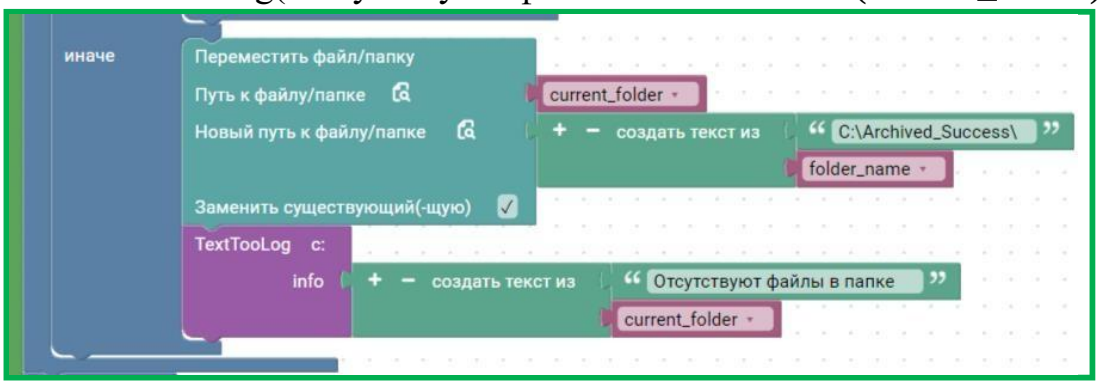
«Переместить файл/папку» (группа «Файлы и папки»):

Исходный путь: current_folder

Путь назначения: 'C:\\Archived_Errors\\' + str(folder_name)

Заменить существующий: TRUE

Вызов TextTooLog('Отсутствуют файлы в папке ' + str(current_folder))



5.5. Получение учетных данных Yandex Cloud (внутри истинной ветки)

Блок – «Получить пароль для учётной записи»

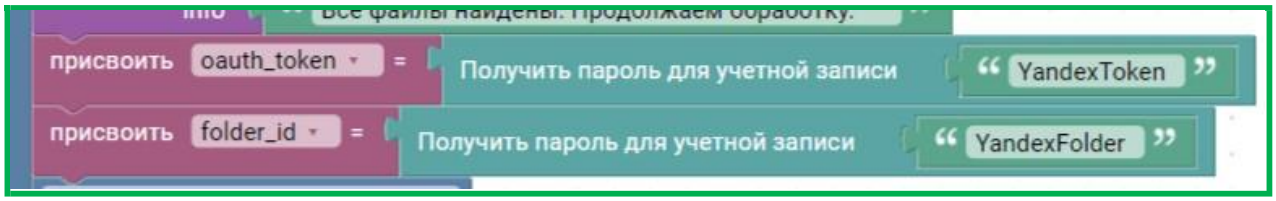
Хранилище: YandexToken

Результат: oauth_token

Блок – «Получить пароль для учётной записи»

Хранилище: YandexFolder

Результат: folder_id



5.6. Распознавание паспорта

- Блок – «Распознать документ (Yandex Vision)» (группа «Распознавание»)

  Файл документа: str(current_folder) + '\\passport.jpg'

  OAuth-токен: oauth_token

  Идентификатор каталога: folder_id

  Языковая модель: список ['ru']

  Результат: passport_text
- Блок – «Присвоить значение переменной»

Выражение: passport_text[1]
Результат: passport_fields_list (это список словарей с полями)

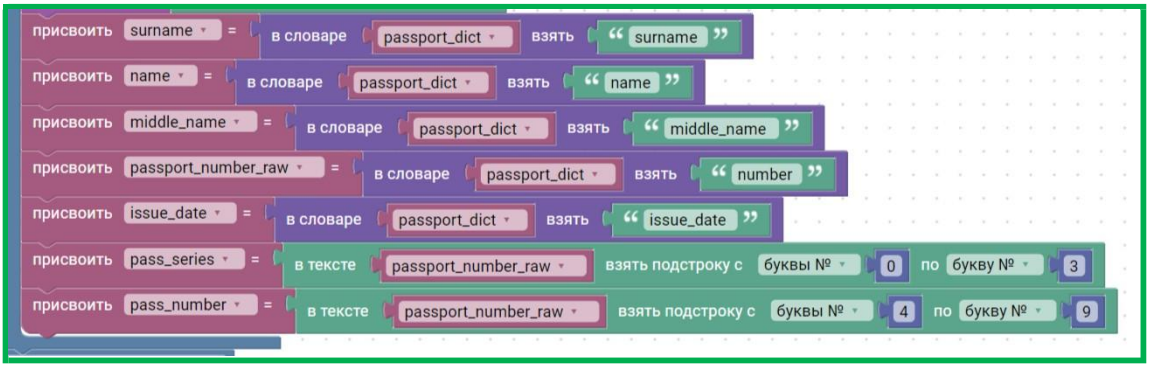


- Блок – «Присвоить значение переменной»
  Выражение: {} (пустой словарь)
  Результат: passport_dict
  Блок – «Цикл для каждого элемента» по passport_fields_list, переменная field
  Внутри: «Добавить элемент в словарь» (группа «Словари»)
  Словарь: passport_dict
  Ключ: field['name']
  Значение: field['text']
  После цикла – вызов TextTooLog('Данные из паспорта извлечены').



5.7. Извлечение конкретных полей паспорта

Блок – surname = passport_dict['surname']
Блок – name = passport_dict['name']
Блок – middle_name = passport_dict['middle_name']
Блок – passport_number_raw = passport_dict['number']
Блок – issue_date = passport_dict[' issue_date']
Блок – pass_series = passport_number_raw[: 4] (первые 4 символа)
Блок – pass_number = passport_number_raw[4 : 10] (следующие 6 символов)

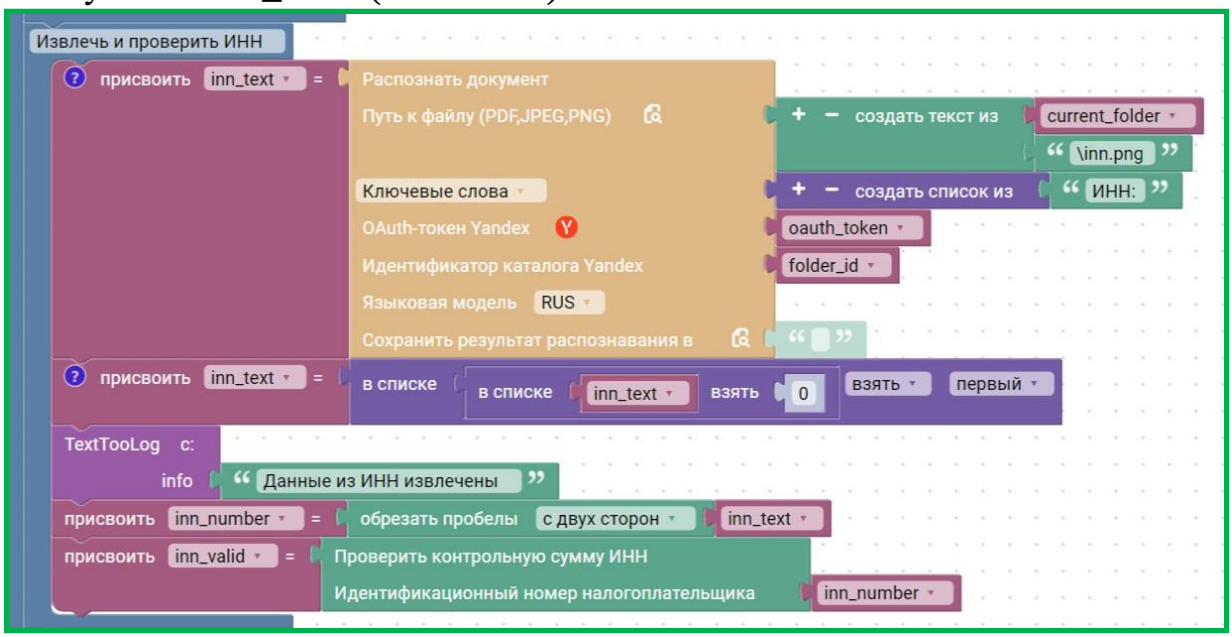


5.8. Распознавание и проверка ИНН

- Блок – «Распознать области» (Yandex Vision) – в коде yandex_vision.yandex_vision_areas
  Файл документа: str(current_folder) + '\\inn.png'

Режим: "ключевые слова")
Параметры поиска: ['ИНН:']
OAuth-токен: oauth_token
Идентификатор каталога: folder_id
Языковая модель: ['ru']
Результат: inn_text (список)

- Блок – inn_text = (inn_text[0])[0] – берём первый элемент первого элемента, получаем строку.
- Блок – inn_number = inn_text.strip() – удаляем пробелы по краям.
- Блок – «Проверить контрольную сумму ИНН» (группа «Валидация»)
  ИНН: inn_number
  Результат: inn_valid (True/False)



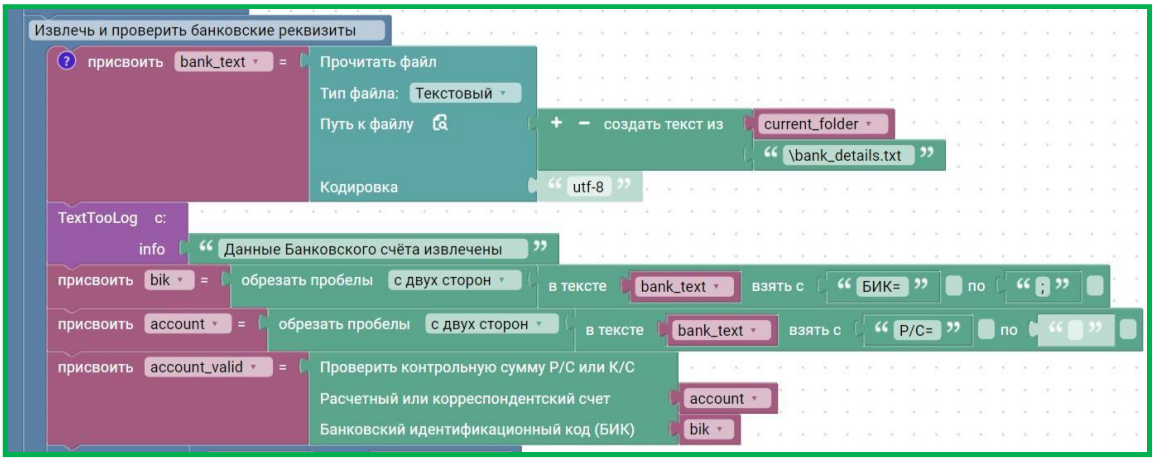
5.9. Извлечение и проверка банковских реквизитов

- Блок – «Прочитать файл»
  Кодировка: utf-8
  Путь: str(current_folder) + '\\bank_details.txt'
  Результат: bank_text
- Блок – «Получить часть текста»
  Исходная строка: bank_text
  Искомая подстрока: 'БИК='
  До подстроки: ';'
  Флаги удаления: FALSE, FALSE
  Результат очистить .strip() → bik
- Блок – «Получить часть текста»
  Исходная строка: bank_text
  Искомая: 'Р/С='
  До подстроки: оставить пустым (до конца строки)
  Результат: account
- Блок – «Проверить контрольную сумму Р/С или К/С»

Расчётный счёт: account

БИК: bik

Результат: account_valid



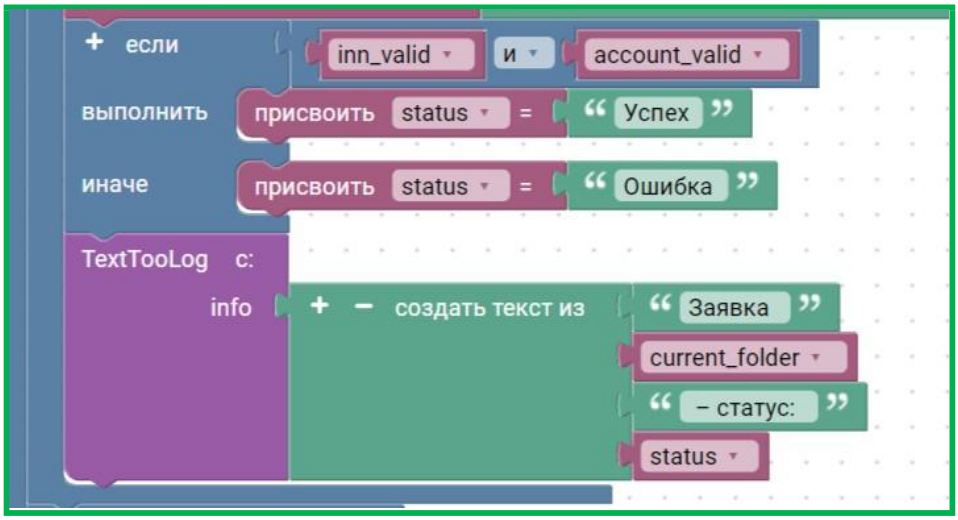
5.10. Определение статуса заявки

Блок – «Если – выполнить» с условием inn_valid and account_valid

Истина: status = 'Успех'

Ложь: status = 'Ошибка'

Блок – Вызов TextTooLog с сообщением 'Заявка ' + current_folder + ' – статус: ' + status



5.11. Формирование Excel-отчёта

Блок – «Существует путь?» для C:\\Reports\\verification_report.xlsx

Если файл существует:

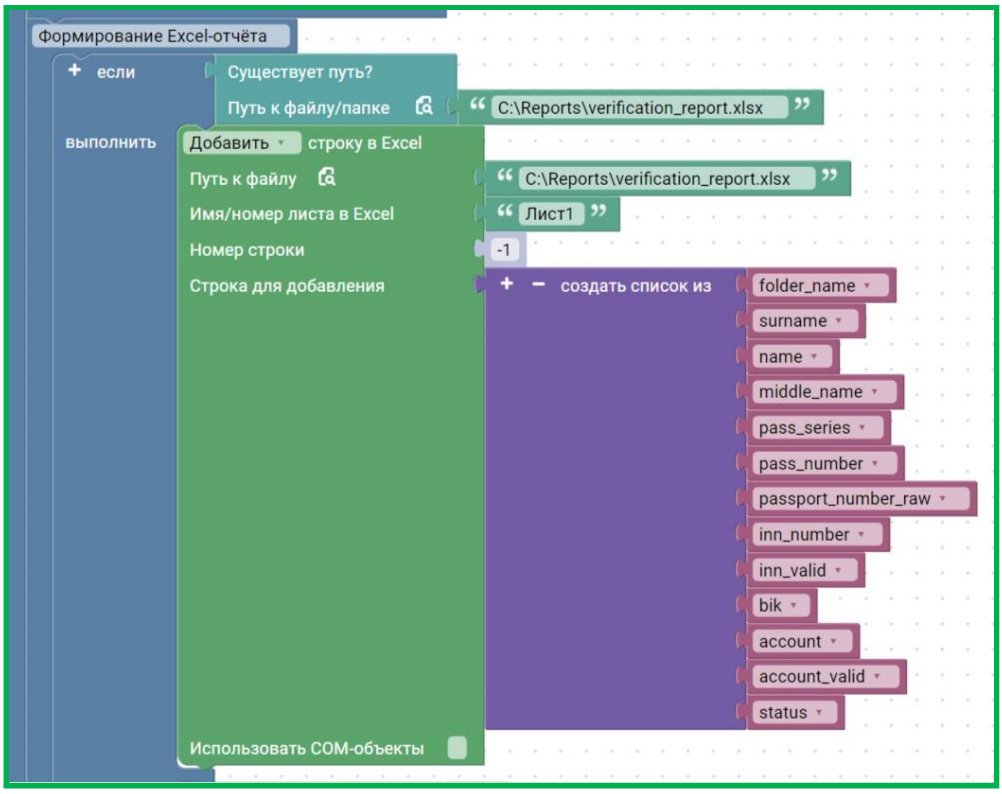
«Модифицировать строку в Excel» (группа «Excel»)

Путь: C:\\Reports\\verification_report.xlsx

Лист: 'Лист1'

Номер строки: -1 (добавить в конец)

Массив данных: [folder_name, surname, name, middle_name, pass_series, pass_number, passport_number_raw, inn_number, inn_valid, bik, account, account_valid, status]



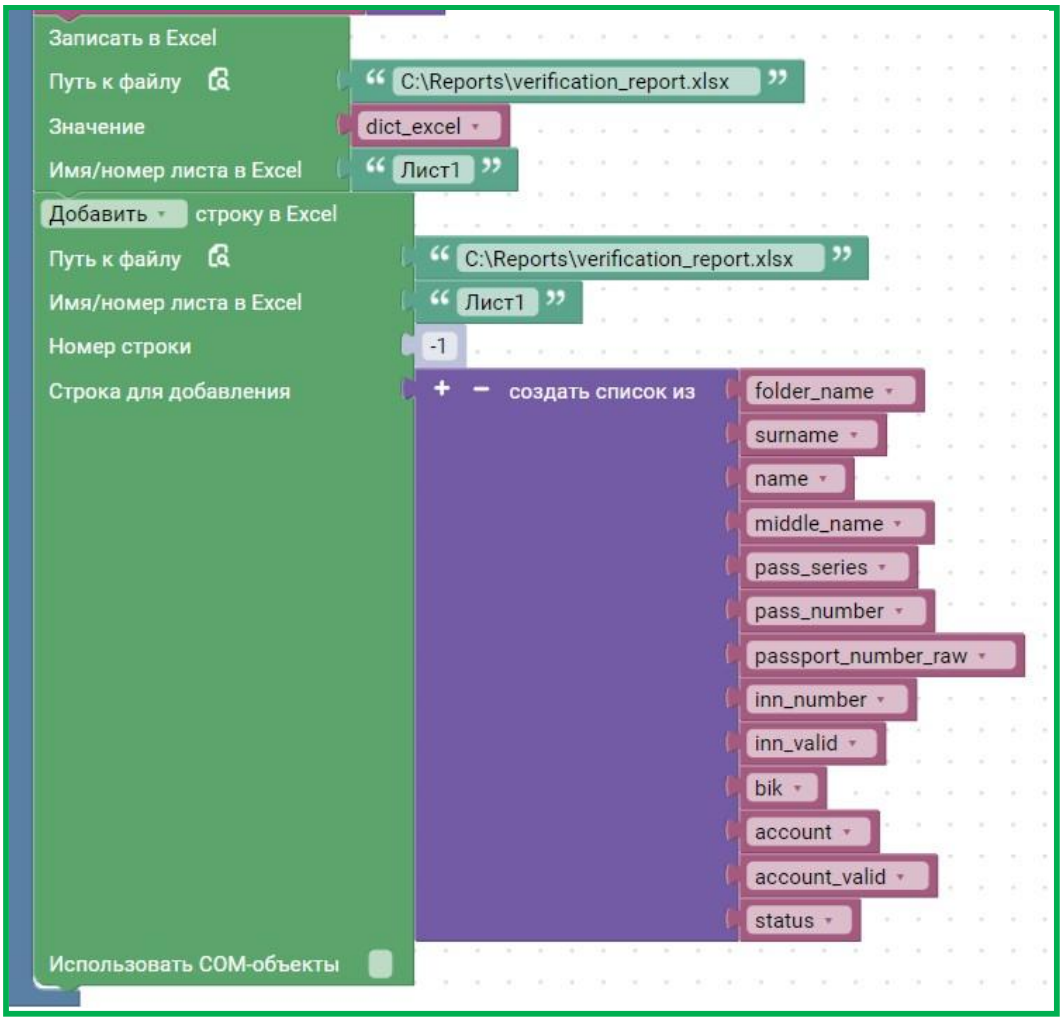
Если файла нет:

Создать словарь dict_excel с ключами-заголовками и пустыми значениями (блок «Присвоить значение переменной» – прямо написать словарь).



«Записать в Excel» (группа «Excel»): путь C:\\Reports\\verification_report.xlsx, данные = dict_excel, лист 'Лист1' – создаст файл с заголовками.

Затем «Модифицировать строку в Excel» (как выше) добавить первую строку данных.

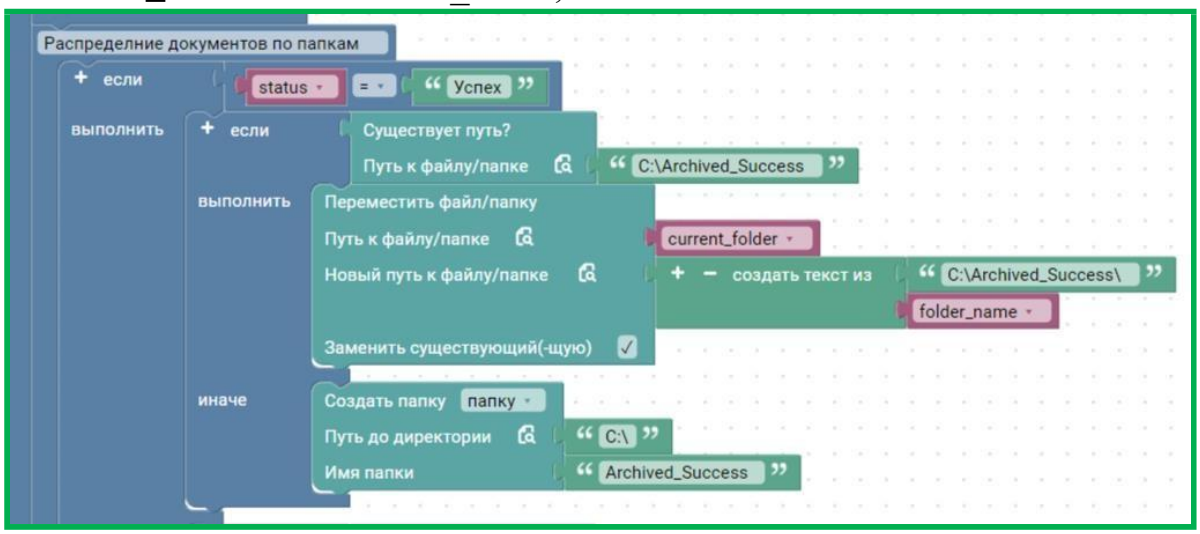


5.12. Перемещение папки в «Успех» или «Ошибки»

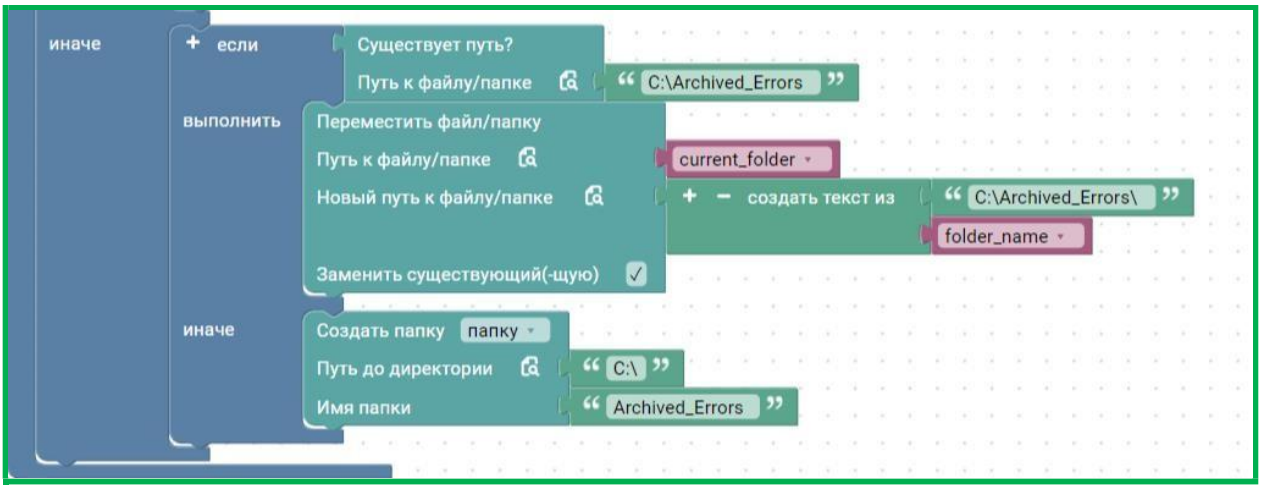
Блок – «Если – выполнить» по status == 'Успех'

Внутри: проверить существование C:\\Archived_Success. Если нет – создать папку.

Затем «Переместить файл/папку»: исходный current_folder, назначение 'C:\\Archived_Success\\' + folder_name, заменять TRUE.



Аналогично для статуса 'Ошибка' – в папку C:\\Archived_Errors.

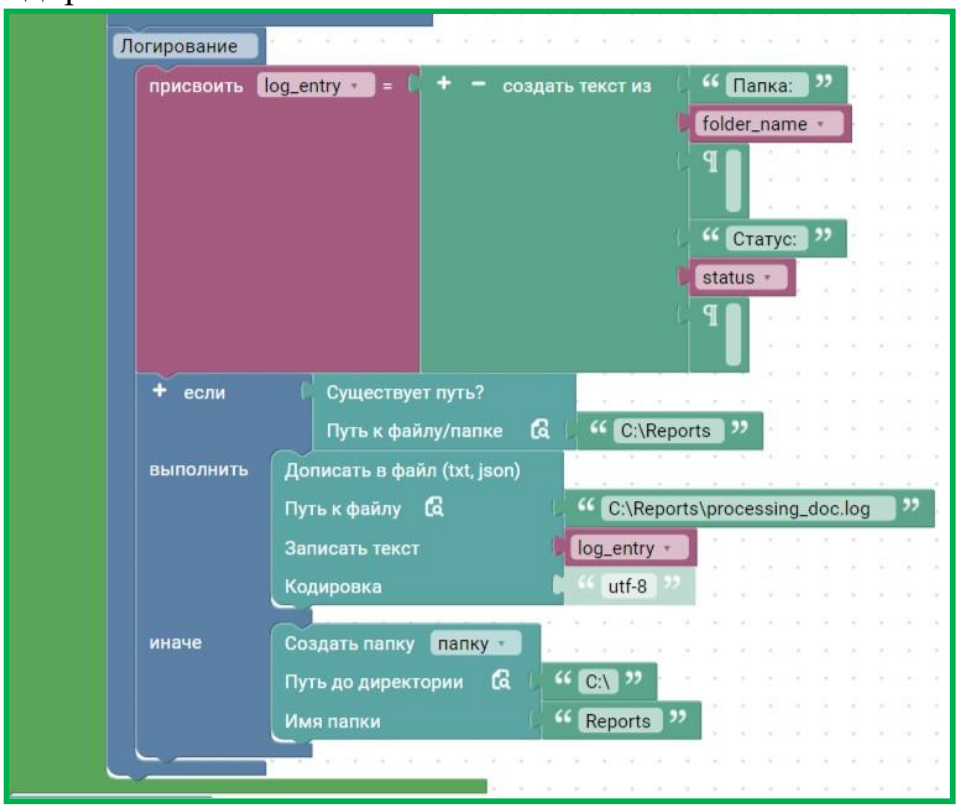


5.13. Логирование в отдельный файл

Блок – Собрать строку log_entry (имя папки, статус, перевод строки).

Блок – Проверить существование папки C:\\Reports. Если нет – создать.

Блок – «Дописать в файл»: путь C:\\Reports\\processing_doc.log, данные log_entry, кодировка utf-8.



Мы завершили выполнение практической работы по автоматизации верификации контрагентов по сканам документов с использованием Puzzle RPA и облачного сервиса Yandex Vision.

В ходе работы мы:

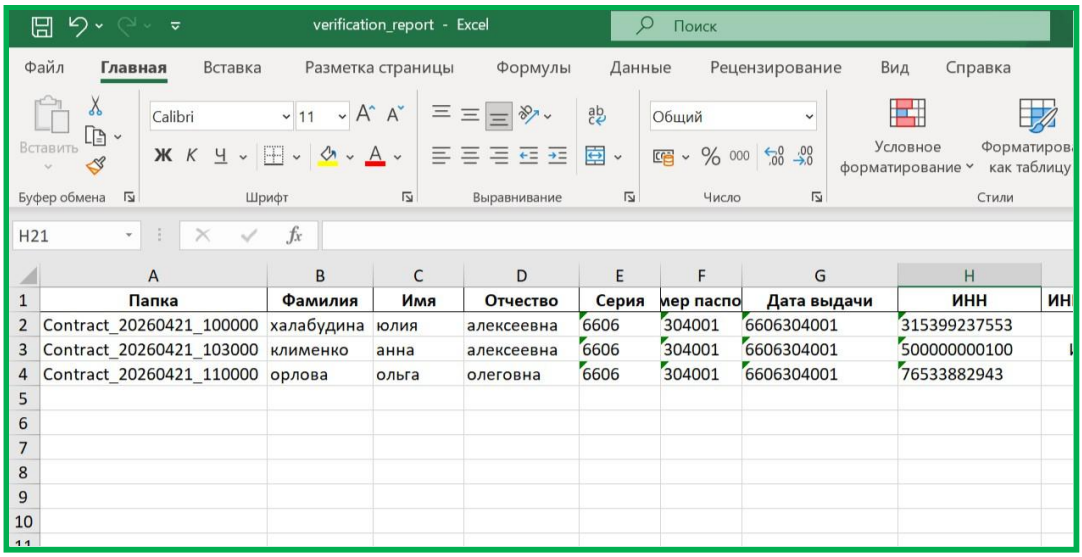
- Создали вспомогательную функцию логирования TextTooLog, которая записывает все ключевые события в файл log.txt для отладки и контроля работы робота.
- Настроили интеграцию с Yandex Cloud: получили OAuth-токен и идентификатор каталога, сохранили их в защищённое хранилище учётных записей.

- Разработали основной сценарий, который автоматически выполняет полный цикл проверки заявок на основе изображений и текстовых файлов:
  - ✓ проверяет наличие входной папки C:\Incoming_Contracts и сигнализирует об ошибке при её отсутствии;
  - ✓ получает список папок-заявок и обрабатывает каждую в цикле;
  - ✓ проверяет наличие обязательных файлов: passport.jpg (или .png), inn.jpg (или .png), bank_details.txt;
  - ✓ распознаёт данные паспорта через блок «Распознать документ» Yandex Vision (фамилия, имя, отчество, номер паспорта, дата выдачи) и преобразует результат в удобный словарь;
  - ✓ распознаёт номер ИНН через блок «Распознать области» с ключевым словом «ИНН» и очищает от нецифровых символов;
  - ✓ проверяет контрольную сумму ИНН с помощью блока «Проверить контрольное число для ИНН»;
  - ✓ извлекает из текстового файла bank_details.txt БИК и расчётный счёт (через поиск подстрок «БИК=» и «Р/С=»);
  - ✓ проверяет контрольную сумму расчётного счёта по БИК с помощью блока «Проверить контрольное число для Р/С или К/С»;
  - ✓ формирует Excel-отчёт в папке C:\Reports с результатами проверок по каждой заявке (имя папки, ФИО, серия и номер паспорта, дата выдачи, ИНН и его валидность, БИК, расчётный счёт, валидность счёта, общий статус);
  - ✓ в зависимости от успешности проверок (оба флага True) перемещает всю папку-заявку в C:\Archived_Success или C:\Archived_Errors с сохранением структуры;
  - ✓ ведёт текстовый лог processing.log с датой, именем папки и результатом проверки.

Все шаги реализованы с помощью стандартных и облачных блоков Puzzle RPA: работа с файлами и папками, строковые функции, циклы и условия, работа со словарями и списками, вызов Yandex Vision для распознавания изображений, валидация ИНН и расчётного счёта, запись в Excel и дополнение логов.

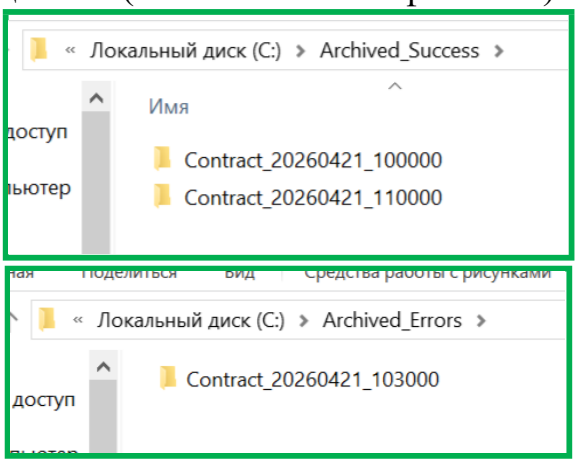
Результат работы робота можно проверить:

В папке C:\Reports создаётся файл verification_report.xlsx, содержащий таблицу со всеми заявками, извлечёнными данными и результатами проверок (ИНН корректен, счёт корректен, общий статус «Успех» или «Ошибка»).



	A	B	C	D	E	F	G	H
1	Папка	Фамилия	Имя	Отчество	Серия	мер паспо	Дата выдачи	ИНН
2	Contract_20260421_100000	халабудина	юлия	алексеевна	6606	304001	6606304001	315399237553
3	Contract_20260421_103000	клименко	анна	алексеевна	6606	304001	6606304001	500000000100
4	Contract_20260421_110000	орлова	ольга	олеговна	6606	304001	6606304001	76533882943
5								
6								
7								
8								
9								
10								

В папках C:\Archived_Success и C:\Archived_Errors раскладываются исходные папки-заявки в зависимости от успешности верификации – успешные заявки отдельно, ошибочные отдельно (вместе со всеми файлами).



Файл log.txt (от TextTooLog) и processing.log фиксируют все этапы обработ-ки, включая ошибки распознавания или отсутствие файлов.

Данный сценарий полностью готов к использованию на реальных бизнес-кейсах, например, при автоматической проверке контрагентов перед открытием расчётного счёта на основе скан-копий паспорта и ИНН. Студенты, выполнившие эту работу, научатся:

- проектировать RPA-процессы, интегрированные с облачными сервисами распознавания;
- применять блоки работы с Yandex Vision для извлечения структурированных данных из изображений паспорта и свидетельства ИНН;
- обрабатывать результаты распознавания (списки словарей) и преобразовывать их в удобные форматы;
- использовать блоки валидации ИНН и расчётного счёта;
- формировать отчёты в Excel и логировать действия робота;
- корректно обрабатывать ошибки и перемещать файлы по результатам проверки.

Теперь вы можете самостоятельно адаптировать робота под другие типы документов (например, водительские удостоверения, счета-фактуры), а также под-

ключать его к сетевым папкам и корпоративным базам данных. Полученные навыки позволяют переходить к более сложным RPA-проектам, включающим распознавание и валидацию документов в реальном времени.

Практическая часть

Вариант 11. Проверка физических лиц (паспорт+ИНН+СНИЛС)

📖 Описание ситуации

В отдел кадров поступают заявки на проверку физических лиц. Каждая заявка представляет собой ZIP-архив, содержащий три PDF-файла (все с текстовым слоем):

- passport.pdf – данные паспорта (фамилия, имя, отчество, серия, номер);
- inn.pdf – номер ИНН (10 или 12 цифр);
- snils.pdf – номер СНИЛС (11 цифр в формате XXX-XXX-XXX-XX).

Робот должен:

1. Распаковать архив.
2. Извлечь текст из каждого PDF.
3. Проверить контрольную сумму СНИЛС.
4. Сформировать Excel-отчёт с результатами проверки.
5. Переместить архив в папку «Успех» или «Ошибки» в зависимости от результата.

🎯 Цель работы

Научиться применять локальные блоки Puzzle RPA для работы с PDF и проверки контрольных сумм:

- Разархивировать файлы
- Считать текст из PDF файла (или Извлечь данные из PDF)
- Проверить контрольную сумму СНИЛС
- Записать в Excel
- Переместить файл/папку
- Создать папку, Существует путь, Список файлов

📥 Входные данные

На диске в папке C:\Incoming_Persons находятся ZIP-архивы с именами вида person_*.zip. Внутри каждого архива находятся три PDF-файла:

Имя файла	Содержимое (текстовый слой)
passport.pdf	Фамилия: Козлов; Имя: Дмитрий; Отчество: Владимирович; Серия: 4503; Номер: 789012
inn.pdf	ИНН: 7725397028
snils.pdf	СНИЛС: 111-222-333-44

Важно: все PDF создаются с текстовым слоем (не сканы). Студент самостоятельно готовит тестовые PDF (инструкция ниже).

📋 **Задание**

Этап 0. Подготовка рабочей среды (один раз)

0.1. Создать структуру папок:

C:\Incoming_Persons\ - папка для входящих ZIP-архивов

C:\Archived_Success\ - папка для успешно обработанных

C:\Archived_Errors\ - папка для заявок с ошибками

C:\Reports\ - папка для итоговых отчётов

C:\Temp\Extracted\ - временная папка для распаковки

0.2. Создать тестовые PDF-файлы согласно инструкции в разделе «Данные для подготовки».

Этап 1. Подготовка и проверка среды

1.1. Проверить существование папки C:\Incoming_Persons. Если нет – создать.

1.2. Получить список всех ZIP-файлов в C:\Incoming_Persons (блок «Список файлов/папок», маска person_*.zip). Сохранить в переменную zip_list.

1.3. Если список пуст – вывести сообщение «Нет заявок» и завершить робота.

Этап 2. Цикл обработки каждой заявки

Для каждого ZIP-файла из zip_list:

2.1. **Разархивировать** во временную папку C:\Temp\Extracted\[имя архива без .zip].

2.2. **Проверить наличие файлов** passport.pdf, inn.pdf, snils.pdf. Если какого-то нет – переместить ZIP в Archived_Errors и перейти к следующему.

2.3. **Извлечь данные из passport.pdf**:

- Использовать блок «Считать текст из PDF файла».
- Из полученного текста извлечь все поля: фамилию, имя, отчество, серию, номер паспорта.

2.4. **Извлечь и проверить СНИЛС** из snils.pdf:

- Считать текст, найти 11 цифр (в формате XXX-XXX-XXX-XX или сплошной строкой).
- Передать в блок «Проверить контрольную сумму СНИЛС». Результат сохранить в snils_valid.

2.5. **Принять решение**:

- Если snils_valid = True → статус «Успех».
- Иначе → статус «Ошибка».

Этап 3. Формирование Excel-отчёта

3.1. Если файл C:\Reports\verification.xlsx не существует – создать его с заголовками: Имя архива, Фамилия, Имя, Отчество, Серия, Номер паспорта, ИНН, СНИЛС, СНИЛС валиден, Статус.

3.2. Добавить строку с результатами текущей заявки.

Этап 4. Завершающие операции

4.1. Переместить ZIP-архив в папку Archived_Success или Archived_Errors в зависимости от статуса.

4.2. Удалить временную распакованную папку.

📊 Ожидаемый результат

- Excel-файл C:\Reports\verification.xlsx с результатами проверки всех заявок.
- ZIP-архивы рассортированы по папкам «Успех» и «Ошибки».

📁 Данные для подготовки (создаются студентом перед выполнением)

Структура тестовых данных

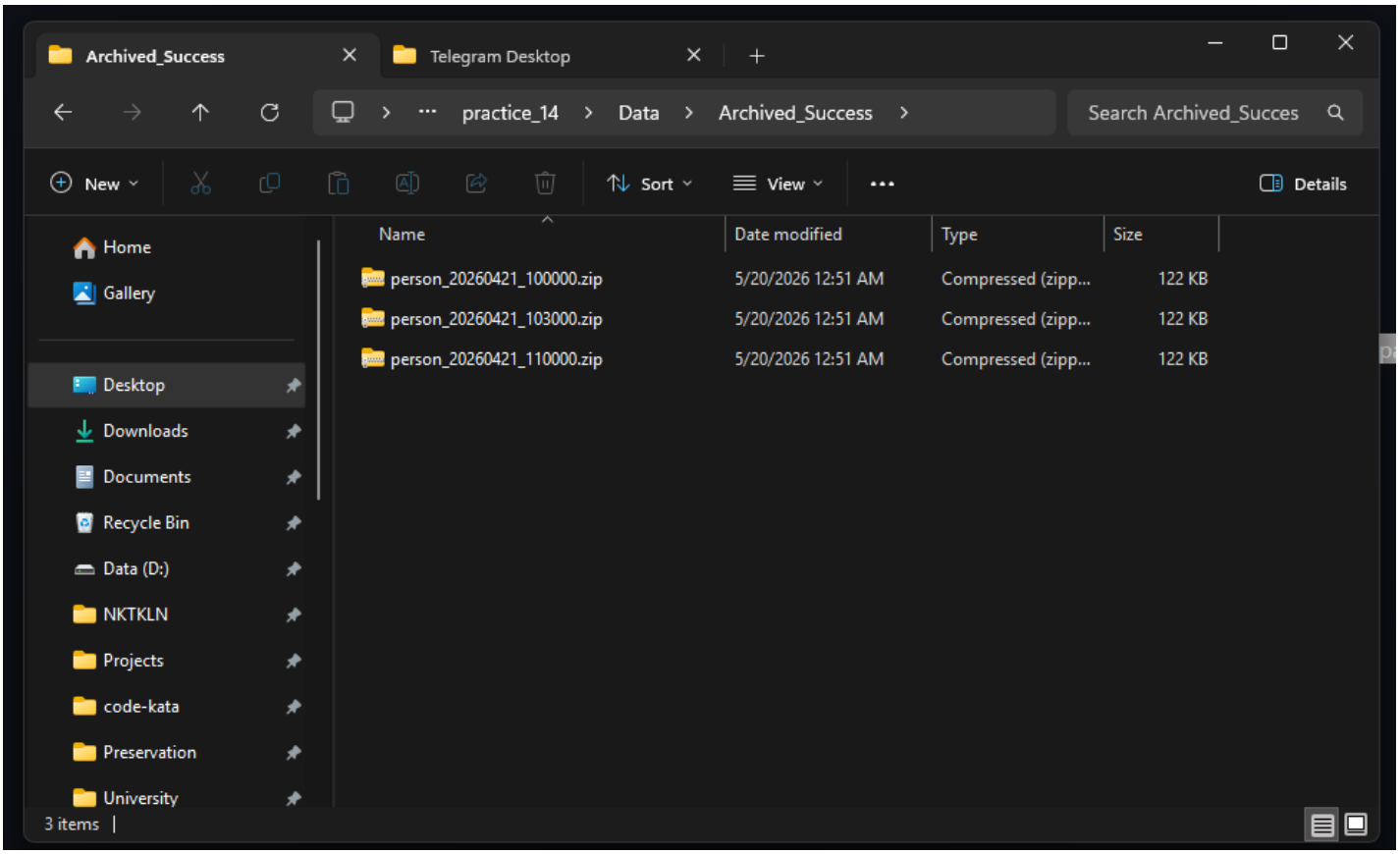
Создать папку C:\Incoming_Persons. Внутри неё создать **три ZIP-архива** с именами:

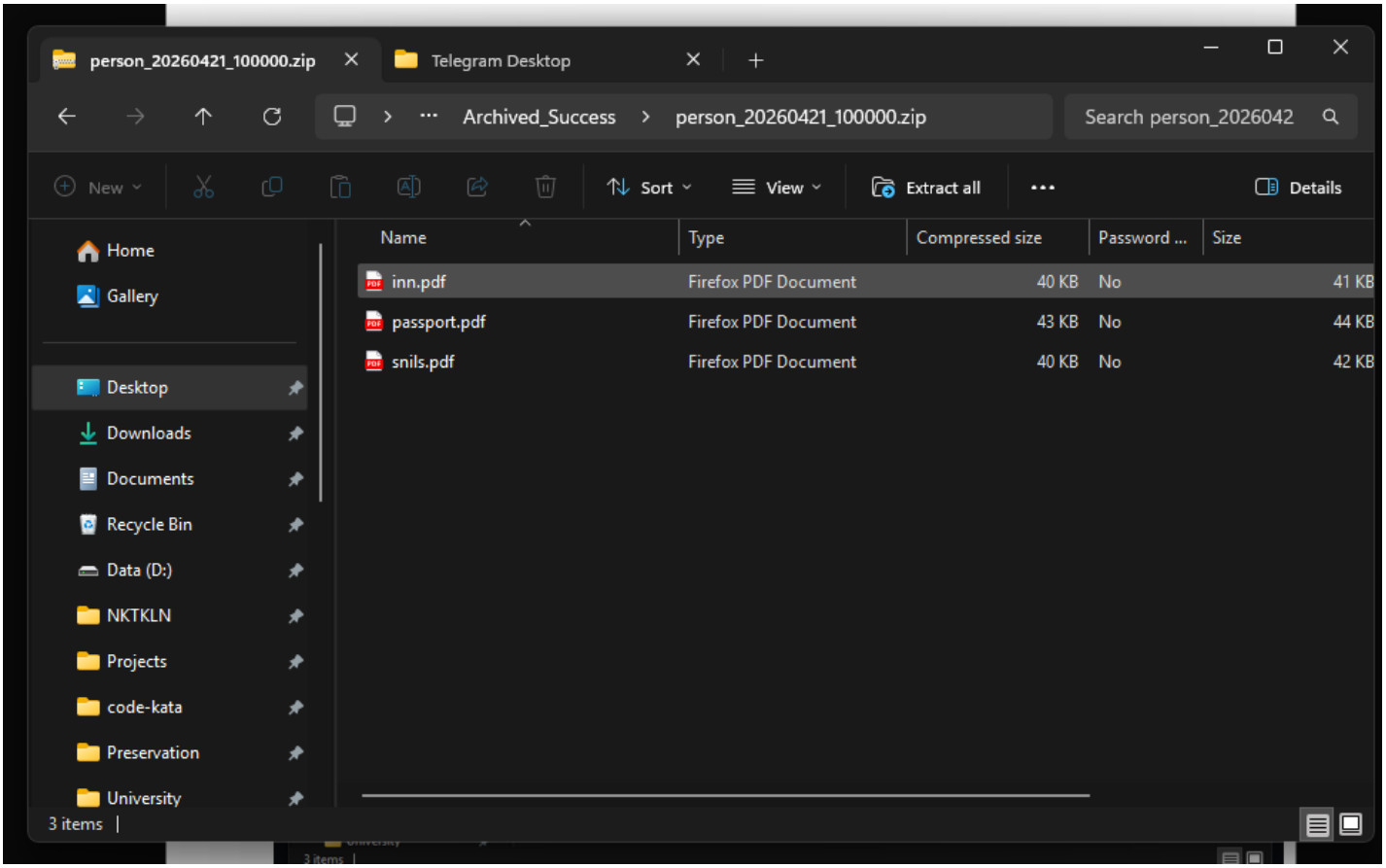
1. person_20260421_100000.zip
2. person_20260421_103000.zip
3. person_20260421_110000.zip

Каждый архив содержит три PDF-файла, созданных по инструкции ниже.

Инструкция по созданию PDF (текстовый слой)

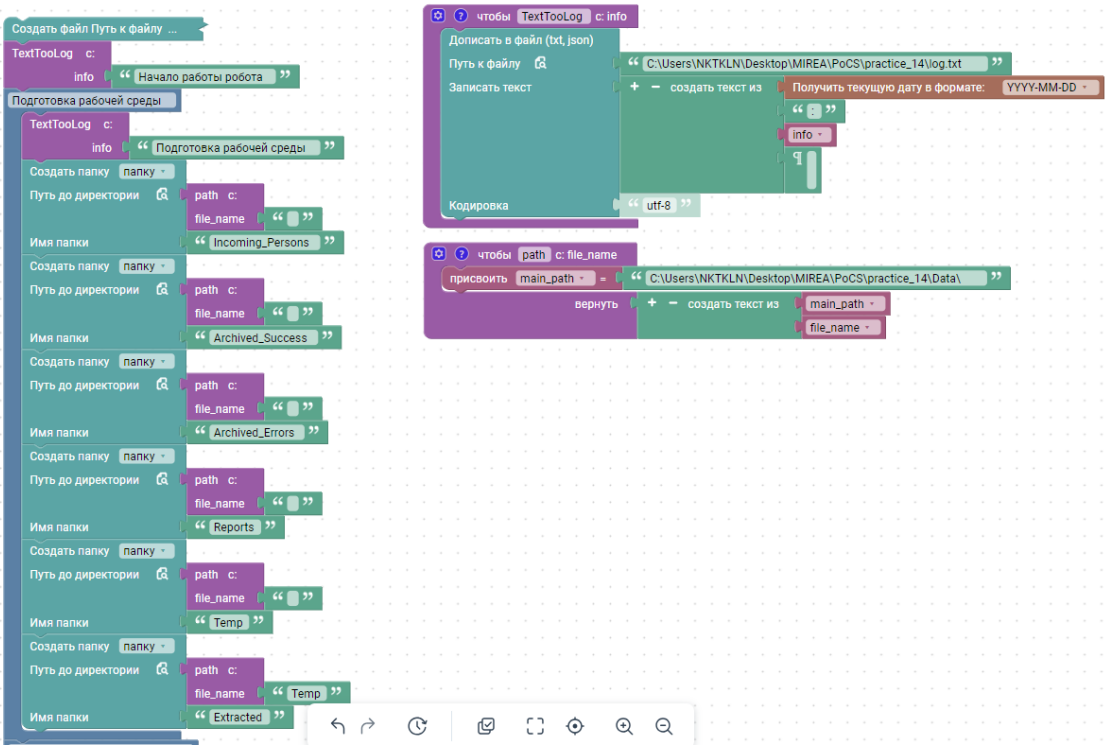
1. **Создайте текстовые файлы в Word (или Google Docs, LibreOffice):**
   - Для passport.pdf напишите:
     Фамилия: Козлов; Имя: Дмитрий; Отчество: Владимирович; Серия: 4503; Номер: 789012
   - Для inn.pdf напишите:
     ИНН: 7725397028
   - Для snils.pdf напишите:
     СНИЛС: 111-222-333-44
2. **Сохраните как PDF с текстовым слоем:**
   В Microsoft Word: Файл → Сохранить как → PDF → Оптимизировать для стандартного (не минимальный размер).
   Убедитесь, что в полученном PDF можно выделить текст мышью.
3. **Повторите для трёх заявок, изменив данные:**
   - **Заявка 2** (некорректный СНИЛС):
     snils.pdf – СНИЛС: 111-222-333-45 (ошибка в контрольной сумме)
   - **Заявка 3** (корректный СНИЛС, сплошной формат):
     snils.pdf – СНИЛС: 98765432101 (сплошной формат, контрольная сумма верна)
4. **Заархивировать каждую заявку:**
   - Создать отдельную папку для заявки, положить туда три PDF.
   - Заархивировать папку (ПКМ → Отправить → Сжатая ZIP-папка).
   - Переименовать ZIP-архив по шаблону и положить в C:\Incoming_Persons.

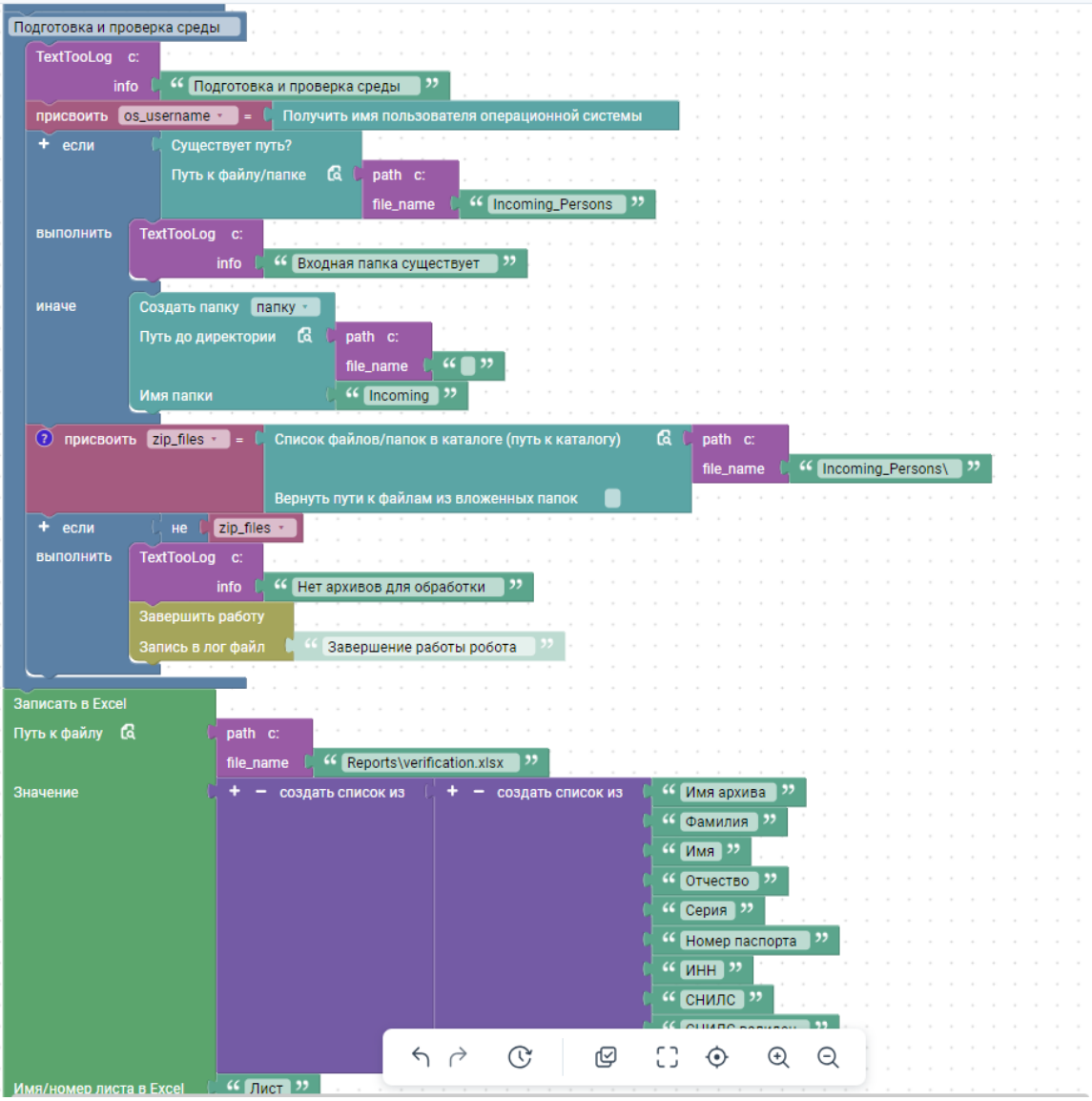

Archived_Success
Telegram Desktop
practice_14 › Data › Archived_Success
Search Archived_Succes
New
Sort
View
Details
Home
Gallery
Desktop
Downloads
Documents
Recycle Bin
Data (D:)
NKTKLN
Projects
code-kata
Preservation
University
Name
Date modified
Type
Size
person_20260421_100000.zip 5/20/2026 12:51 AM Compressed (zipp... 122 KB
person_20260421_103000.zip 5/20/2026 12:51 AM Compressed (zipp... 122 KB
person_20260421_110000.zip 5/20/2026 12:51 AM Compressed (zipp... 122 KB
3 items

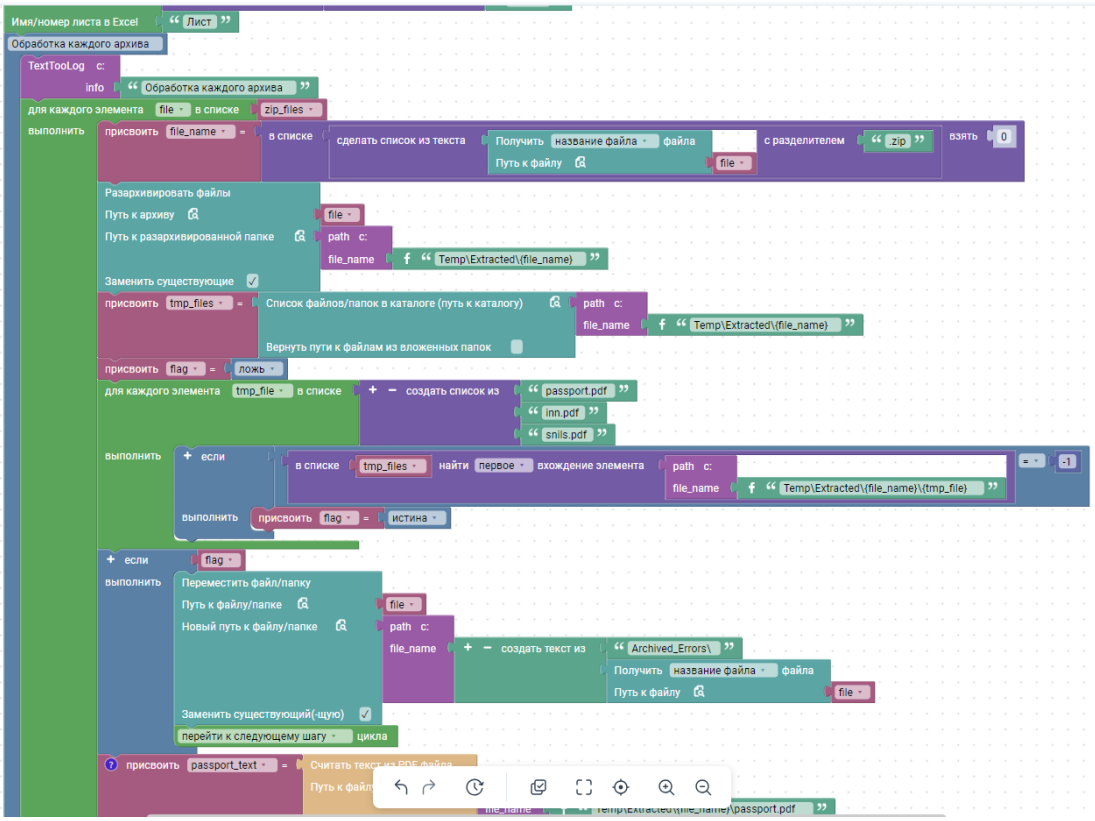

person_20260421_100000.zip
Telegram Desktop
Archived_Success › person_20260421_100000.zip
Search person_2026042
New
Sort
View
Extract all
Details
Home
Gallery
Desktop
Downloads
Documents
Recycle Bin
Data (D:)
NKTKLN
Projects
code-kata
Preservation
University
Name
Type
Compressed size
Password ...
Size
inn.pdf Firefox PDF Document 40 KB No 41 KB
passport.pdf Firefox PDF Document 43 KB No 44 KB
snils.pdf Firefox PDF Document 40 KB No 42 KB
3 items

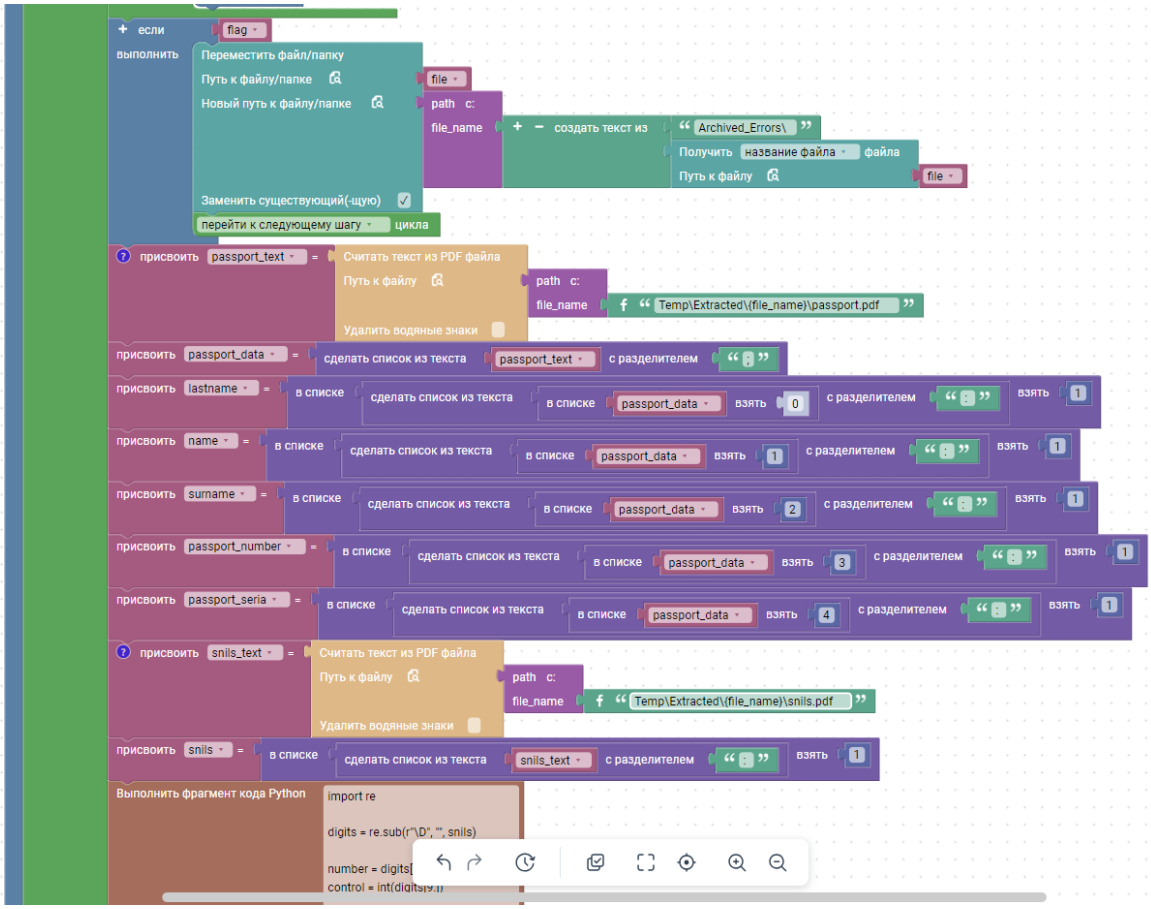
A1 | Имя архива

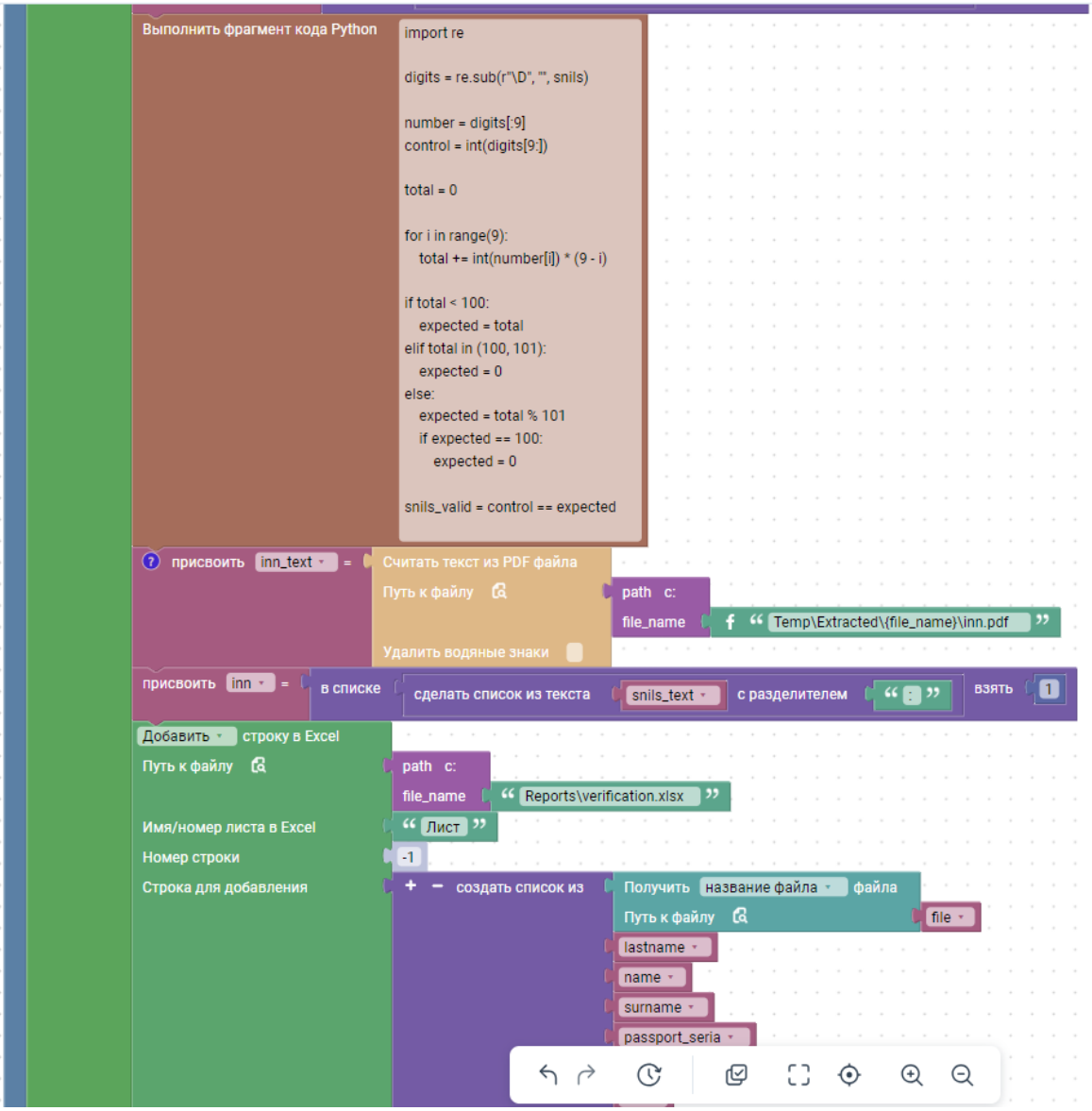
	A	B	C	D	E	F	G	H	I	J	K
1	Имя архи	Фамилия	Имя	Отчество	Серия	Номер па	ИНН	СНИЛС	СНИЛС ва	Статус	
2	person_2(	Иванов	Алексей	Сергееви	567891	4510	112-233-4	112-233-4	TRUE	Успех	
3	person_2(	Петрова	Марина	Викторов	123456	4608	111-222-3	111-222-3	FALSE	Ошибка	
4	person_2(	Смирнов	Дмитрий	Андрееви	778899	4507	987654321	987654321	FALSE	Ошибка	
5											
6											
7											
8											

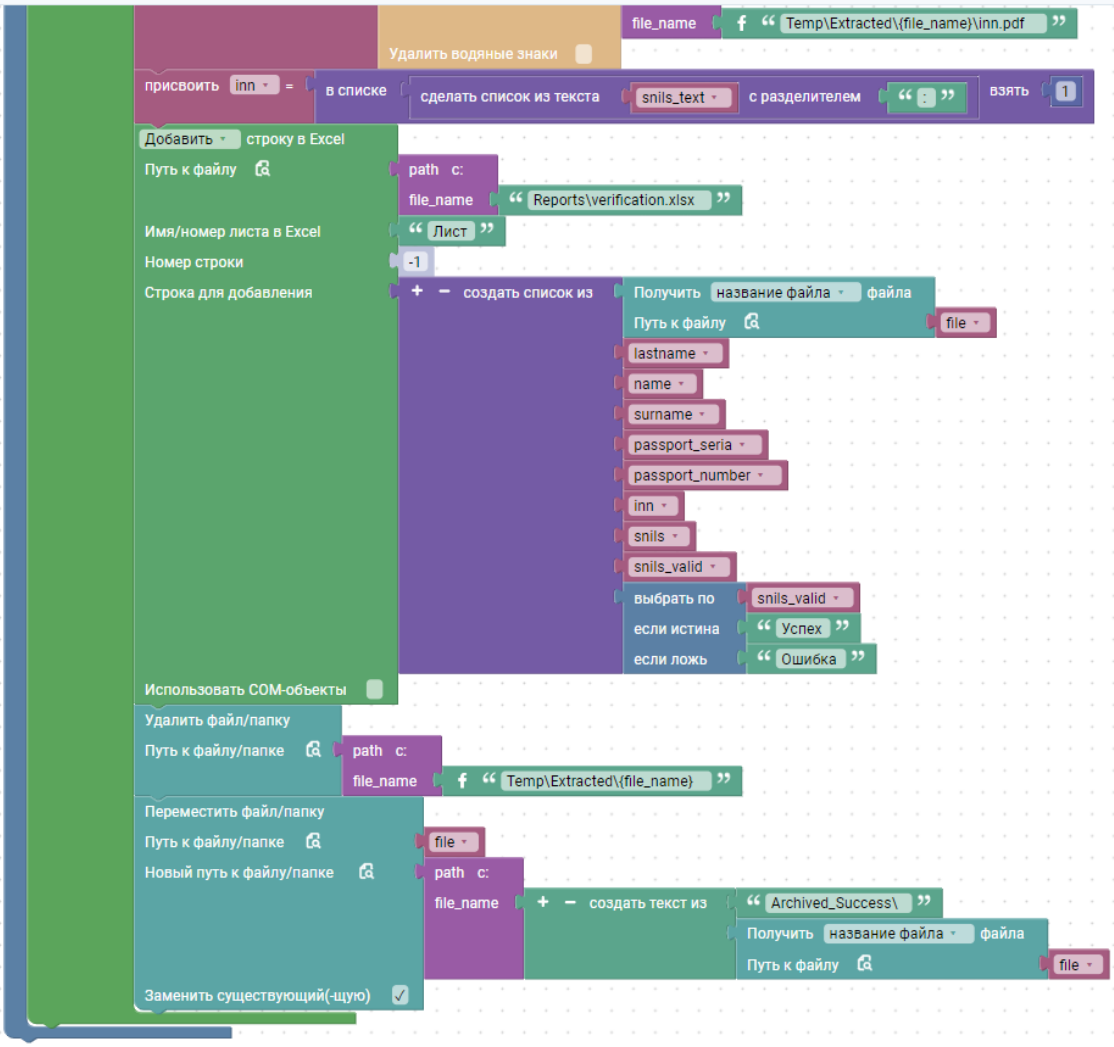



Подготовка и проверка среды
TextTooLog с:
info
Подготовка и проверка среды
присвоить os_username =
Получить имя пользователя операционной системы
если
Существует путь?
Путь к файлу/папке
path с:
file_name
Incoming_Persons
выполнить
TextTooLog с:
info
Входная папка существует
иначе
Создать папку папку
Путь до директории
path с:
file_name
Имя папки
Incoming
присвоить zip_files =
Список файлов/папок в каталоге (путь к каталогу)
path с:
file_name
Incoming_Persons\
Вернуть пути к файлам из вложенных папок
если
не
zip_files
выполнить
TextTooLog с:
info
Нет архивов для обработки
Завершить работу
Запись в лог файл
Завершение работы робота
Записать в Excel
Путь к файлу
path с:
file_name
Reports\verification.xlsx
Значение
создать список из
создать список из
Имя архива
Фамилия
Имя
Отчество
Серия
Номер паспорта
ИНН
СНИЛС
Имя/номер листа в Excel
Лист


Имя/номер листа в Excel
Лист
Обработка каждого архива
TextTooLog с:
info
Обработка каждого архива
для каждого элемента file в списке zip_files
выполнить
присвоить file_name =
в списке
сделать список из текста
Получить название файла файла
Путь к файлу
file
с разделителем
zip
взять
0
Разархивировать файлы
Путь к архиву
file
Путь к разархивированной папке
path с:
file_name
Temp\Extracted\{file_name}
Заменить существующие
присвоить tmp_files =
Список файлов/папок в каталоге (путь к каталогу)
path с:
file_name
Temp\Extracted\{file_name}
Вернуть пути к файлам из вложенных папок
присвоить flag = ложь
для каждого элемента tmp_file в списке
создать список из
passport.pdf
inn.pdf
snils.pdf
выполнить
если
в списке tmp_files найти первое вхождение элемента
path с:
file_name
Temp\Extracted\{file_name}\{tmp_file}
= -1
выполнить
присвоить flag = истина
если
flag
выполнить
Переместить файл/папку
Путь к файлу/папке
file
Новый путь к файлу/папке
path с:
file_name
создать текст из
Archived_Errors\
Получить название файла файла
Путь к файлу
file
Заменить существующий(-щую)
перейти к следующему шагу цикла
присвоить passport_text =
Считать текст из PDF-файла
Путь к файл
Temp\Extracted\{file_name}\passport.pdf


если
flag
выполнить
Переместить файл/папку
Путь к файлу/папке
file
Новый путь к файлу/папке
path c:
file_name
создать текст из
Archived_Errors\
Получить название файла файла
Путь к файлу
file
Заменить существующий(-щую)
перейти к следующему шагу цикла
присвоить passport_text =
Считать текст из PDF файла
Путь к файлу
path c:
file_name
Temp\Extracted\{file_name}\passport.pdf
Удалить водяные знаки
присвоить passport_data =
сделать список из текста
passport_text
с разделителем
:
присвоить lastname =
в списке
сделать список из текста
в списке
passport_data
взять
0
с разделителем
:
взять
1
присвоить name =
в списке
сделать список из текста
в списке
passport_data
взять
1
с разделителем
:
взять
1
присвоить surname =
в списке
сделать список из текста
в списке
passport_data
взять
2
с разделителем
:
взять
1
присвоить passport_number =
в списке
сделать список из текста
в списке
passport_data
взять
3
с разделителем
:
взять
1
присвоить passport_seria =
в списке
сделать список из текста
в списке
passport_data
взять
4
с разделителем
:
взять
1
присвоить snils_text =
Считать текст из PDF файла
Путь к файлу
path c:
file_name
Temp\Extracted\{file_name}\snils.pdf
Удалить водяные знаки
присвоить snils =
в списке
сделать список из текста
snils_text
с разделителем
:
взять
1
Выполнить фрагмент кода Python
import re
digits = re.sub(r"\D", "", snils)
number = digits[
control = int(digits[9:])


Выполнить фрагмент кода Python
import re
digits = re.sub(r"\D", "", snils)
number = digits[:9]
control = int(digits[9:])
total = 0
for i in range(9):
total += int(number[i]) * (9 - i)
if total < 100:
expected = total
elif total in (100, 101):
expected = 0
else:
expected = total % 101
if expected == 100:
expected = 0
snils_valid = control == expected
присвоить inn_text =
Считать текст из PDF файла
Путь к файлу
path c:
file_name
Temp\Extracted\{file_name}\inn.pdf
Удалить водяные знаки
присвоить inn =
в списке
сделать список из текста
snils_text
с разделителем
:
взять
1
Добавить
строку в Excel
Путь к файлу
path c:
file_name
Reports\verification.xlsx
Имя/номер листа в Excel
Лист
Номер строки
-1
Строка для добавления
создать список из
Получить
название файла
файла
Путь к файлу
file
lastname
name
surname
passport_seria



DeepSeek

https://chat.deepseek.com/share/rz5auuw4fkf6519k0j

GitHub https://github.com/NKTKLN/mirea-puzzle-rpa/tree/main/practice_14